



TRƯỜNG ĐẠI HỌC FPT

PHARMACY CHAIN MANAGEMENT SYSTEM

CAPSTONE PROJECT REPORT

Report 4 - Software Design Document

Group 6 - SE1926-JS(IT)

Members: Tran Trung Hieu, Pham Vu Dinh Truong, Bui Duc Luong, Nguyen Duc Han, Nguyen Hoang Hung

- Hanoi, June 2026 -

Table of Contents

Table of Contents	2
I. Record of Changes	5
II. Software Design Document	6
1. System Design	6
1.1 System Architecture	6
Architectural Communication Rules	7
1.2 Package Diagram	9
Package Dependency Constraints	10
2. Database Design	11
2.1 Database Design Principles	11
2.2 File Descriptions	11
2.3 Database Table Relationships	11
2.4 Table Descriptions	12
Database Integrity Rules	13
3. Detailed Design	15
Detailed Design Rules Applied	15
3.1 Authentication Management	15
3.1.1 Class Diagram	16
3.1.2 Class Specifications	16
3.1.3 Login Sequence Diagram	17
3.2 Founder Detail Design	17
3.2.1 UC02- Create Business Admin Account	18
3.2.1.1 Class Diagram	18
3.2.1.2 Sequence Diagram	22
3.2.2 UC27- Update Business Admin Account Profile	23
3.2.2.1 Class Diagram	23
3.2.2.2 Sequence Diagram	26
3.2.3 UC28- View Business Admin Account Details	27
3.2.3.1 Class Diagram	27
3.2.3.2 Sequence Diagram	30
3.2.4 UC29- Deactivate Business Admin Account	31
3.2.4.1 Class Diagram	31
3.2.4.2 Sequence Diagram	34
3.2.5 UC23- Generate Revenue Reports	35
3.2.5.1 Class Diagram	35
3.2.5.2 Sequence Diagram	38
3.2.6 UC24- Generate Cash Flow Statistics	39
3.2.6.1 Class Diagram	39
3.2.6.2 Sequence Diagram	42
3.2.7 UC25- Export Financial Reports	43
3.2.7.1 Class Diagram	43
3.2.7.2 Sequence Diagram	46
3.3 Business Admin Detail Design	46
3.3.1 UC034- View Business Analysis Report	47

3.3.1.1 Class Diagram	47
3.3.1.2 Sequence Diagram	48
3.3.2 UC06- View Business Medicine Statistics	49
3.3.2.1 Class Diagram	49
3.3.2.2 Sequence Diagram	51
3.3 UC03- View Branch Management	52
3.3.1 Class Diagram	52
3.3.2 Sequence Diagram	53
3.4 Branch Manager Design Details	54
3.4.1 UC09 - View Branch DashBoard	54
3.4.1.1 Class Diagram	54
3.4.1.2 Sequence Diagram	55
3.5 Inventory Management	56
3.5.1. UC07 Receive Goods from Vendor	56
3.5.1.1 Class Diagram	56
3.5.1.2 Class Specifications	56
3.5.1.3 Approve Internal Transfers Sequence Diagram	58
3.5.2. UC08 QC Inspection	59
3.5.2.1 Class Diagram	59
3.5.2.2 Class Specifications	59
3.5.2.3 QC Inspection Sequence Diagram	60
3.5.3. UC11 Issue Stock to Stores	61
3.5.3.1 Class Diagram	61
3.5.3.2 Class Specifications	61
3.5.3.3 Issue Stock to Stores Sequence Diagram	62
3.5.4. UC12 Approve Internal Transfers	63
3.5.4.1 Class Diagram	63
3.5.4.2 Class Specifications	63
3.5.4.3 Approve Internal Transfers Sequence Diagram	64
3.5.5. UC13 View Inventory Reports	65
3.5.5.1 Class Diagram	65
3.5.5.2 Class Specifications	65
3.5.5.3 View Inventory Reports Sequence Diagram	66
3.5.6. UC14 Perform Stocktake	67
3.5.6.1 Class Diagram	67
3.5.6.2 Class Specifications	67
3.5.6.3 Perform Stocktake Sequence Diagram	68
3.5.7. UC19 Set Safety Stock Levels	69
3.5.7.1 Class Diagram	69
3.5.7.2 Class Specifications	69
3.5.7.3 Set Safety Stock Levels Sequence Diagram	71
3.5.8. UC30 Identify Expired/Damaged Stock	72
3.5.8.1 Class Diagram	72
3.5.8.2 Class Specifications	72

3.5.8.3 Identify Expired/Damaged Stock Sequence Diagram	74
3.5.9. UC31 Track Batch & Lot Number	75
3.5.9.1 Class Diagram	75
3.5.9.2 Class Specifications	75
3.5.9.3 Identify Expired/Damaged Stock Sequence Diagram	76
3.4.1 Class Diagram	77
3.4.2 Class Specifications	77
3.4.3 Approve Internal Transfer Sequence Diagram	78
3.4.4 Design Notes	78
3.6 Sales and Payment Management	78
3.6.1. Generate Invoice	79
3.6.1.1 Class Diagram	79
3.6.1.2 Class Specifications	79
3.6.1.3 Generate Invoice Sequence Diagram	80
3.6.2. Create Payment Transaction	81
3.6.2.1 Class Diagram	81
3.6.2.2 Class Specifications	81
3.6.2.3 Create Payment Transaction Sequence Diagram	82
3.6.3. View Generated Invoices	83
3.6.3.1 Class Diagram	83
3.6.2.2 Class Specifications	83
3.6.2.3 View Generated Invoices Sequence Diagram	84
3.6.4. View Payment Transaction	85
3.6.4.1 Class Diagram	85
3.6.4.2 Class Specifications	85
3.6.4.3 View Payment Transaction Sequence Diagram	86
3.6.5. View Medicine Information	87
3.6.5.1 Class Diagram	87
3.6.5.2 Class Specifications	87
3.6.5.3 View Medicine Information Sequence Diagram	88

I. Record of Changes

Date	A*, M, D	In charge	Change Description
17/06/2026	A	Group 6	Create initial SDD document according to Report 4 template.
17/06/2026	M	Group 6	Revise architecture according to COMET standards: Multiple Client/Multiple Service, strict layering, proxy objects, and interface-driven communication.
17/06/2026	M	Group 6	Revise package design to reduce shotgun surgery, enforce ADP, DTO boundaries, and no upward / skip-layer / circular dependencies.
17/06/2026	M	Group 6	Revise database design to apply 3NF, UUID keys for core entities, UPPER_SNAKE_CASE table names, soft-delete, state management, and full AUDIT LOG structure.
17/06/2026	M	Group 6	Revise detailed design with COMET stereotypes, interface specifications, private entity attributes, dependency injection, and sequence diagrams with alt frames and traceability.

*A - Added M - Modified D - Deleted

II. Software Design Document

1. System Design

1.1 System Architecture

The Pharmacy Chain Management System (PCMS) is designed as a centralized web-based system for a multi-branch pharmacy chain. The revised design applies the Multiple Client/Multiple Service Architectural Pattern together with Layers of Abstraction. Browser clients for each role communicate with service subsystems through a single API boundary, while the server side separates security middleware, coordinators, business services, repository interfaces, database wrapper classes, and external proxy objects.

The architecture follows strict layering: the Web Client may call only API routes; API routes and middleware may call only controller/service interfaces; services may call repository interfaces or external proxy interfaces; repositories hide all SQL/ORM details and are the only components allowed to access the physical database. External systems such as payment, supplier, Google authentication, Google Maps, and email are never called directly from controllers or repositories.

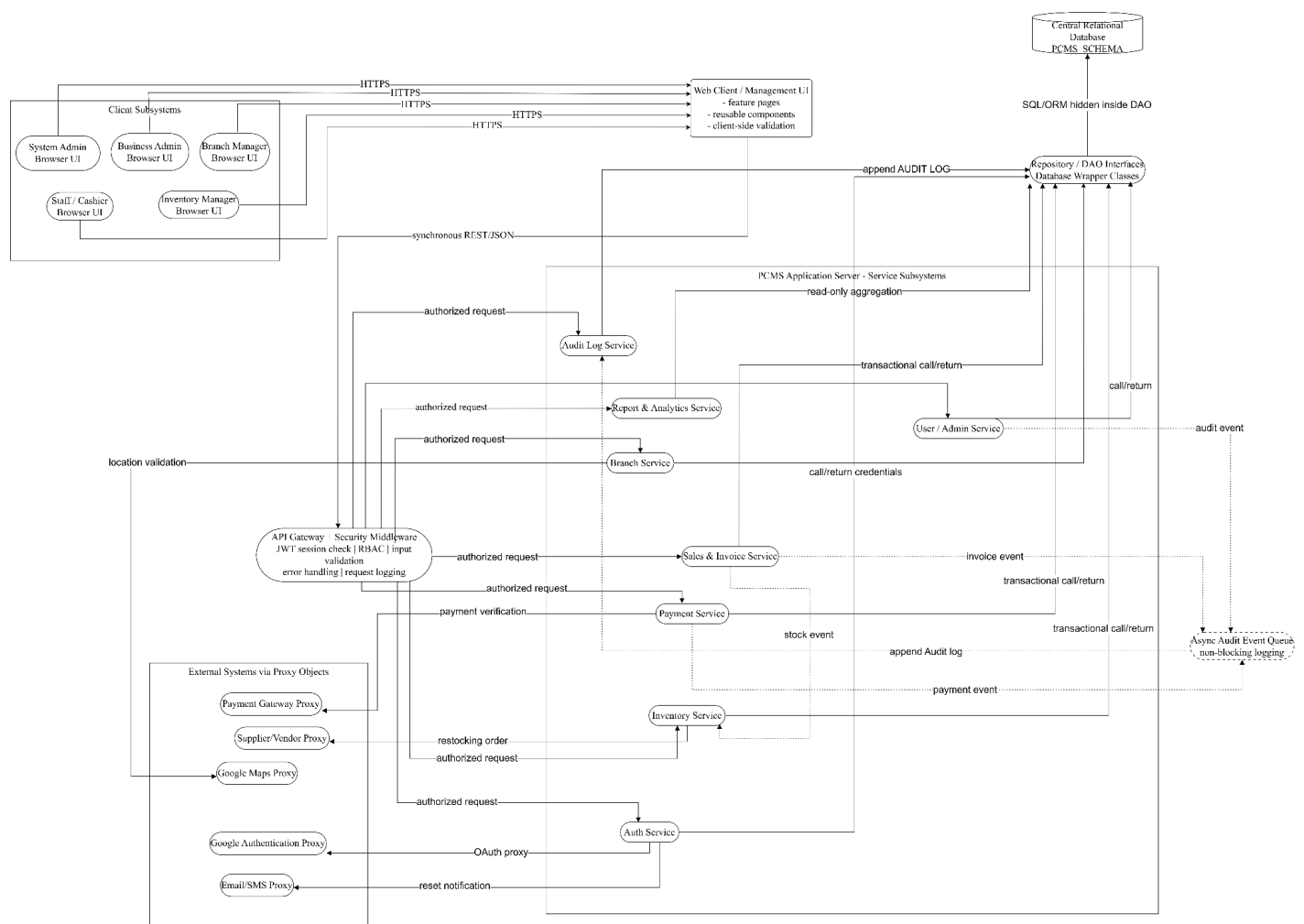


Figure 1-1: PCMS System Architecture

No	Component	Design Responsibility
----	-----------	-----------------------

01	Internal Users	Founder, Business Admin, Branch Manager, Inventory Manager, and Staff. Each role interacts with the system through role-specific access rights.
02	Flutter Client / Management UI	Developed using Flutter. Provides role-specific screens, widget components, and client-side state management (e.g., Bloc/Provider). It communicates with the backend exclusively via DTO-based REST HTTP requests.
03	API Gateway & Security Middleware	The ASP.NET Core entry point. It authenticates JWT tokens, enforces Role-Based Access Control (RBAC), validates incoming DTOs, handles global exceptions, and routes requests to the appropriate controllers.
04	ASP.NET Service Subsystems	Core backend services including Auth, User/Admin, Branch, Inventory, Sales/Invoice, Payment, Report, and Audit. These services encapsulate all business rules and coordinate transactions.
05	Repository / DAO Layer	Data access layer (e.g., using Entity Framework Core). It acts as database wrapper classes, hiding all SQL/ORM complexities and providing clean, object-oriented repository interfaces to the business services.
06	Central Relational Database	The physical SQL database stores normalized PCMS data: users, roles, branches, medicines, inventory, invoices, payments, and audit logs.
07	External Proxy Objects	PaymentGatewayProxy, SupplierProxy, GoogleAuthProxy, GoogleMapsProxy, and EmailProxy. These objects encapsulate all external API communication details, protecting the core services from infrastructure changes.
08	Async Audit/Event Queue	A non-blocking event queue or message broker used to write audit logs asynchronously, ensuring that core business transactions are not delayed by logging operations.

Architectural Communication Rules

- **Client-to-API Communication:** Uses synchronous REST/JSON over HTTPS. This pattern is applied when the user must immediately receive a result, such as login, stock lookup, invoice generation, or payment verification.

- Service-to-Repository Communication: Follows the synchronous call/return pattern through repository interfaces. Services never expose SQL queries, ORM sessions, or raw database rows to the controllers.
- Asynchronous Communication: Audit logging and non-critical operational events utilize an asynchronous message queue to avoid blocking the main business transactions.
- Proxy Communication: External systems (payment gateway, supplier system, email service, Google Authentication, and Google Maps) are accessed only through dedicated proxy objects. They are not directly called by controllers or repositories.
- Strict Layering: Bypassing an architectural layer is strictly forbidden. A controller must call a service, not a repository. A repository must only interact with the database and must not depend on a service or controller.

1.2 Package Diagram

The package design is revised from a purely horizontal layer decomposition into a feature-cohesive design with explicit interface boundaries. This reduces the Shotgun Surgery risk when business rules change. For example, a future Discount rule for Product/Medicine is localized in the sales_payment and inventory packages plus their DTOs and repository interfaces, instead of leaking into unrelated packages.

The design still keeps reusable infrastructure packages where the Common Reuse Principle applies: shared UI components, DTOs, configuration utilities, repository interfaces, and external proxies. All package dependencies form a Directed Acyclic Graph; upward, skip-layer, and circular dependencies are forbidden.

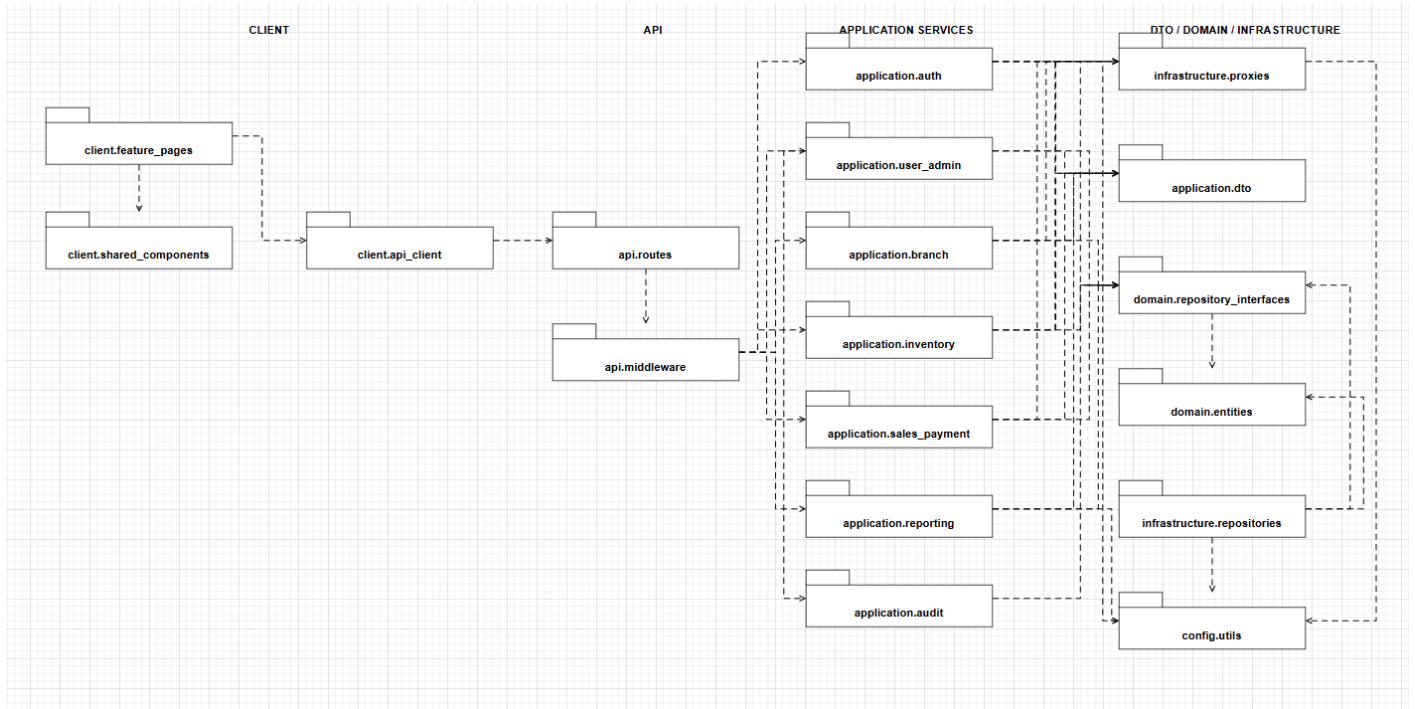


Figure 1-2: PCMS Package Diagram

No	Package	Description	Allowed Dependencies / Rule
01	client.feature_pages	Boundary screens grouped by feature: authentication, admin, branch, inventory, sales/payment, reporting, and profile.	May use client.shared_components and client.api_client only.
02	client.shared_components	Reusable UI components such as Button, Table, Modal, Chart, Filter, FormField, and ErrorMessage.	No dependency on feature pages.
03	client.api_client	Client REST adapter that stores token locally and converts UI input into RequestDTO objects.	May call api.routes only.
04	api.routes	HTTP route definitions and endpoint mapping by feature.	May call api.middleware only.
05	api.middleware	AuthFilter, RBAC, payload validation, error handling, and request logging.	May call application feature controllers/services. Must not call repositories.
06	application.dto	RequestDTO, ResponseDTO, ErrorDTO, and validation contracts crossing API boundaries.	Used by controllers and services; prevents entity leakage.
07	application.auth	Authentication controller, service interface, service implementation, token logic coordination, forgot/reset password workflows.	Uses repository interfaces, security utilities, and email/auth proxies.
08	application.user_admin	Business Admin account management and profile/account status workflows.	Uses IUserRepository, IRoleRepository, and IAuditLogService interfaces.
09	application.branch	Branch management, branch manager authorization, dashboard aggregation, and location validation.	Uses branch/user repository interfaces and GoogleMapsProxy.

10	application.inventory	Receive goods, QC, FEFO, safety stock, stock transfer, expired/damaged stock, and inventory reports.	Uses inventory/batch/transfer repository interfaces and supplier proxy.
11	application.sales_payment	Prescription, invoice generation, invoice details, payment transaction, and gateway verification.	Uses invoice/payment/inventory repository interfaces and PaymentGatewayProxy.
12	application.reporting	Revenue, medicine, inventory, financial, and business analysis report generation and export.	Read-only repository interfaces and export utilities.
13	application.audit	Creates immutable audit records from service events.	May consume async events and append to AUDIT LOG repository.
14	domain.entities	Entity classes mapped to database tables: User, Branch, Medicine, MedicineBatch, Inventory, Invoice, PaymentTransaction, etc.	Must not cross the service-to-controller boundary; mapped to DTO before API output.
15	domain.repository_interfaces	Repository abstractions such as IUserRepository, IInventoryRepository, IInvoiceRepository, and IPaymentRepository.	Implemented by infrastructure.repositories.
16	infrastructure.repositories	DAO/ORM classes that implement repository interfaces and hide SQL/physical data access.	May depend on domain.entities and config only.
17	infrastructure.proxies	Proxy objects for Google, Email, Supplier, and Payment external systems.	Used through interfaces by application services.
18	config.utils	Constants, enumerations, security helpers, time/currency helpers, and application configuration.	Shared low-level package.

Package Dependency Constraints

- No upward dependency: repositories must not import services; services must not import controllers or middleware.
- No skip-layer dependency: controllers and middleware must not call repositories directly.
- No circular dependency: if two services need the same behavior, the shared behavior must be extracted to a separate common service or interface.
- No entity leakage: Entity objects may be used inside services and repositories only. Controllers and API responses must use DTOs.
- Dependency Inversion is required for infrastructure: services depend on interfaces such as IUserRepository and IPaymentGatewayProxy, not concrete classes.

2. Database Design

2.1 Database Design Principles

The database design is revised to comply with normalization, integrity, naming, audit, and soft-delete standards for a healthcare and financial information system. The database is centralized but supports multiple branches and batch-level medicine traceability.

Principle	Application in PCMS
3NF and Integrity	INVOICE is separated from INVOICE_DETAIL, PURCHASE_ORDER from PURCHASE_ORDER_DETAIL, STOCK_TRANSFER from STOCK_TRANSFER_DETAIL, and MEDICINE from MEDICINE_BATCH. This avoids repeating groups and transitive dependencies.
Controlled Denormalization	Derived totals such as invoice total may be stored only when synchronized inside application-level transactions or database triggers. The authoritative line values remain in detail tables.
Primary Keys	Core business entities use UUID/ULID primary keys to reduce IDOR risks. Lookup tables such as ROLE and PAYMENT_GATEWAY may use SMALLINT/TINYINT keys.
Naming Convention	Tables use singular UPPER_SNAKE_CASE. Columns use lower_snake_case. Foreign keys use the suffix _id. Boolean fields start with is_ or has_.
Timestamps	Business tables include created_at and updated_at. Immutable log tables include created_at.
Soft Delete / State Management	Physical DELETE is not used for business data. Records are deactivated by status/state transitions such as ACTIVE, INACTIVE, DELETED, PENDING, COMPLETED, CANCELLED, APPROVED, REJECTED, PAID, or FAILED.
Audit Log	AUDIT_LOG records actor_id, entity_name, entity_id, action, old_value JSON, new_value JSON, and created_at for traceability.

2.2 File Descriptions

No	File / Package	Description
01	src/client/feature_pages	Frontend feature screens for login, profile, admin account, branch, inventory, sales/payment, and reports.
02	src/client/shared_components	Reusable UI components that satisfy the Common Reuse Principle.
03	src/api/routes	REST route mapping by feature. Routes are thin and delegate to middleware/controllers.
04	src/api/middleware	Authentication, authorization, request validation, exception handling, and request logging.
05	src/application/*	Feature-level controllers, service interfaces, and service implementations. Business logic is centralized here.
06	src/application/dto	DTO definitions used at API and service boundaries.
07	src/domain/entities	Entity classes mapped to tables. Attributes are private and state changes are exposed through public operations.
08	src/domain/repository_interfaces	Interfaces that services depend on, enabling Dependency Injection and unit testing.
09	src/infrastructure/repositories	Concrete repository/DAO implementations hiding SQL/ORM details.
10	src/infrastructure/proxies	External system adapters/proxies for payment, supplier, email, Google authentication, and Google Maps.
11	database/schema.sql	Database schema script containing tables, PK/FK constraints, indexes, UUID defaults, and status constraints.
12	database/seed.sql	Initial seed data for roles, default Founder, payment gateway definitions, and controlled code values.

2.3 Database Table Relationships

The database is divided into administration, branch, inventory, stock movement, prescription, sales/payment, reporting, and audit groups. The relationship design follows the Object-Relational Mapping method: entity classes map to tables; 1:M

relationships are implemented with foreign keys on the many side; M:N relationships are resolved through association/detail tables.

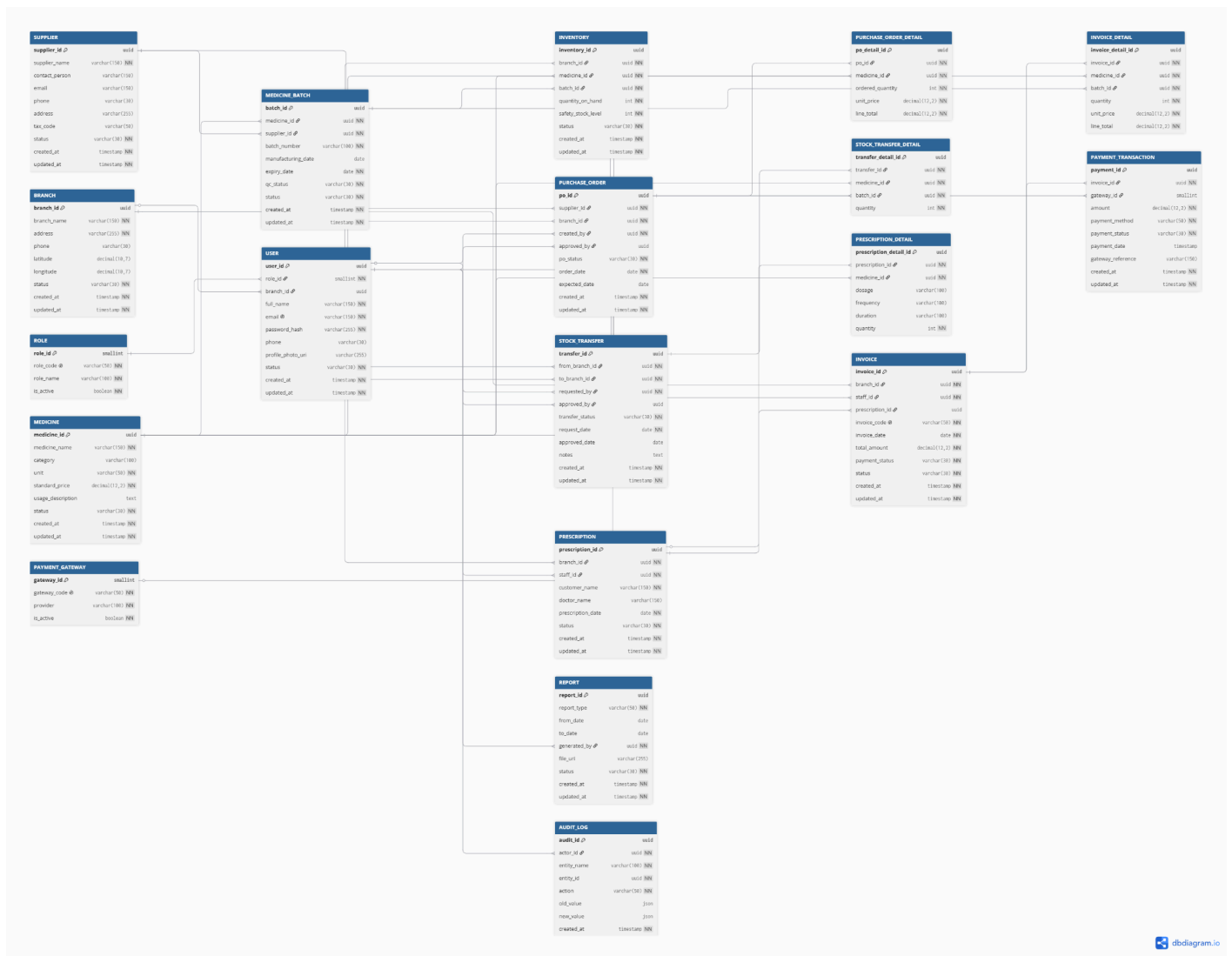


Figure 2-1: Administration, Medicine, Inventory, Purchase Order, and Stock Transfer ERD

Figure 2-2: Prescription, Sales, Payment, Reporting, and Audit ERD

2.4 Table Descriptions

No	Table	Description
01	ROLE	Lookup table for authorization roles. PK: role_id SMALLINT. FK: none. Columns include role_code, role name, is_active.
02	USER	Internal user account data. PK: user_id UUID. FK: role_id -> ROLE(role_id), branch_id -> BRANCH(branch_id). Contains full_name, email, password_hash, phone, profile_photo_uri, status, created_at, updated_at.
03	BRANCH	Pharmacy branch data. PK: branch_id UUID. FK: none. Contains branch_name, address, phone, latitude, longitude, status, created_at, updated_at.
04	SUPPLIER	Supplier/vendor information. PK: supplier_id UUID. FK: none. Contains supplier_name, contact details, tax_code, status, created_at, updated_at.

05	MEDICINE	Medicine master data independent from batch/lot data. PK: medicine_id UUID. FK: none. Contains medicine_name, category, unit, standard_price, usage_description, status, created_at, updated_at.
06	MEDICINE_BATCH	Batch/lot traceability. PK: batch_id UUID. FK: medicine_id -> MEDICINE, supplier_id -> SUPPLIER. Contains batch_number, manufacturing_date, expiry_date, qc_status, status, created_at, updated_at.
07	INVENTORY	Branch-level quantity by medicine batch. PK: inventory_id UUID. FK: branch_id -> BRANCH, medicine_id -> MEDICINE, batch_id -> MEDICINE_BATCH. Contains quantity_on_hand, safety_stock_level, status, created_at, updated_at.
08	PURCHASE_ORDER	Restocking order header. PK: po_id UUID. FK: supplier_id -> SUPPLIER, branch_id -> BRANCH, created_by/approved_by -> USER. Contains po_status, order_date, expected_date, created_at, updated_at.
09	PURCHASE_ORDER_DETAIL	Purchase order line items. PK: po_detail_id UUID. FK: po_id -> PURCHASE_ORDER, medicine_id -> MEDICINE. Contains ordered_quantity, unit_price, line_total.
10	STOCK_TRANSFER	Internal stock movement header. PK: transfer_id UUID. FK: from_branch_id/to_branch_id -> BRANCH, requested_by/approved_by -> USER. Contains transfer_status, request_date, approved_date, notes, created_at, updated_at.
11	STOCK_TRANSFER_DETAIL	Stock transfer line items. PK: transfer_detail_id UUID. FK: transfer_id -> STOCK_TRANSFER, medicine_id -> MEDICINE, batch_id -> MEDICINE_BATCH. Contains quantity.
12	PRESCRIPTION	Prescription header created by staff. PK: prescription_id UUID. FK: branch_id -> BRANCH, staff_id -> USER. Contains customer_name, doctor_name, prescription_date, status, created_at, updated_at.
13	PRESCRIPTION_DETAIL	Prescription medicine details. PK: prescription_detail_id UUID. FK: prescription_id -> PRESCRIPTION, medicine_id -> MEDICINE. Contains dosage, frequency, duration, quantity.
14	INVOICE	Sales invoice header. PK: invoice_id UUID. FK: branch_id -> BRANCH, staff_id -> USER, prescription_id -> PRESCRIPTION nullable. Contains invoice_code, invoice_date, total_amount, payment_status, status, created_at, updated_at.
15	INVOICE_DETAIL	Invoice line items. PK: invoice_detail_id UUID. FK: invoice_id -> INVOICE, medicine_id -> MEDICINE, batch_id -> MEDICINE_BATCH. Contains quantity, unit_price, line_total.
16	PAYMENT_GATEWAY	Payment gateway lookup/configuration. PK: gateway_id SMALLINT. FK: none. Contains gateway_code, provider, is_active.
17	PAYMENT_TRANSACTION	Payment transaction records. PK: payment_id UUID. FK: invoice_id -> INVOICE, gateway_id -> PAYMENT_GATEWAY nullable. Contains amount, payment_method, payment_status, payment_date, gateway_reference, created_at, updated_at.
18	REPORT	Generated report metadata. PK: report_id UUID. FK: generated_by -> USER. Contains report_type, from_date, to_date, file_uri, status, created_at, updated_at.
19	AUDIT_LOG	Immutable audit records. PK: audit_id UUID. FK-style actor_id -> USER(user_id). Contains entity_name, entity_id, action, old_value JSON, new_value JSON, created_at.

Database Integrity Rules

- Every change to inventory quantity, invoice payment status, transfer status, and user status must be executed inside a service-level transaction.
- INVOICE.total_amount is treated as a cached derived value and must equal the sum of INVOICE_DETAIL.line_total at transaction commit.
- MEDICINE does not store total_quantity. Available stock is calculated from INVENTORY records by branch and batch to preserve batch traceability.

- STOCK_TRANSFER does not store duplicated branch names or medicine names. Such values are resolved through BRANCH and MEDICINE foreign keys to avoid transitive dependencies.
- AUDIT_LOG records must never be physically deleted through application functions.

3. Detailed Design

Detailed design follows COMET-style static and dynamic modeling. Each class diagram uses stereotypes to separate boundary, control, entity, interface, data abstraction, and proxy responsibilities. Each sequence diagram realizes SRS use cases with operation names that match the class specifications. Sequence diagrams show repository interfaces rather than physical database lifelines to preserve Information Hiding.

Detailed Design Rules Applied

- Boundary classes exist at client or external-system boundaries and contain no business rules.
- Controller classes coordinate request/response handling and basic DTO validation only.
- Service classes are business logic coordinators and are the only classes allowed to apply business rules.
- Entity attributes are private and can only be changed through public operations such as deactivate(), approve(), markPaid(), or decrease(qty).
- Services receive dependencies through interfaces to support Dependency Injection, loose coupling, and unit testing.
- Sequence diagrams include alternative flows using alt frames, and return messages are named as DTOs or typed values, not narrative sentences.

3.1 Authentication Management

Authentication Management covers Login, Forgot Password, Reset Password, Manage Profile session verification, and role-based dashboard redirection. The SRS identifies Login as a common use case for all internal users and requires access control by role. The design therefore separates LoginPage as a client boundary from server-side AuthController, AuthService, repository interfaces, token service, password hashing strategy, and email proxy.

3.1.1 Class Diagram

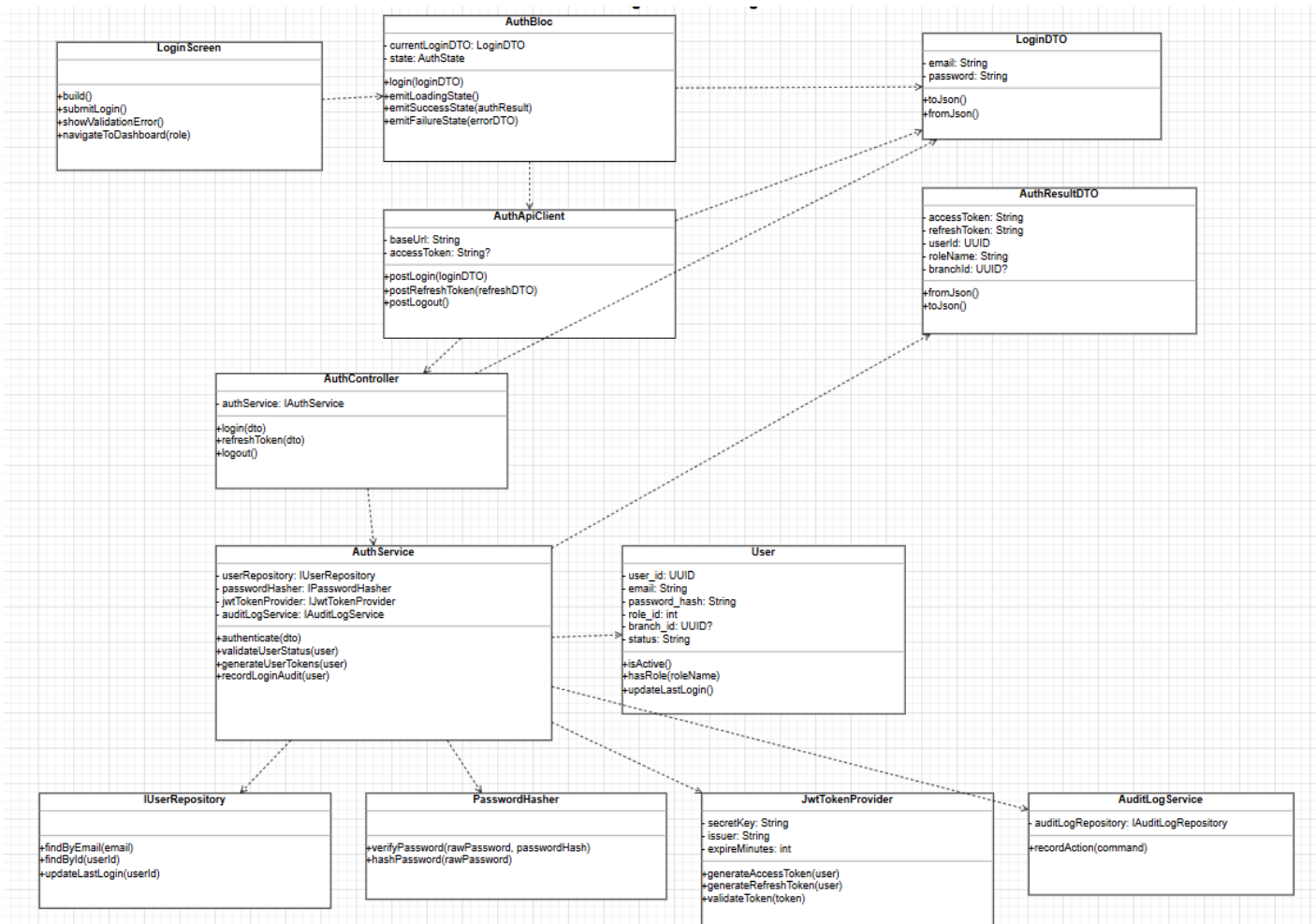


Figure 3-1: Class Diagram - Authentication Management

3.1.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
LoginPage	<<boundary>>	Collects credentials, calls login endpoint, displays errors, and redirects by role.	submitLogin(credentials), redirectByRole(role_code), showError(errorDTO)	LoginRequestDTO, AuthResultDTO
AuthController	<<control>>	Receives authentication API calls, validates request DTOs, calls IAuthService, returns response DTOs.	login(dto), forgotPassword(dto), resetPassword(dto), logout(sessionId)	LoginRequestDTO, AuthResultDTO, ErrorDTO
IAuthService	<<interface>>	Defines authentication service contract for controller usage.	authenticate(dto), requestPasswordReset(dto), resetPassword(dto)	DTO contracts
AuthService	<<control>>	Validates credentials, checks status, maps User entity to DTO, issues token, and coordinates reset password.	authenticate(dto), buildUserDTO(user), ensureActive(user)	User, Role, AuthResultDTO
IUserRepository	<<interface>>	Defines user persistence contract.	findActiveByEmail(email), updatePasswordHash(user_id, hash), save(user)	User
UserRepository	<<entity/data abstraction>>	Implements IUserRepository and hides SQL/ORM access to USER and ROLE tables.	findActiveByEmail(email), save(user)	USER, ROLE
ITokenService	<<interface>>	Defines session token operations.	issueJwt(user_id, role_code), verifySession(token)	SessionDTO
IEmailService	<<interface>>	Defines reset-code notification operations.	sendResetCode(email, code)	External Email Service
IPasswordHashingStrategy	<<interface>>	Allows password hashing algorithm to change without modifying AuthService.	verify(raw, hash), hash(raw)	Password hash
User	<<entity>>	Encapsulates user identity, credentials, account status, and branch/role assignment.	isActive(), changePassword(newHash), deactivate()	USER
Role	<<entity>>	Encapsulates role code and authorization meaning.	hasPermission(permission)	ROLE

3.1.3 Login Sequence Diagram

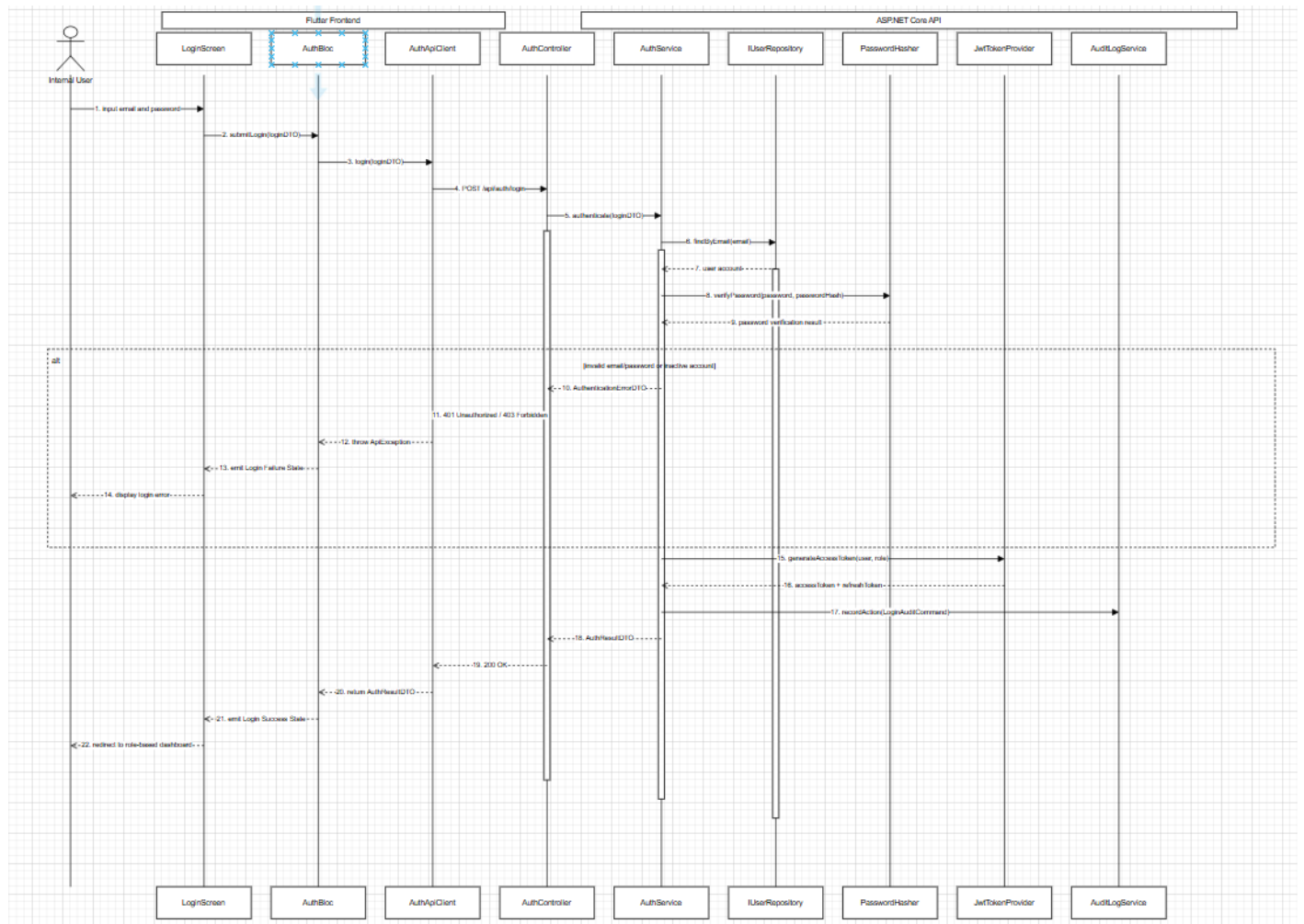


Figure 3-2: Sequence Diagram - Login

Forgot Password and Reset Password reuse the same controller-service-repository pattern. AuthService generates or validates a reset token/code with expiry rules, hashes the new password through IPasswordHashingStrategy, updates password_hash through IUserRepository, and sends notification through IEmailService. The Founder may use the normal login flow, while Forgot Password is restricted to Business Admin, Branch Manager, Inventory Manager, and Staff according to the SRS screen permission rules.

3.2 Founder Detail Design

This feature realizes the Founder workflow for creating, viewing, updating, and deactivating Business Admin accounts. The design emphasizes unique email validation, controlled role assignment, status-based deactivation instead of hard delete, and audit logging.

3.2.1 UC02- Create Business Admin Account

3.2.1.1 Class Diagram

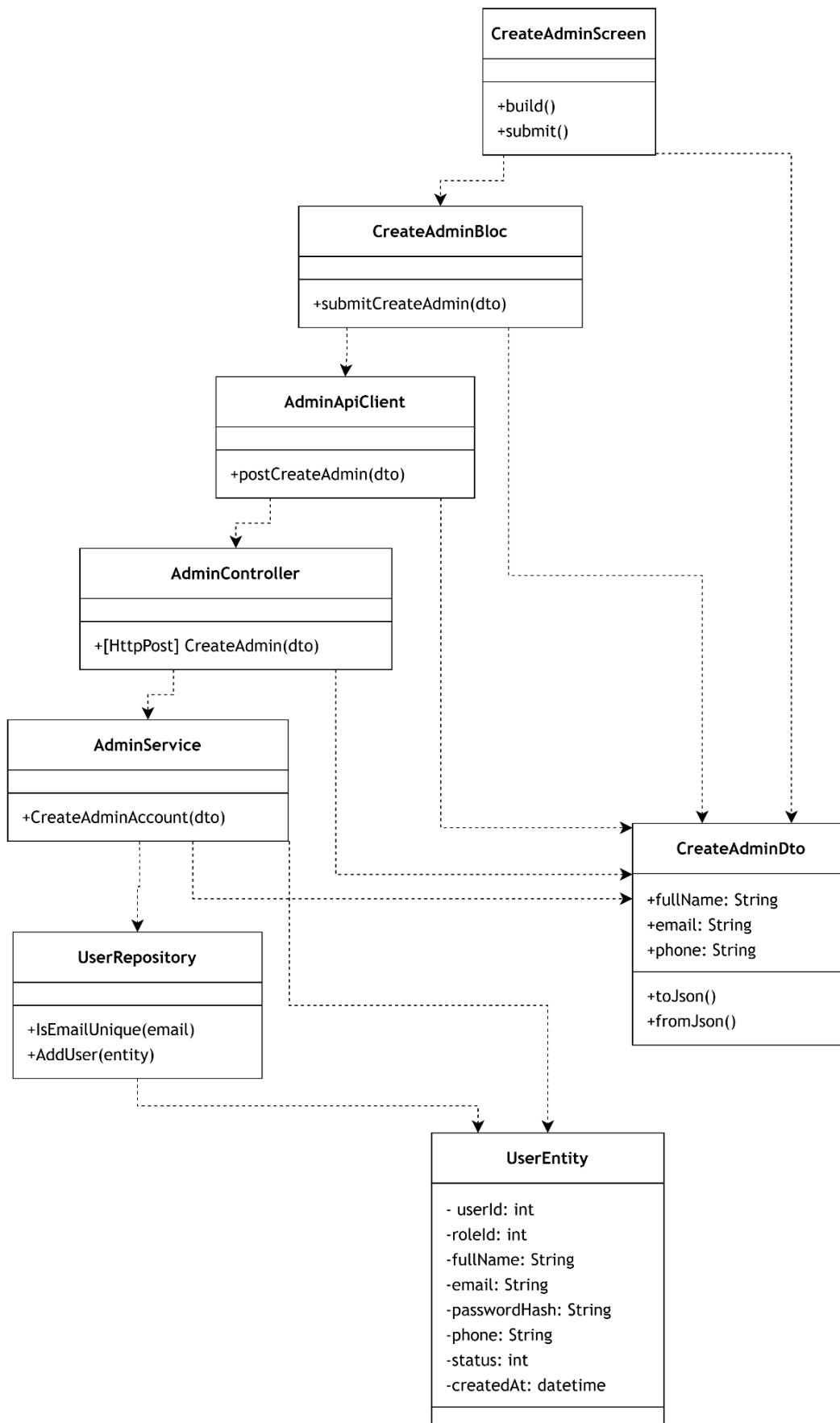


Figure 3.2.1.1: Create Business Admin Account Class Diagram

Class	Type	Main Responsibility	Main Operations	Related Data
CreateAdminScreen	<<boundary>>	Provides the user interface for the Founder to input new Business Admin account details. Hides UI rendering and user event capturing mechanisms.	+build() +submit()	CreateAdminDto
CreateAdminBloc	<<control>>	Acts as a coordinator that orchestrates the state of the account creation screen, captures form submission events, and delegates the payload to the API client.	+submitCreateAdmin(dto)	CreateAdminDto
AdminApiClient	<<boundary>>	Serves as an external system proxy that hides the implementation details of HTTP network communication, transmitting the DTO from the client to the backend.	+postCreateAdmin(dto)	CreateAdminDto
AdminController	<<boundary>>	Acts as the API gateway receiving incoming HTTP requests, hiding endpoint routing details, and forwarding the parsed request payload to the business service layer.	+ [HttpPost] CreateAdmin(dto)	CreateAdminDto
AdminService	<<control>>	Encapsulates the core business logic for creating a Business Admin. Orchestrates the workflow of validating email uniqueness and transforming the DTO into a persistent entity.	+CreateAdminAccount(dto)	CreateAdminDto, UserEntity
UserRepository	<<interface>>	Provides a data access contract that hides the underlying database queries and persistent storage mechanisms for user accounts.	+IsEmailUnique(email) +AddUser(entity)	UserEntity

CreateAdminDto	<<entity>>	A Data Transfer Object that encapsulates the form data (fullName, email, phone) to safely transport state across architectural boundaries without exposing domain models.	+toJson() +fromJson()	N/A
UserEntity	<<entity>>	Represents the persistent data model for a user account in the system. Encapsulates all physical database attributes (ID, role, credentials, and timestamps).		N/A

3.2.1.2 Sequence Diagram

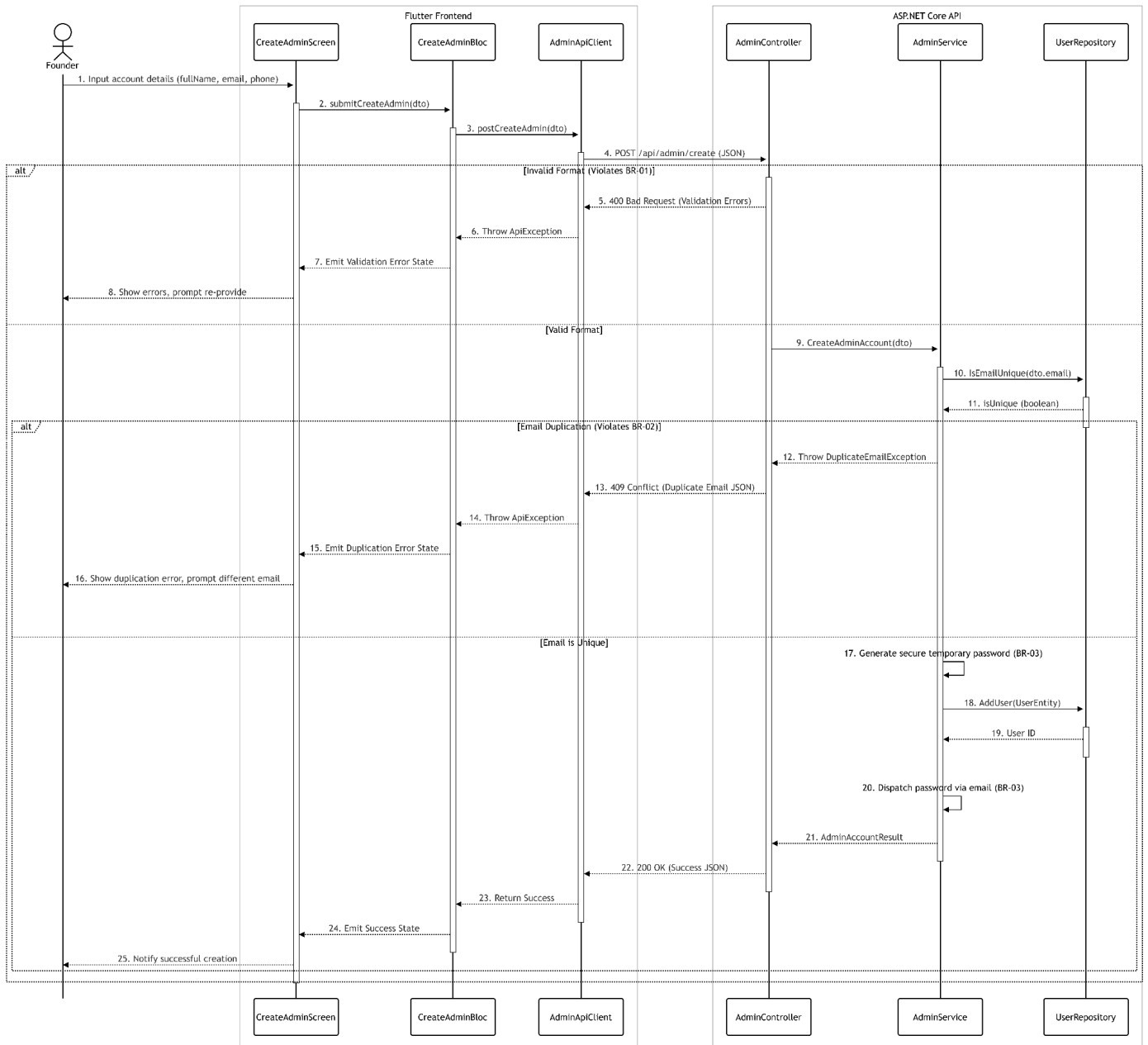


Figure 3-2.1.2: SequenceDiagram - Create Business Admin Account

3.2.2 UC27- Update Business Admin Account Profile

3.2.2.1 Class Diagram

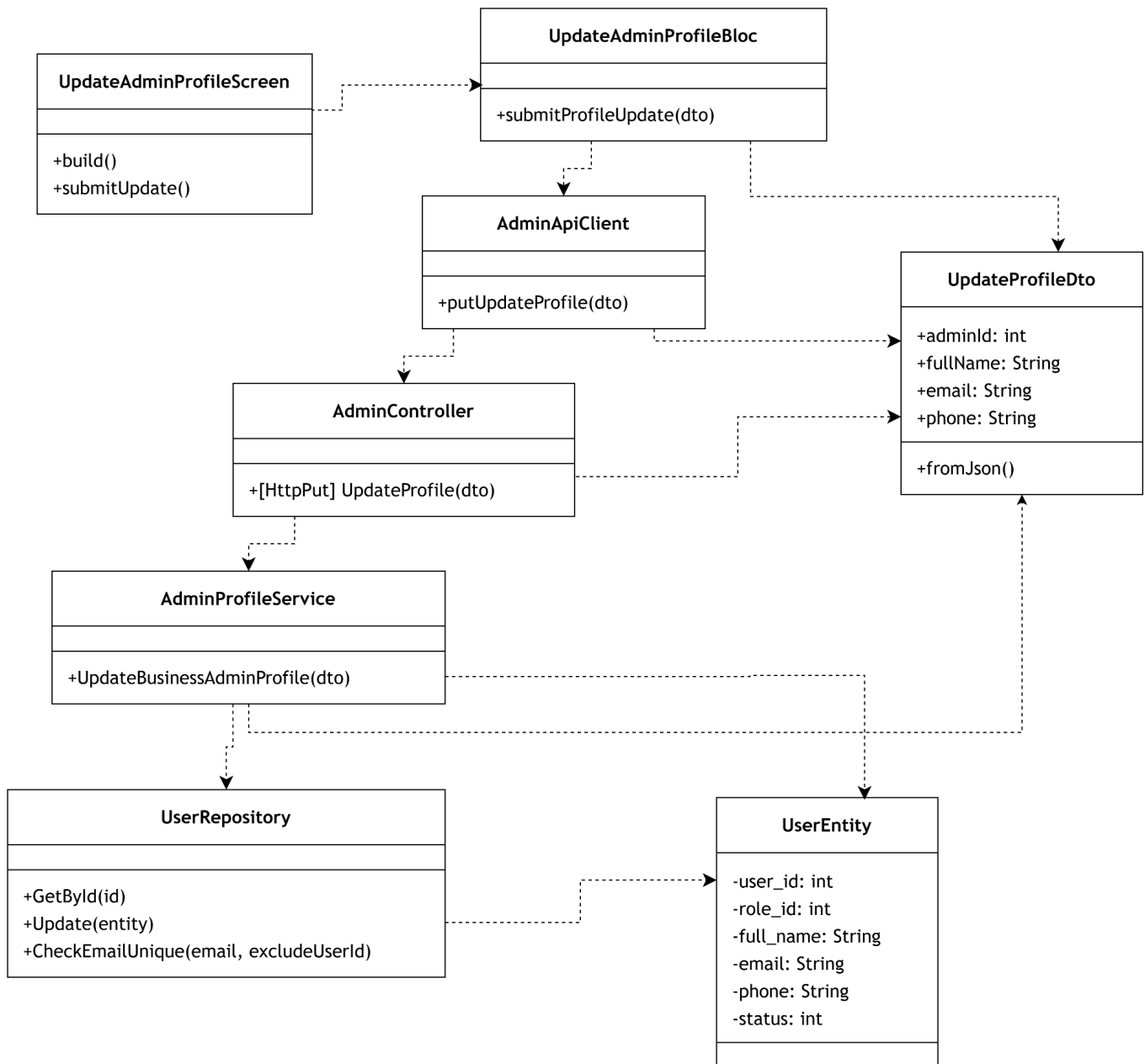


Figure 3.2.2.1: Update Business Admin Account Profile

Class	Type	Main Responsibility	Main Operations	Related Data

UpdateAdminProfileScreen	<<boundary>>	Provides the user interface for the Founder to view and input updated details for a Business Admin profile.	build() submitUpdate()	UpdateProfileDto
UpdateAdminProfileBloc	<<control>>	Orchestrates the profile update workflow on the client side, handling UI state and delegating data submission.	submitProfileUpdate(dto)	UpdateProfileDto
AdminApiClient	<<boundary>>	Acts as a proxy object that handles external HTTP communication with the server to update the admin profile.	putUpdateProfile(dto)	UpdateProfileDto
AdminController	<<boundary>>	Serves as the API gateway to receive external HTTP requests from the client and route them to internal services.	[HttpPut] UpdateProfile(dto)	UpdateProfileDto
AdminProfileService	<<control>>	Encapsulates the business logic required to update a Business Admin account, including data validation and rule enforcement (e.g., email uniqueness).	UpdateBusinessAdminProfile(dto)	UpdateProfileDto UserEntity
UserRepository	<<interface>>	Hides the implementation details of data access, providing a contract to retrieve, update, and validate user data in the database.	GetById(id) Update(entity) CheckEmailUnique(email, excludeUserId)	UserEntity

UpdateProfileDto	<<entity>>	A Data Transfer Object that encapsulates the profile information being transferred between the client and server.	fromJson()	N/A
UserEntity	<<entity>>	Represents the persistent data structure and attributes of a user account within the system.	N/A	N/A

3.2.2.2 Sequence Diagram

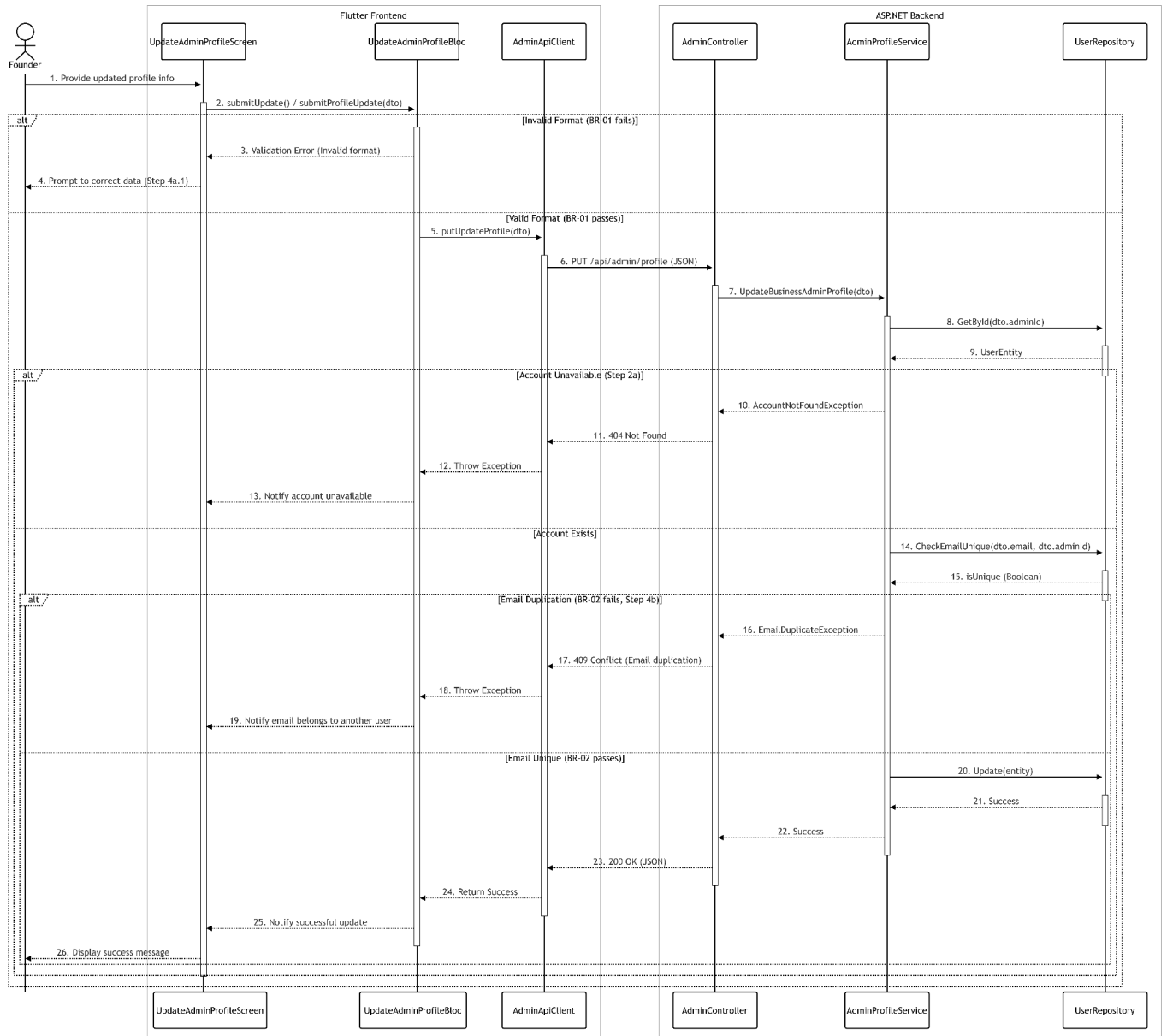


Figure 3-2.2.2: SequenceDiagram - Update Business Admin Account Profile

3.2.3 UC28- View Business Admin Account Details

3.2.3.1 Class Diagram

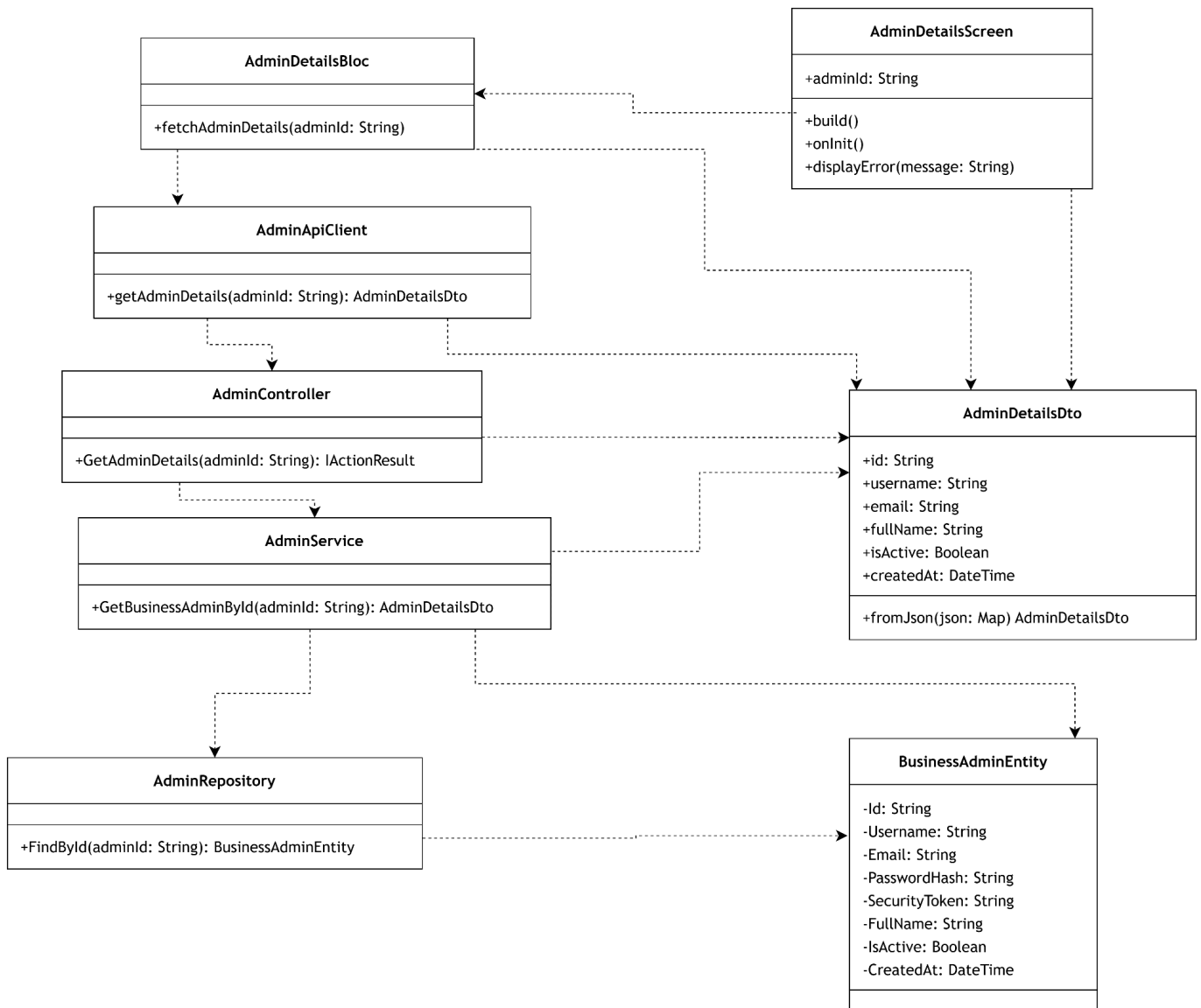


Figure 3.2.3.1: View Business Admin Account Details

Class	Type	Main Responsibility	Main Operations	Related Data

BusinessAdminDetailsPage	<<boundary>>	Provides the user interface for the Founder to request and view the detailed profile of a specific Business Admin account. Handles the presentation of the account data or error messages.	requestAccountDetails(accountId: UUID) displayAccountDetails(dto: AdminAccountDTO) displayError(message: String)	AdminAccountDTO
ViewAdminAccountController	<<control>>	Orchestrates the "View Business Admin Account Details" use case workflow. Receives the request from the boundary, delegates data fetching to the service layer, and returns the appropriately masked DTO to the UI.	getAdminDetails(accountId: UUID): AdminAccountDTO	AdminAccountDTO
IAccountService	<<interface>>	Defines the contract for business logic operations related to account management. Enforces abstraction between the controller and the core business logic processing.	fetchAccountDetails(accountId: UUID): AdminAccountDTO	AdminAccountDTO

AccountService	<<control>>	Implements business logic to validate account existence and retrieve real-time data from the repository. Responsible for mapping the raw Entity into a masked DTO to comply with data security rules (BR-01).	fetchAccountDetails(accountId: UUID): AdminAccountDTO mapToMaskedDTO(entity: BusinessAdminAccount): AdminAccountDTO	BusinessAdminAccount, AdminAccountDTO
IAccountRepository	<<interface>>	Defines the data access contract for interacting with the core database, ensuring real-time fetching (BR-02) and abstraction of persistence mechanisms.	findById(accountId: UUID): BusinessAdminAccount	BusinessAdminAccount
BusinessAdminAccount	<<entity>>	Encapsulates the persistent data and state of a Business Admin account, including profile information and active/inactive status. Hides the internal structural representation of the database record.	getId(): UUID getProfileData(): ProfileData getStatus(): AccountStatus getSensitiveCredentials(): Credentials	ProfileData, AccountStatus

AdminAccountDTO	<<entity>>	A Data Transfer Object designed to carry aggregated, sanitized account data to the boundary layer. Specifically structured to exclude highly sensitive data like hashed passwords and security tokens.	getMaskedProfileData(): Map<String, String> getCurrentStatus(): String	None
-----------------	------------	--	---	------

3.2.3.2 Sequence Diagram

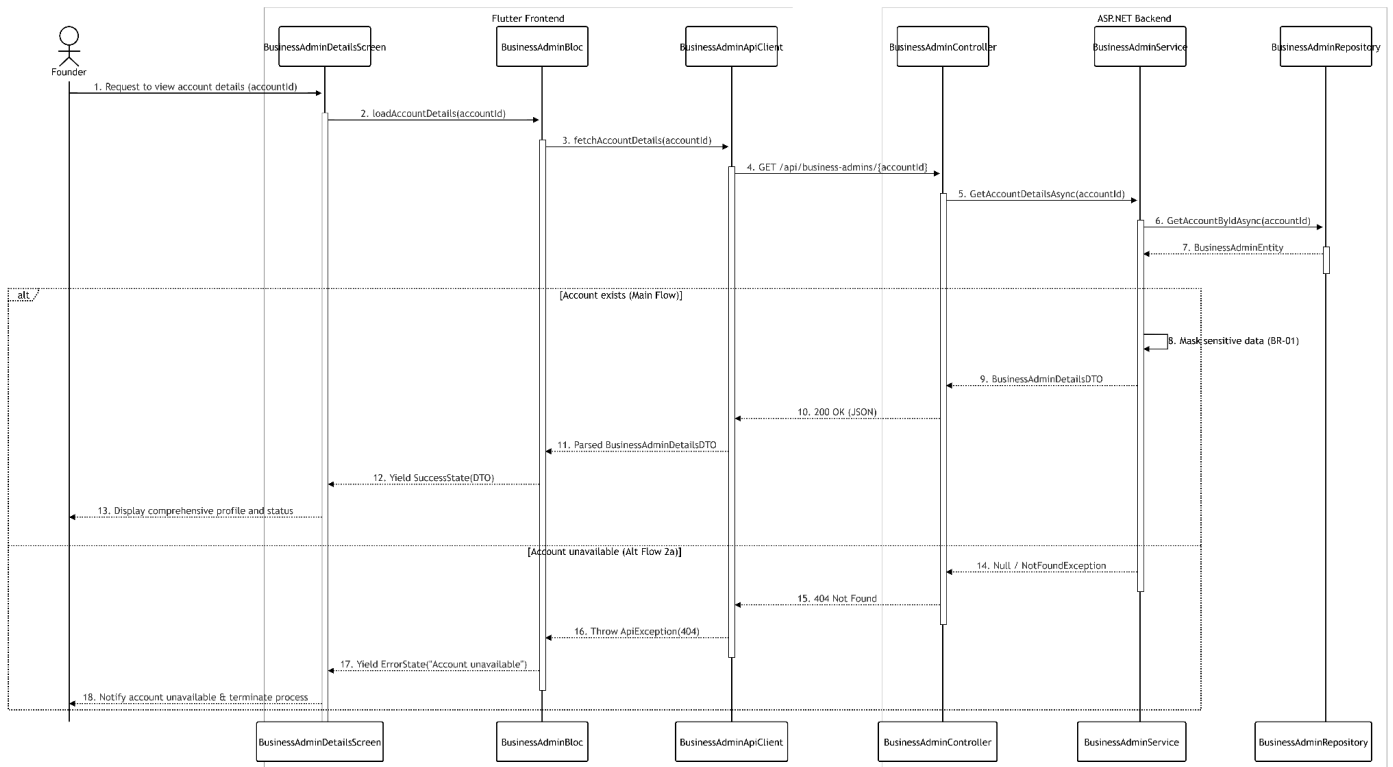


Figure 3-2.3.2: SequenceDiagram - View Business Admin Account Details

3.2.4 UC29- Deactivate Business Admin Account

3.2.4.1 Class Diagram

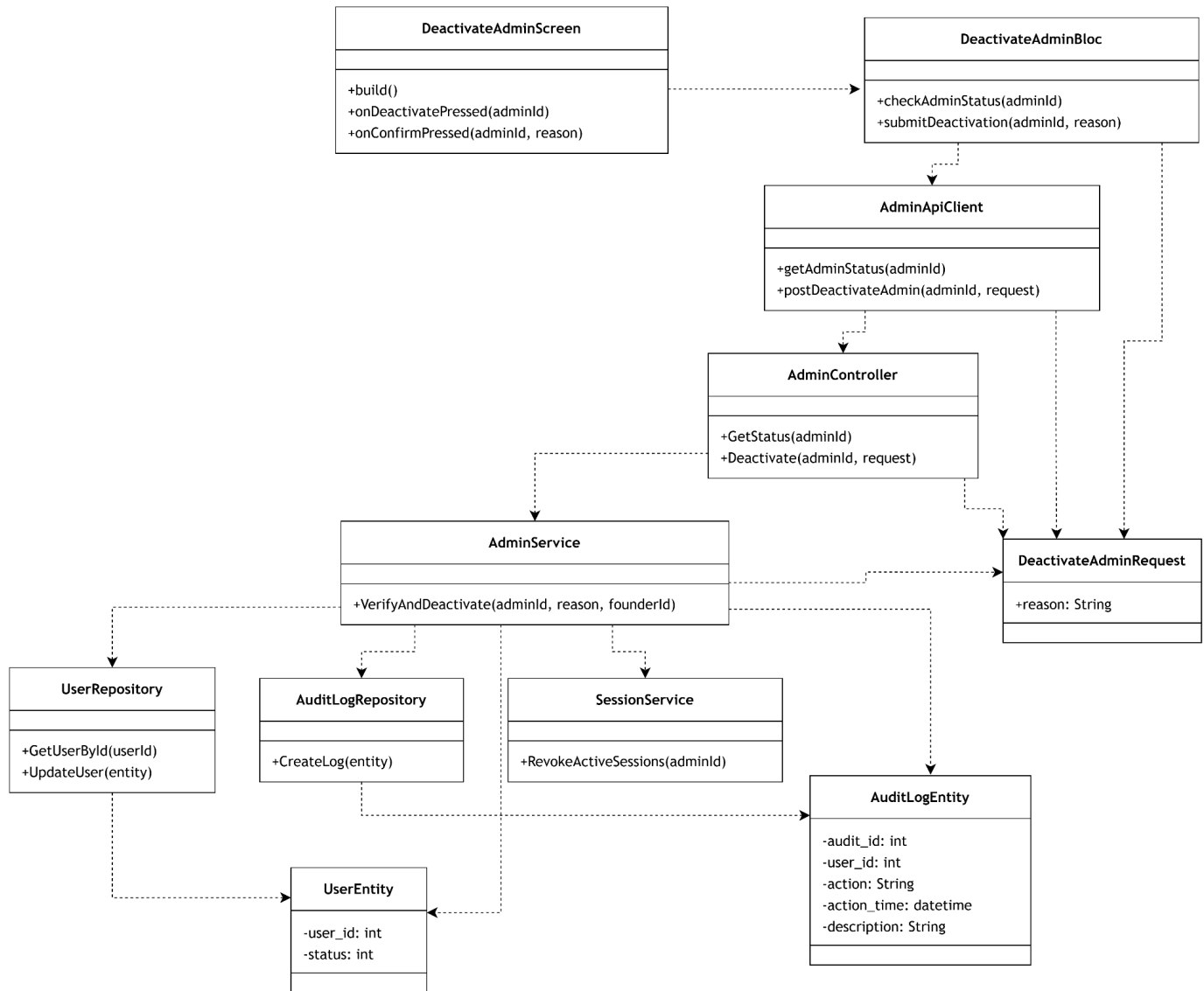


Figure 3.2.4.1: Deactivate Business Admin Account

Class	Type	Main Responsibility	Main Operations	Related Data

DeactivateAdminScreen	<<boundary>>	Provides the user interface for the Founder to initiate the deactivation process, input the deactivation reason, and provide final confirmation.	build() onDeactivatePressed(adminId) onConfirmPressed(adminId, reason)	None
DeactivateAdminBloc	<<control>>	Orchestrates presentation logic and state management, acting as an intermediary between the UI screen and the external API client.	checkAdminStatus(adminId) submitDeactivation(adminId, reason)	DeactivateAdminRequest
AdminApiClient	<<boundary>>	Acts as a proxy object that interfaces with and communicates external HTTP requests to the backend server.	getAdminStatus(adminId) postDeactivateAdmin(adminId, request)	DeactivateAdminRequest
AdminController	<<boundary>>	Serves as the API gateway receiving external HTTP requests, extracting payload data, and routing it to the appropriate backend service.	GetStatus(adminId) Deactivate(adminId, request)	DeactivateAdminRequest
AdminService	<<control>>	Encapsulates the core business logic for the deactivation workflow, coordinating state validation, user updates, session revocation, and audit logging.	VerifyAndDeactivate(adminId, reason, founderId)	UserEntity AuditLogEntity

SessionService	<<control>>	Manages authentication sessions and ensures immediate revocation of all active JWT tokens for the deactivated account.	RevokeActiveSessions(adminId)	None
UserRepository	<<interface>>	Provides a data access abstraction layer to query and persist changes to user account records.	GetUserById(userId) UpdateUser(entity)	UserEntity
AuditLogRepository	<<interface>>	Provides a data access abstraction layer to securely record immutable audit trails.	CreateLog(entity)	AuditLogEntity
DeactivateAdminRequest	<<entity>>	A Data Transfer Object (DTO) that encapsulates the payload required for the deactivation request.	None	None
UserEntity	<<entity>>	A persistent data abstraction modeling the structure of a user account and its current activation status.	None	None
AuditLogEntity	<<entity>>	A persistent data abstraction modeling the structure of a historical system audit log.	None	None

3.2.4.2 Sequence Diagram

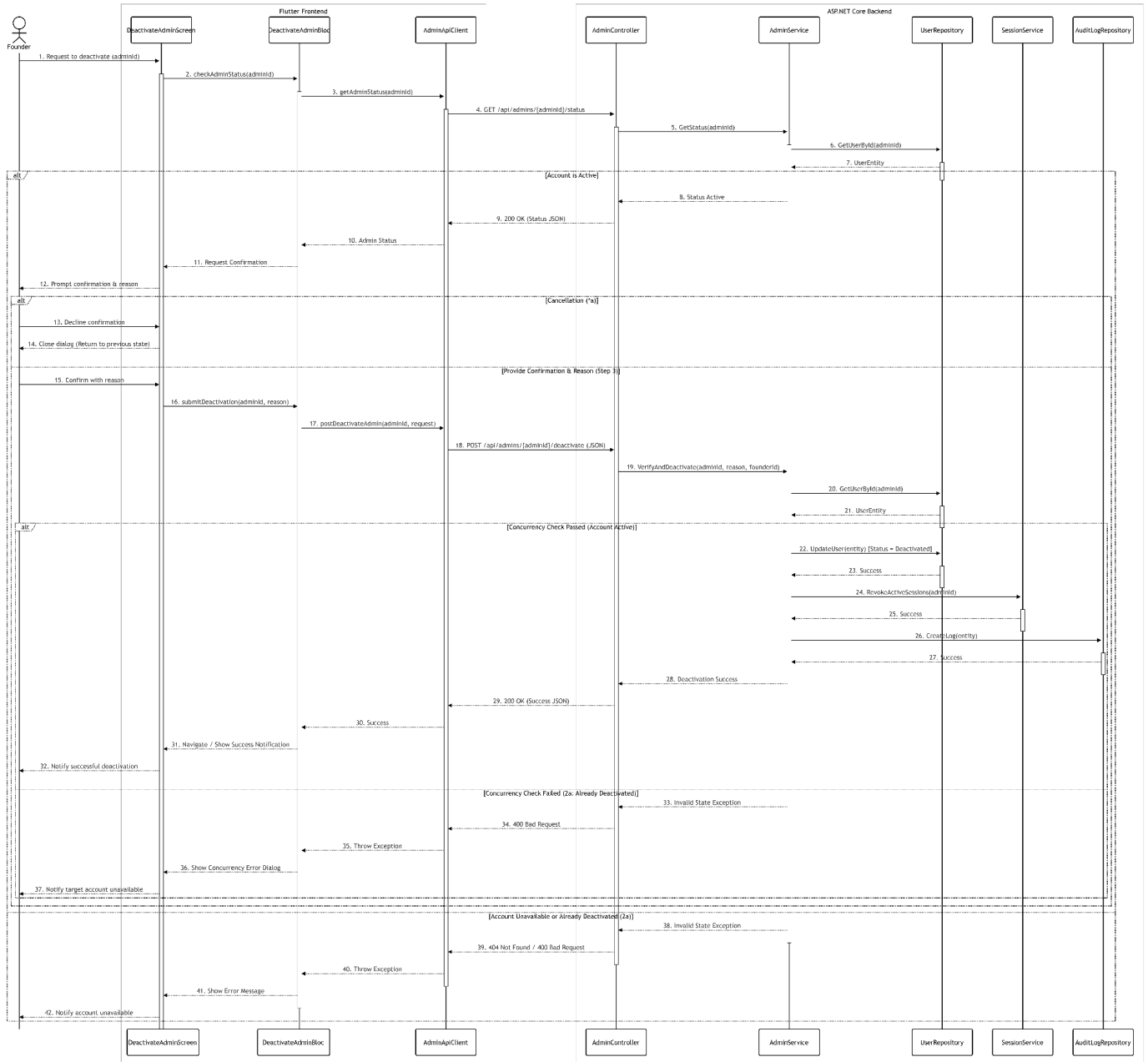


Figure 3-2.4.2: SequenceDiagram - Deactivate Business Admin Account

3.2.5 UC23- Generate Revenue Reports

3.2.5.1 Class Diagram

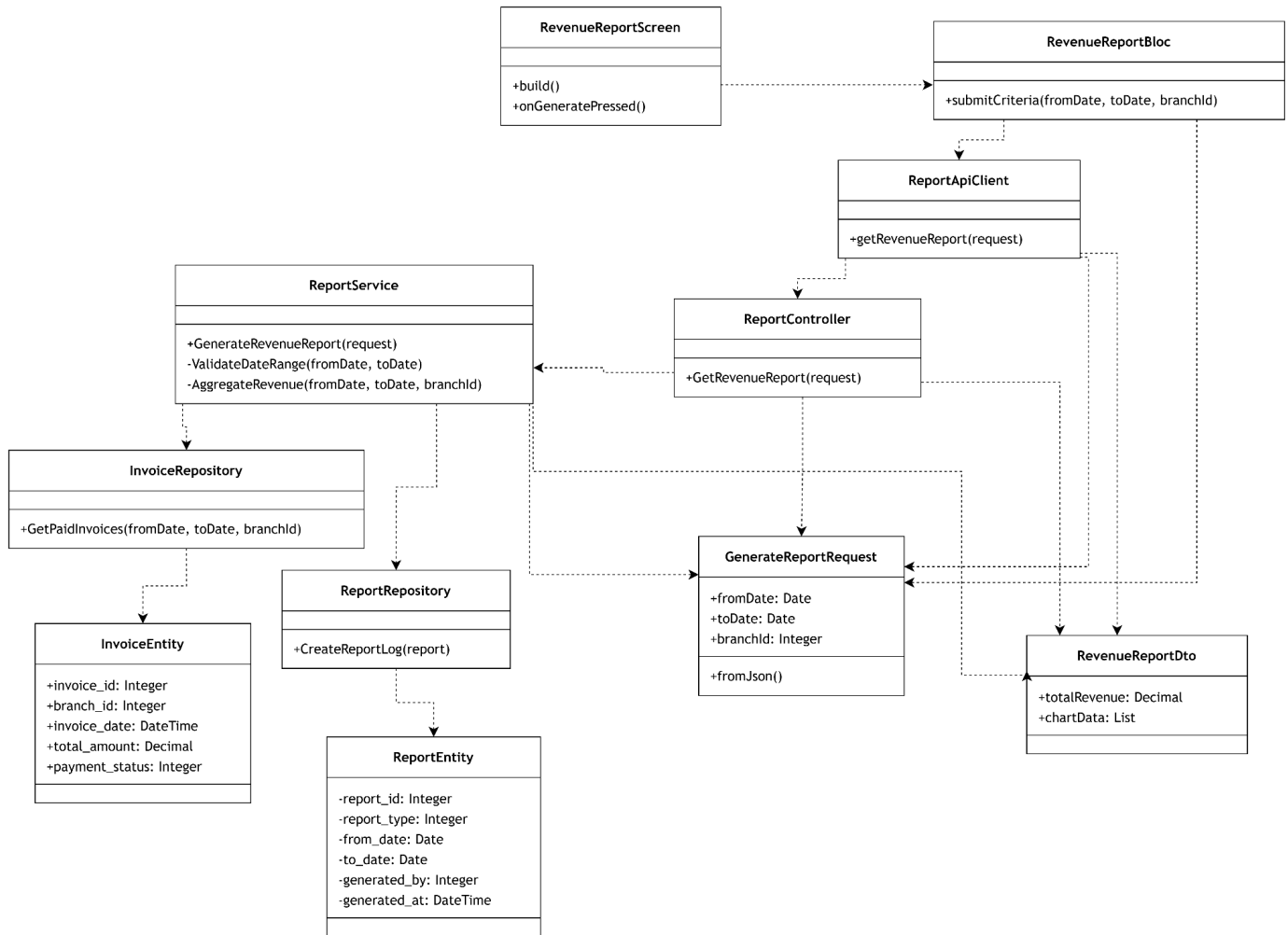


Figure 3.2.5.1: Generate Revenue Reports

Class	Type	Main Responsibility	Main Operations	Related Data

RevenueReportPage	<<boundary>>	Acts as the user interface for the Founder. It is responsible for displaying the report filtering form, capturing criteria inputs (date range, branches), and rendering the final aggregated revenue charts and metrics.	displayFilteringForm() submitCriteria(criteria: ReportCriteria) displayReport(reportData: RevenueReportDTO) displayError(message: String)	ReportCriteria, RevenueReportDTO
RevenueReportController	<<control>>	Orchestrates the "Generate Revenue Report" use case workflow. It receives requests from the boundary, enforces business rule BR-01 (Date Range Validity), and delegates data aggregation to the reporting service.	generateReport(criteria: ReportCriteria) validateCriteria(criteria: ReportCriteria): Boolean	ReportCriteria, RevenueReportDTO
FinancialReportingService	<<control>>	Encapsulates the core financial aggregation logic. It interacts with the repository to fetch orders, applies business rule BR-02 (Revenue Aggregation Logic to filter only "Completed" or "Paid" statuses), and processes the raw data into structured reporting metrics.	aggregateRevenueData(criteria: ReportCriteria): RevenueReportDTO calculateTotals(orders: List<Order>): Double	ReportCriteria, Order, RevenueReportDTO

IOrderRepository	<<interface>>	Provides an abstraction layer for persistent data access, hiding the underlying database implementation for retrieving transaction and order records.	findOrdersByCriteria(startDate: Date, endDate: Date, branchId: String): List<Order>	Order
Order	<<entity>>	Represents the persistent domain entity for customer transactions. It encapsulates details such as the order status, total amount, transaction date, and associated branch.	getStatus(): String getTotalAmount(): Double getTransactionDate(): Date getBranchId(): String	None
RevenueReportDTO	<<entity>>	A Data Transfer Object utilized to hold the computed financial summaries, charts datasets, and aggregated metrics required for presentation without exposing raw database entities.	setTotalRevenue(amount: Double) getTotalRevenue(): Double setChartData(data: ChartData) getChartData(): ChartData	None

3.2.5.2 Sequence Diagram

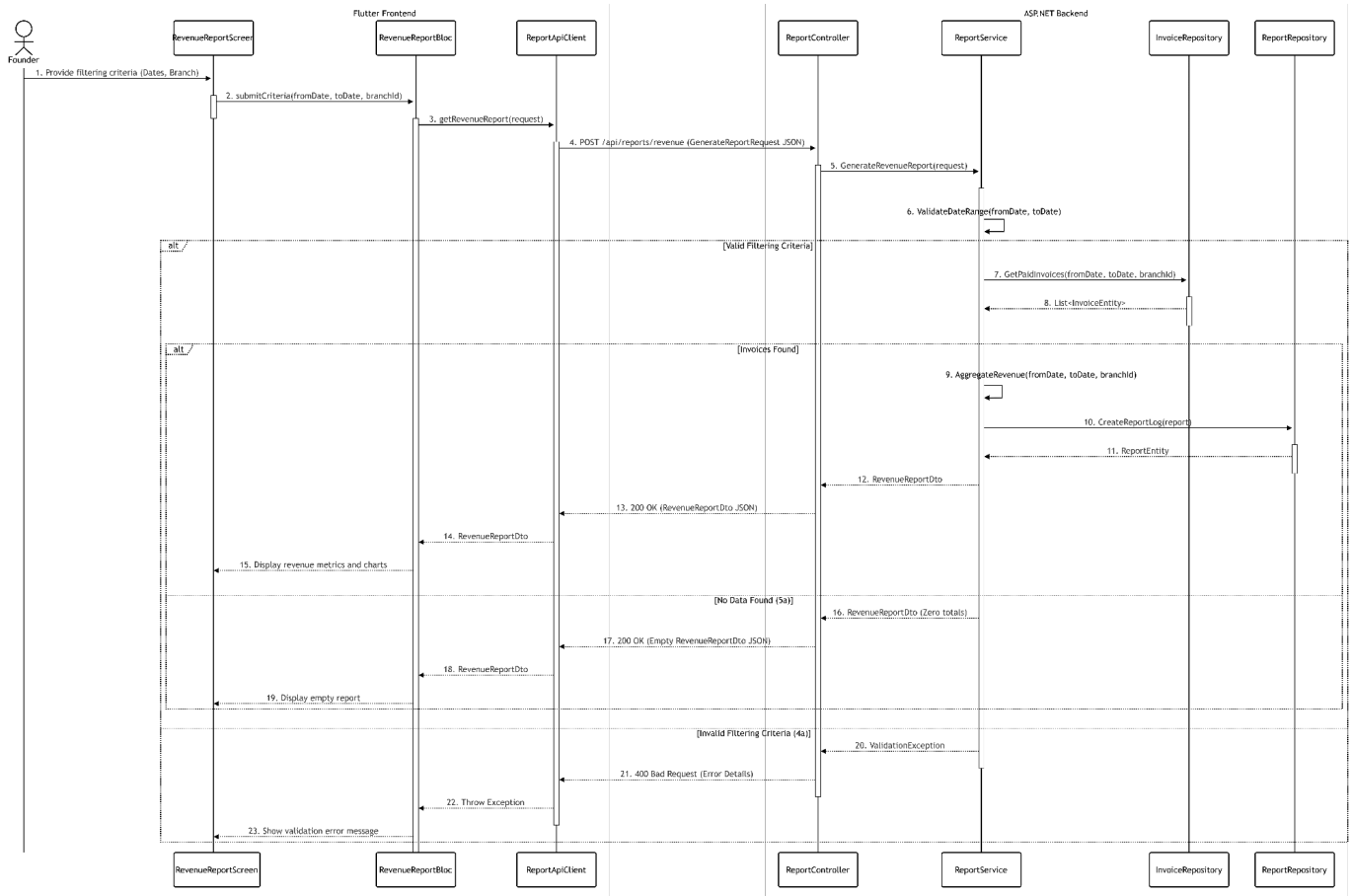


Figure 3-2.1.2: SequenceDiagram - Generate Revenue Reports

3.2.6 UC24- Generate Cash Flow Statistics

3.2.6.1 Class Diagram

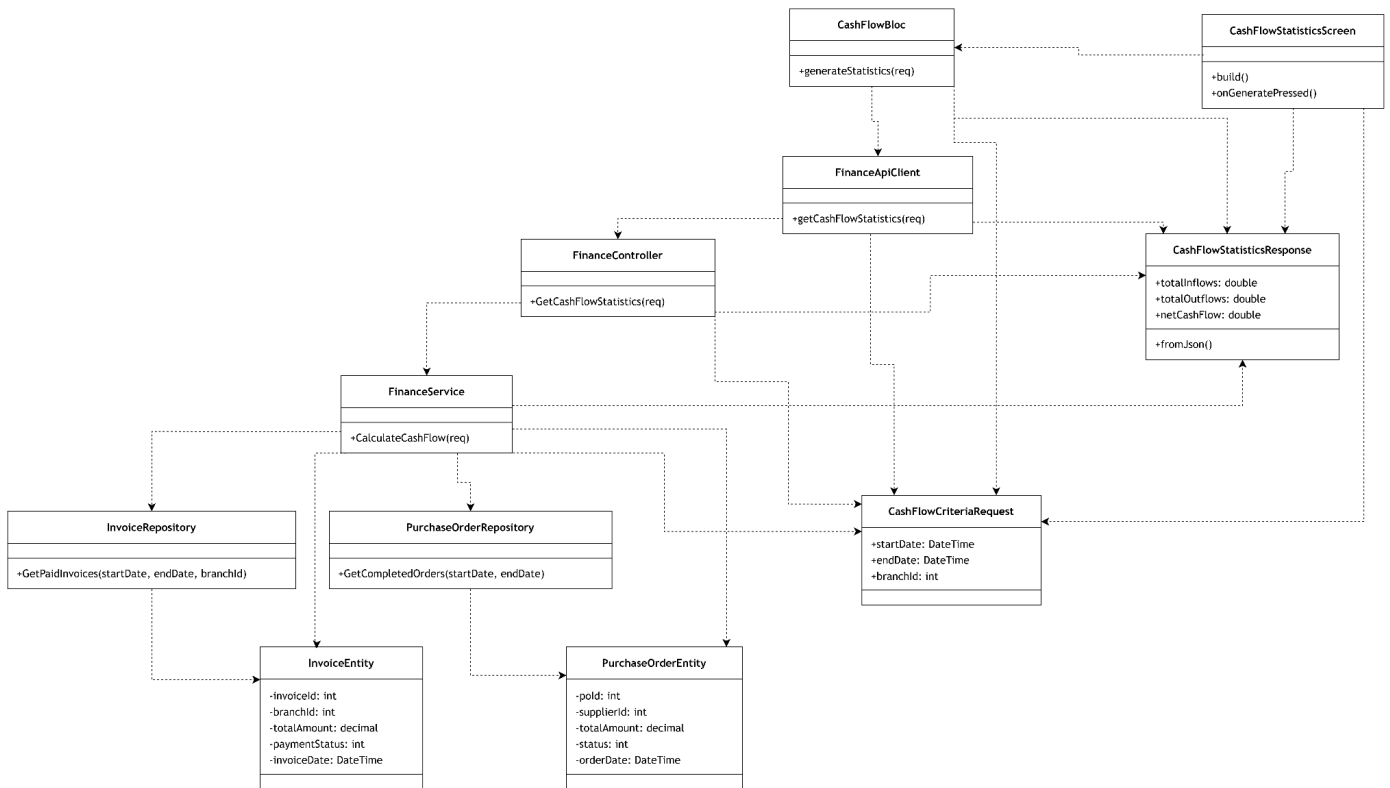


Figure 3.2.6.1: Create Business Admin Account

Class	Type	Main Responsibility	Main Operations	Related Data
CashFlowBloc	<<control>>	Orchestrates the workflow for generating statistics by managing the user interaction state and triggering service requests.	generateStatistics(req)	CashFlowCriteriaRequest

CashFlowStatisticsScreen	<<boundary>>	Provides the user interface for the Founder to define filtering criteria and displays the calculated financial statistics.	build() onGeneratePressed()	CashFlowCriteriaRequest, CashFlowStatisticsResponse
FinanceApiClient	<<control>>	Acts as a proxy for remote API communication to retrieve financial datasets.	getCashFlowStatistics(req)	CashFlowCriteriaRequest, CashFlowStatisticsResponse
FinanceController	<<control>>	Coordinates the application logic flow for handling cash flow request delegation.	GetCashFlowStatistics(req)	CashFlowCriteriaRequest
FinanceService	<<control>>	Implements the core business logic for aggregating and calculating net cash flow based on inflow and outflow transactions (BR-02).	CalculateCashFlow(req)	InvoiceEntity, PurchaseOrderEntity, CashFlowStatisticsResponse
InvoiceRepository	<<interface>>	Provides a data access contract for retrieving finalized sales revenue records.	GetPaidInvoices(startDate, endDate, branchId)	InvoiceEntity
PurchaseOrderRepository	<<interface>>	Provides a data access contract for retrieving completed supplier payment records.	GetCompletedOrders(startDate, endDate)	PurchaseOrderEntity

InvoiceEntity	<<entity>>	Represents a successfully paid sales transaction.	N/A	N/A
PurchaseOrderEntity	<<entity>>	Represents a completed supplier payment or operating cost.	N/A	N/A
CashFlowStatisticsResponse	<<entity>>	Data Transfer Object encapsulating calculated financial totals.	fromJson()	N/A
CashFlowCriteriaRequest	<<entity>>	Encapsulates the filtering criteria provided by the user.	N/A	N/A

3.2.6.2 Sequence Diagram

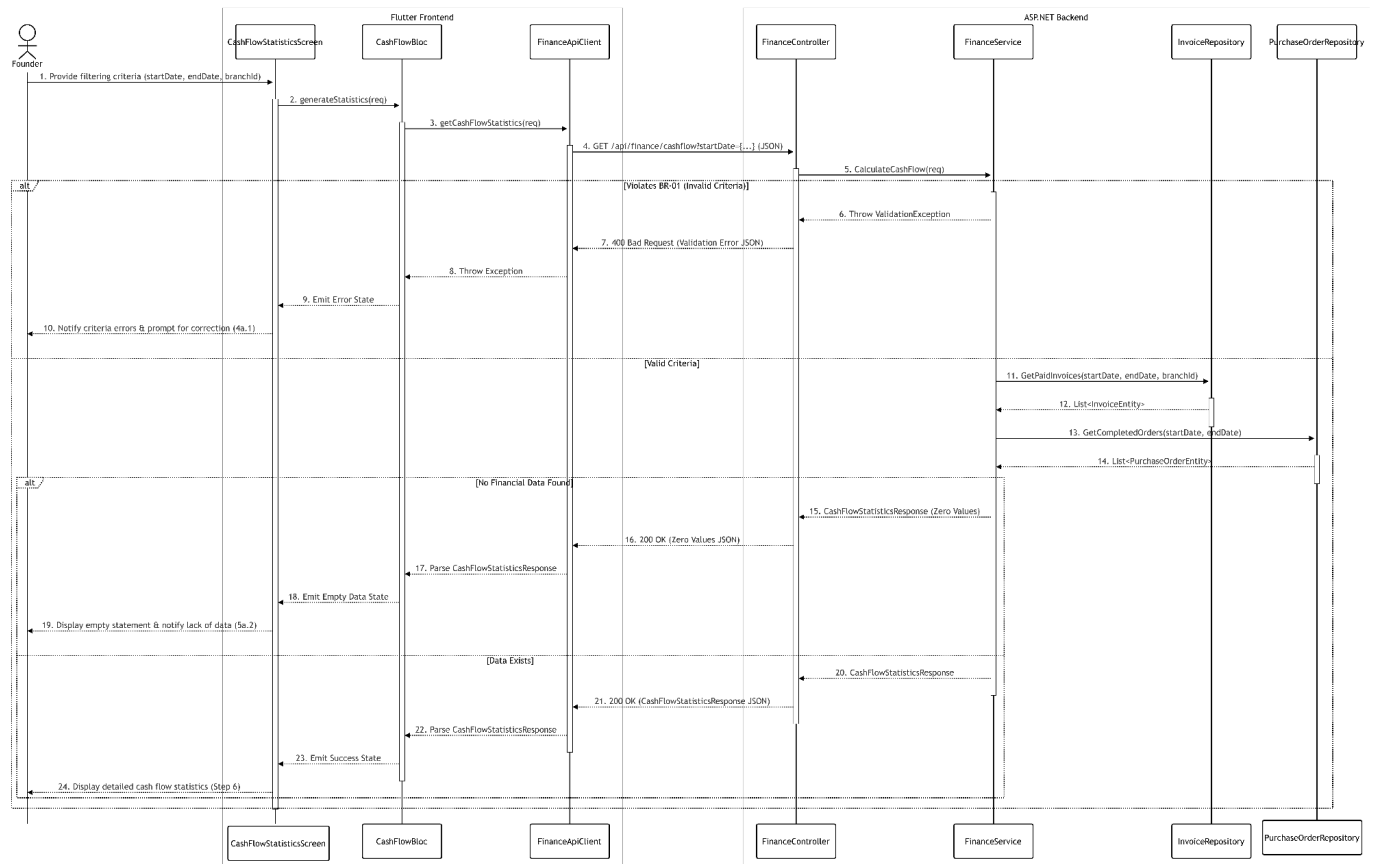


Figure 3-2.6.2: SequenceDiagram - Create Business Admin Account

3.2.7 UC25- Export Financial Reports

3.2.7.1 Class Diagram

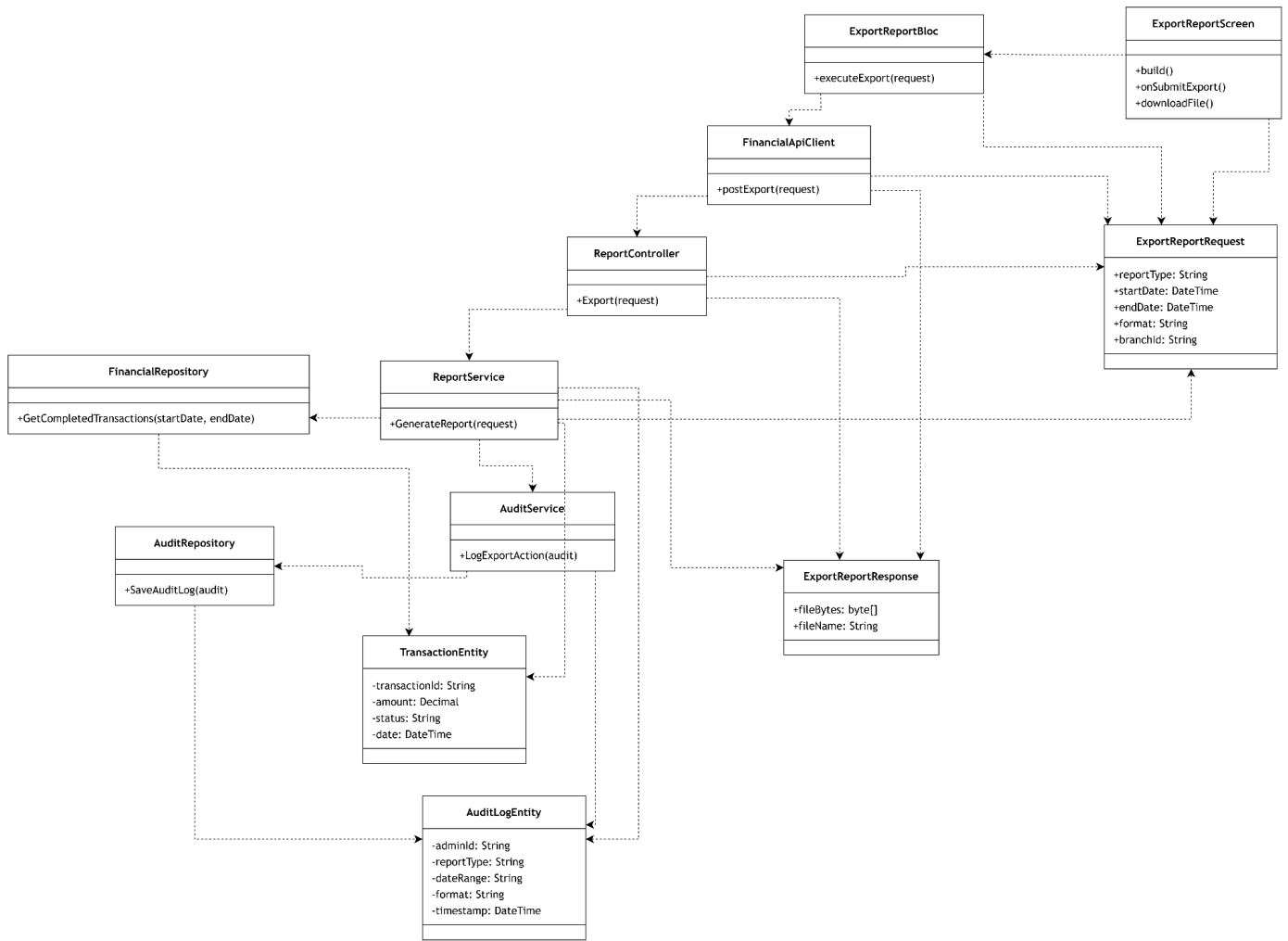


Figure 3.2.7.1: Export Financial Reports

Class	Type	Main Responsibility	Main Operations	Related Data

ExportReportPage	<<boundary>>	Provides the user interface for the Founder to input export criteria (Report Type, Date Range, Branch, Format). Displays validation errors, no-data alerts, and facilitates the secure download of the generated report file.	promptExportCriteria() submitCriteria(criteria) displayAlert(message) downloadFile(file)	ExportCriteriaDTO, ReportFile
ExportReportController	<<control>>	Orchestrates the export workflow. Validates export criteria against business rules (BR-01), coordinates data retrieval, delegates file generation (BR-02), manages failure states, and triggers the audit logging process (BR-03).	processExportRequest(criteria) validateCriteria(criteria) handleGenerationFailure(error)	ExportCriteriaDTO, ReportFile
FinancialRepository	<<interface>>	Abstracts data access logic to query financial records from the database. Ensures that only records with a 'Completed/Paid' status within the specified scope are retrieved (BR-01).	checkDataAvailability(criteria) getCompletedRecords(criteria)	FinancialRecord, ExportCriteriaDTO
FinancialRecord	<<entity>>	Encapsulates the persistent data of a single financial transaction, hiding internal database structures. Contains attributes such as transaction date, amount, branch ID, and payment status.	getStatus() getTransactionDetails()	FinancialRecord

ReportGeneratorEngine	<<control>>	Responsible for formatting raw financial data into standard PDF or Excel (.xlsx) files. Automatically embeds required metadata headers (Timestamp, Founder ID, Data Scope) into the file structure (BR-02).	generateFile(data, format) embedMetadata(metadata)	FinancialRecord, ReportFile, MetadataDTO
AuditLogRepository	<<interface>>	Handles the persistent storage of audit trails. Ensures every successful financial data exfiltration event is securely permanently recorded in the system.	recordExportEvent(logEntry)	AuditLogEntry
AuditLogEntry	<<entity>>	Represents the structured data of an export audit event, encapsulating the Admin ID, Report Type, Date Range, Export Format, and timestamp of the action (BR-03).	getLogDetails() setTimestamp(time)	AuditLogEntry

3.2.7.2 Sequence Diagram

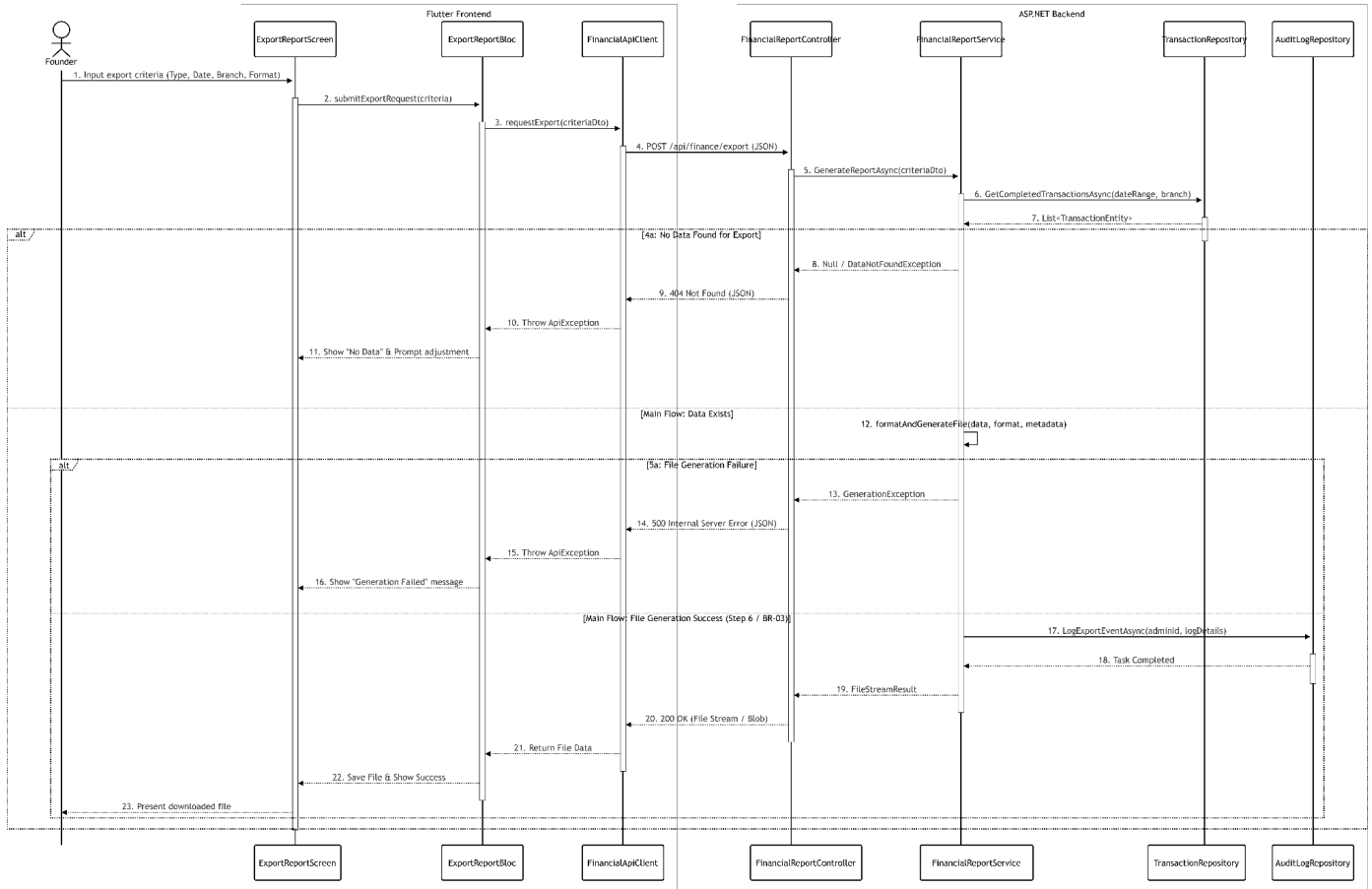


Figure 3-2.1.2: SequenceDiagram - Export Financial Reports

3.3 Business Admin Detail Design

3.3.1 UC034- View Business Analysis Report

3.3.1.1 Class Diagram

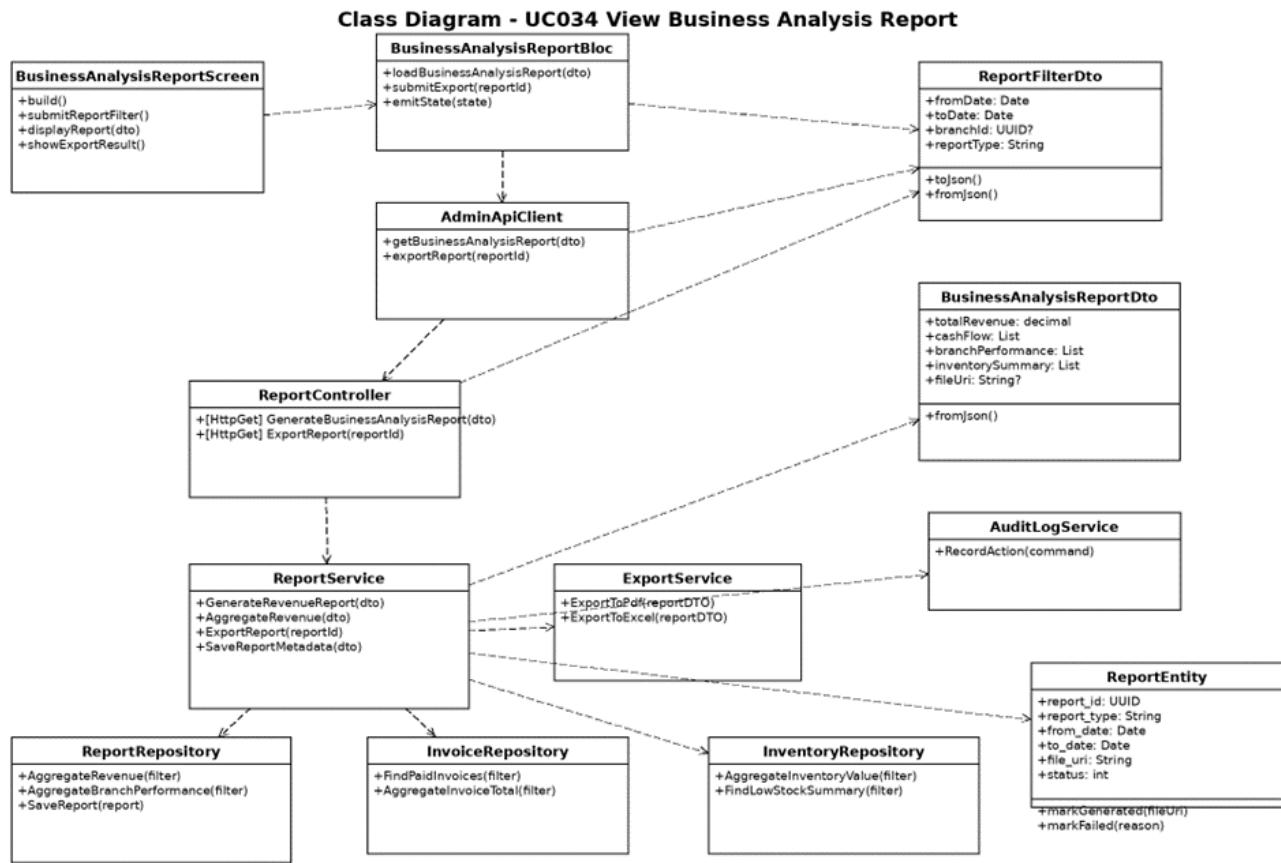


Figure 3.3.1.1: Class Diagram - UC034 View Business Analysis Report

Class	Type	Main Responsibility	Main Operations	Related Data
BusinessAnalysisReportScreen	<<boundary>>	Provides the user interface for Business Admin to select report filters, view business analysis results, and request report export.	build(); submitFilter(); clickExportPdf(); clickExportExcel(); displayReport(dto)	BusinessAnalysisFilterDTO; BusinessAnalysisReportDTO
BusinessAnalysisReportBloc	<<control>>	Orchestrates presentation logic and state management between the UI screen and the API client.	loadBusinessAnalysis(filterDTO); exportReport(reportId, format); emitLoadingState(); emitSuccessState(dto); emitErrorState(errorDTO)	BusinessAnalysisFilterDTO; BusinessAnalysisReportDTO
AdminApiClient	<<boundary>>	Acts as a frontend proxy object that converts Business Admin actions into REST requests to the backend API.	getBusinessAnalysisReport(filterDTO); postExportReport(reportId, format)	BusinessAnalysisFilterDTO; ExportRequestDTO
ReportController	<<control>>	Receives report-related HTTP requests, validates request DTOs, and delegates business rules to ReportService.	generateBusinessAnalysisReport(dto); exportReport(reportId, format)	BusinessAnalysisFilterDTO; ReportDTO

ReportService	<<control>>	Aggregates business data from revenue, payment, inventory, and branch performance sources to generate business analysis reports.	generateBusinessAnalysisReport(dto); aggregateRevenue(dto); aggregateBranchPerformance(dto); saveReportMetadata(dto)	Invoice; PaymentTransaction; Inventory; Report
IReportRepository	<<interface>>	Defines report metadata persistence and read-only aggregation operations.	saveReport(report); findReports(filter); aggregateBusinessAnalysis(filter)	REPORT
IInvoiceRepository	<<interface>>	Provides read-only invoice data for revenue and branch performance analysis.	findInvoicesByFilter(filter); aggregateInvoiceTotal(filter)	INVOICE; INVOICE_DETAIL
IPaymentRepository	<<interface>>	Provides payment transaction data for paid revenue, failed payment, and cash flow analysis.	findPaymentsByFilter(filter); aggregatePaymentStatus(filter)	PAYMENT_TRANSACTION
IInventoryRepository	<<interface>>	Provides inventory data used to analyze stock value, low-stock items, and inventory movement.	findInventoryByBranch(branchId); aggregateInventoryValue(filter)	INVENTORY; MEDICINE_BATCH
ExportService	<<control>>	Generates downloadable PDF or Excel report files from the report result DTO.	exportToPdf(reportDTO); exportToExcel(reportDTO)	Report file
Report	<<entity>>	Encapsulates generated report metadata, report status, file URI, and generated time.	markGenerated(fileUri); markFailed(reason)	REPORT

3.3.1.2 Sequence Diagram

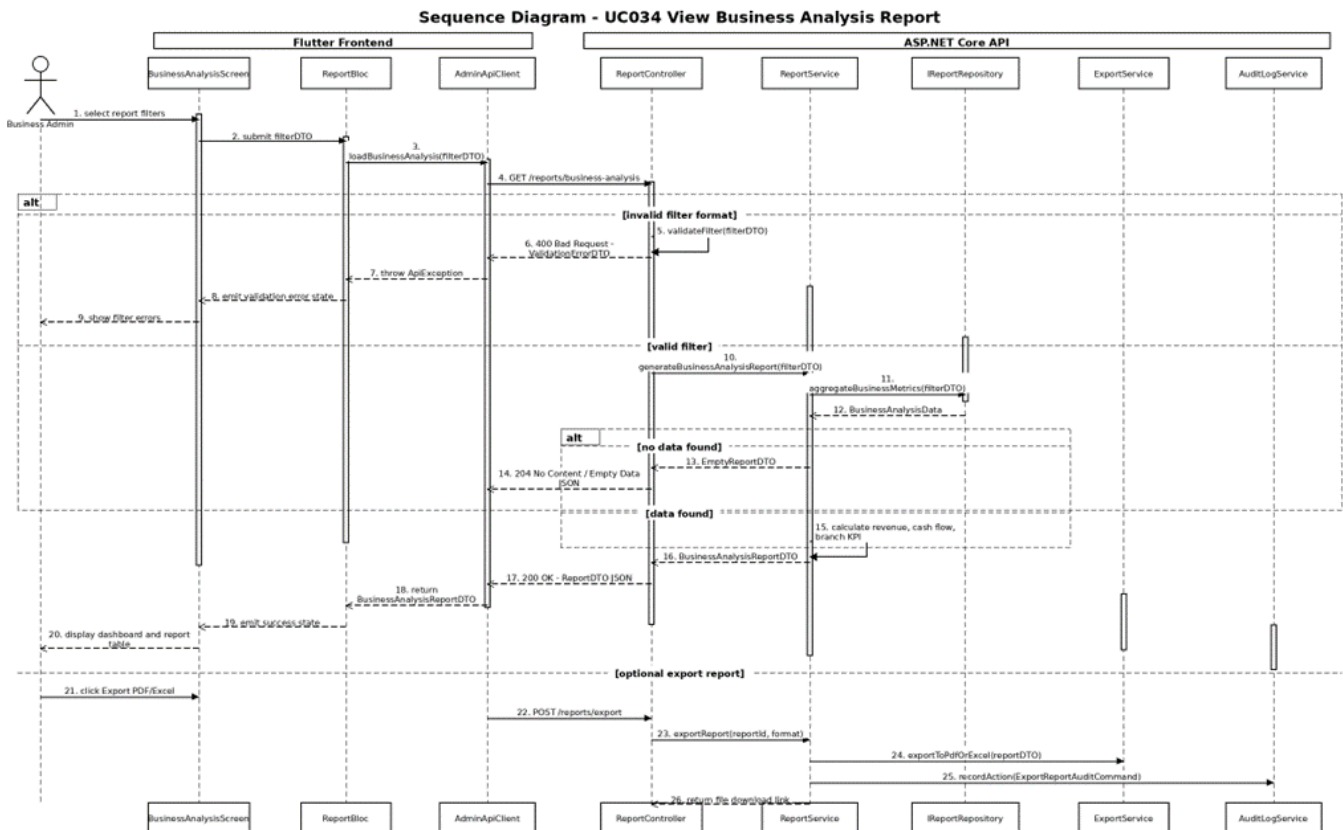


Figure 3.3.1.1: Class Diagram - UC034 View Business Analysis Report

3.3.2 UC06- View Business Medicine Statistics

3.3.2.1 Class Diagram

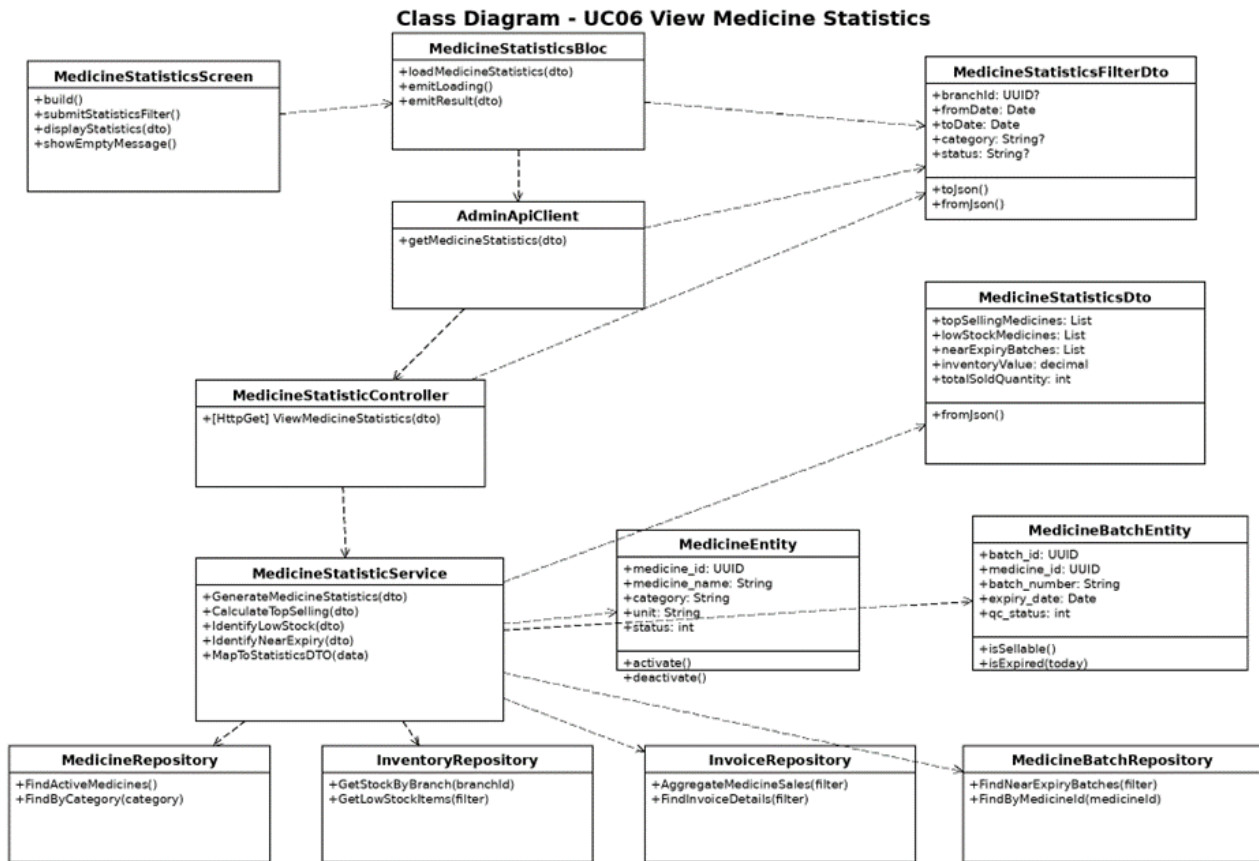


Figure 3.3.1.1: Class Diagram - UC06 View Medicine Statistics

Class Specifications - UC06 View Medicine Statistics

Class	Type	Main Responsibility	Main Operations	Related Data
MedicineStatisticsScreen	<<boundary>>	Provides the user interface for Business Admin to view medicine statistics such as best-selling, low-stock, and near-expiry medicines.	build(); submitStatisticFilter(); displayMedicineStatistics(dto); showError(errorDTO)	MedicineStatisticFilterDTO; MedicineStatisticsDTO
MedicineStatisticsBloc	<<control>>	Manages UI state and coordinates medicine statistics requests between the screen and API client.	loadMedicineStatistics(filterDTO); emitLoadingState(); emitSuccessState(dto); emitErrorState(errorDTO)	MedicineStatisticFilterDTO; MedicineStatisticsDTO
AdminApiClient	<<boundary>>	Sends medicine statistics requests from Flutter frontend to ASP.NET Core API.	getMedicineStatistics(filterDTO)	MedicineStatisticFilterDTO

ReportController	<<control>>	Receives medicine statistics requests and forwards them to the report or statistic service.	generateMedicineStatistics(dto)	MedicineStatisticFilterDTO
ReportService	<<control>>	Aggregates medicine, sales, inventory, and batch data to generate medicine statistics for Business Admin.	generateMedicineStatistics(dto); aggregateBestSellingMedicines(dto); aggregateLowStockMedicines(dto); aggregateNearExpiryMedicines(dto)	Medicine; Inventory; InvoiceDetail; MedicineBatch
IMedicineRepository	<<interface>>	Provides medicine master data for statistic calculation.	findAllActiveMedicines(); findById(medicineId); findByCategory(category)	MEDICINE
IInventoryRepository	<<interface>>	Provides stock quantity and safety stock data for low-stock and inventory analysis.	findInventoryByFilter(filter); findLowStockItems(branchId); aggregateStockQuantity(filter)	INVENTORY
IMedicineBatchRepository	<<interface>>	Provides batch and expiry data for near-expiry or expired medicine statistics.	findNearExpiryBatches(dateRange); findExpiredBatches(today); findByMedicineId(medicineId)	MEDICINE_BATCH
IInvoiceRepository	<<interface>>	Provides sales invoice detail data to calculate best-selling and slow-moving medicines.	findInvoiceDetailsByFilter(filter); aggregateMedicineSales(filter)	INVOICE; INVOICE_DETAIL
MedicineStatisticsDTO	<<dto>>	Carries summarized medicine statistic data from backend to frontend.	fromEntityData(); toJson()	bestSellingList; lowStockList; nearExpiryList
Medicine	<<entity>>	Encapsulates medicine identity, category, unit, standard price, and active status.	activate(); deactivate(); updateInfo(dto)	MEDICINE
MedicineBatch	<<entity>>	Encapsulates batch number, expiry date, QC status, and sellable state.	isExpired(today); isSellable(); updateQcStatus(status)	MEDICINE_BATCH

3.3.2.2 Sequence Diagram

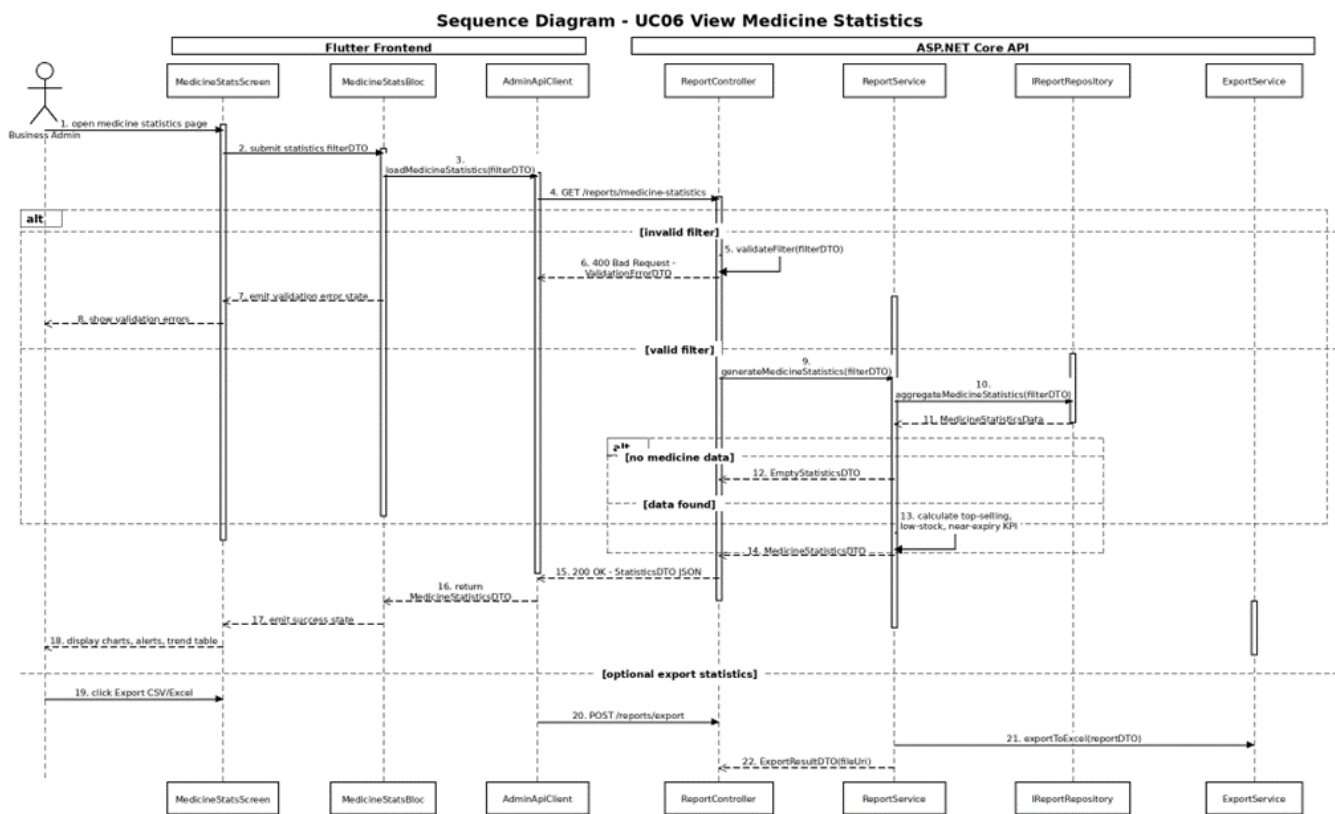


Figure 3.3.1.2: Sequence Diagram - UC06 View Medicine Statistics

3.3 UC03- View Branch Management

3.3.1 Class Diagram

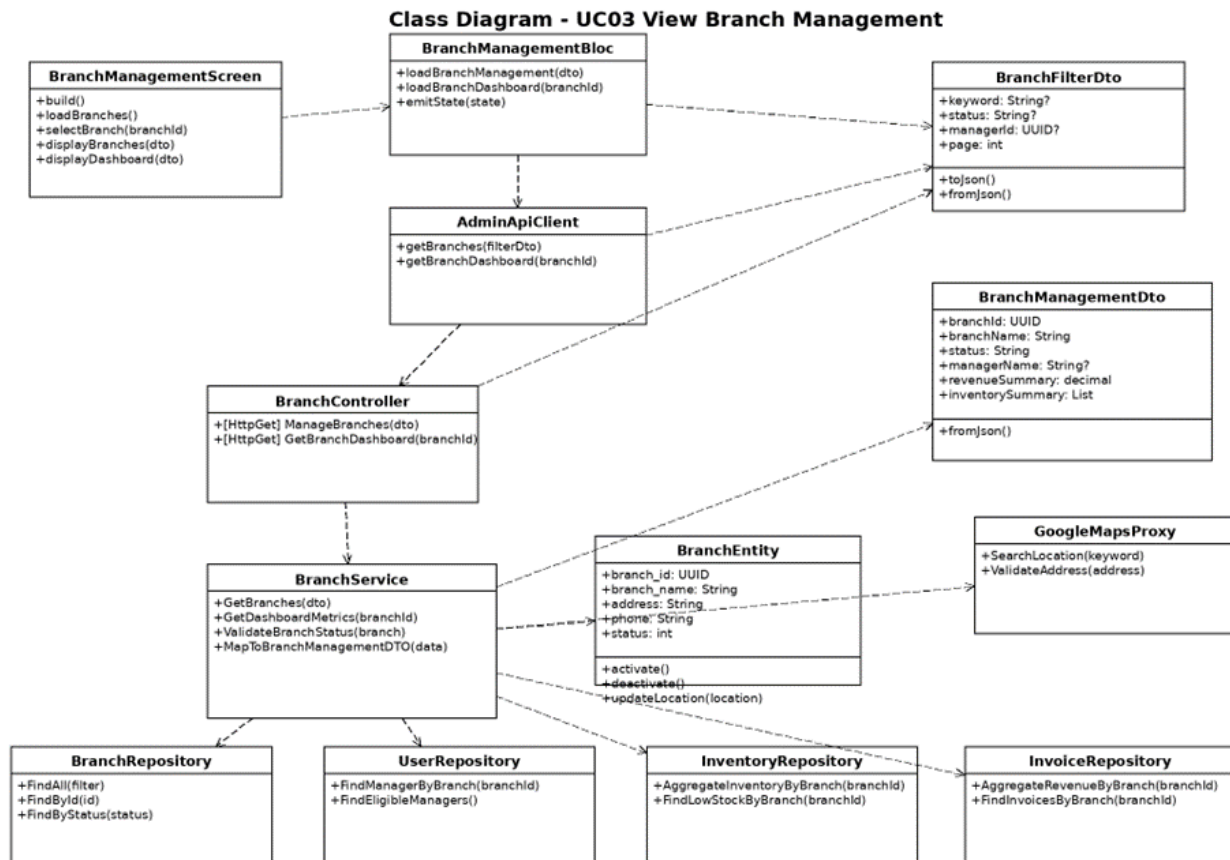


Figure 3.3.3.1: Class Diagram - UC03 View Branch Management

Class Specifications - UC03 View Branch Management

Class	Type	Main Responsibility	Main Operations	Related Data
BranchManagementScreen	<<boundary>>	Provides the user interface for Business Admin to view branch list, branch details, manager assignment, and branch dashboard information.	build(); submitBranchFilter(); submitBranchForm(dto); submitManagerAssignment(dto); displayBranchList(dto)	BranchDTO; BranchManagementDTO
BranchBloc	<<control>>	Handles presentation state for branch management and communicates with AdminApiClient.	loadBranches(filterDTO); saveBranch(dto); assignManager(dto); emitSuccessState(dto); emitErrorState(errorDTO)	BranchRequestDTO; BranchDTO
AdminApiClient	<<boundary>>	Acts as the frontend API adapter that sends branch management requests to backend endpoints.	getBranches(filterDTO); postBranch(dto); putBranch(branchId, dto); postAssignManager(dto)	BranchRequestDTO; ManagerAssignmentDTO
BranchController	<<control>>	Handles branch-related HTTP requests, validates DTOs, and	manageBranches(); saveBranch(dto); authorizeManager(dto); getBranchDashboard(branchId)	BranchRequestDTO; DashboardDTO

		delegates branch rules to BranchService.		
BranchService	<<control>>	Applies branch business rules, validates branch status, checks manager eligibility, and aggregates dashboard metrics.	saveBranch(dto); assignManager(dto); getDashboardMetrics(branchId); validateBranchStatus(branch)	Branch; User; Inventory; Invoice
IBranchRepository	<<interface>>	Defines branch persistence and query operations.	findAll(); findById(id); save(branch); findByStatus(status)	BRANCH
IUserRepository	<<interface>>	Finds eligible Branch Manager accounts and updates branch assignment.	findEligibleManagers(); findById(userId); updateBranchAssignment(userId, branchId)	USER
IGoogleMapsProxy	<<interface>>	Validates and normalizes branch address/location information through Google Maps integration.	searchLocation(keyword); validateAddress(address)	External Google Maps API
AuditLogService	<<control>>	Records branch creation, update, manager assignment, and status change actions for traceability.	recordAction(command)	AUDIT_LOG
Branch	<<entity>>	Encapsulates branch identity, address, phone number, location, and operational status.	activate(); deactivate(); updateLocation(location); updateInfo(dto)	BRANCH
User	<<entity>>	Encapsulates internal user identity, role, branch assignment, and account status.	assignRole(role); assignBranch(branchId); deactivate(); isActive()	USER

3.3.2 Sequence Diagram

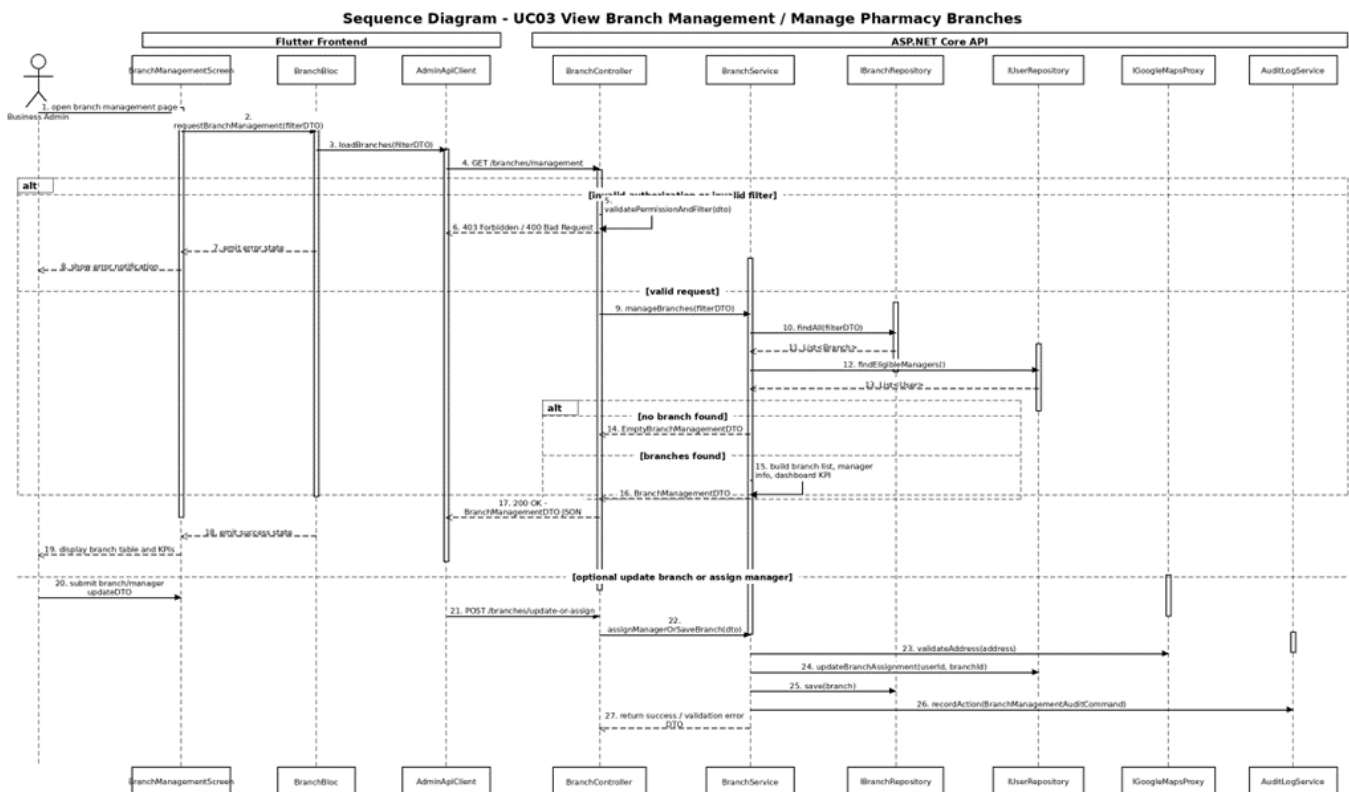


Figure 3.3.3.2: Sequence Diagram - UC03 View Branch Management

3.4 Branch Manager Design Details

The **Branch Manager module** provides functions for managing and monitoring pharmacy branch operations. It allows branch managers to view business information, analyze revenue, monitor employee performance, and track inventory status.

3.4.1 UC09 - View Branch Dashboard

3.4.1.1 Class Diagram

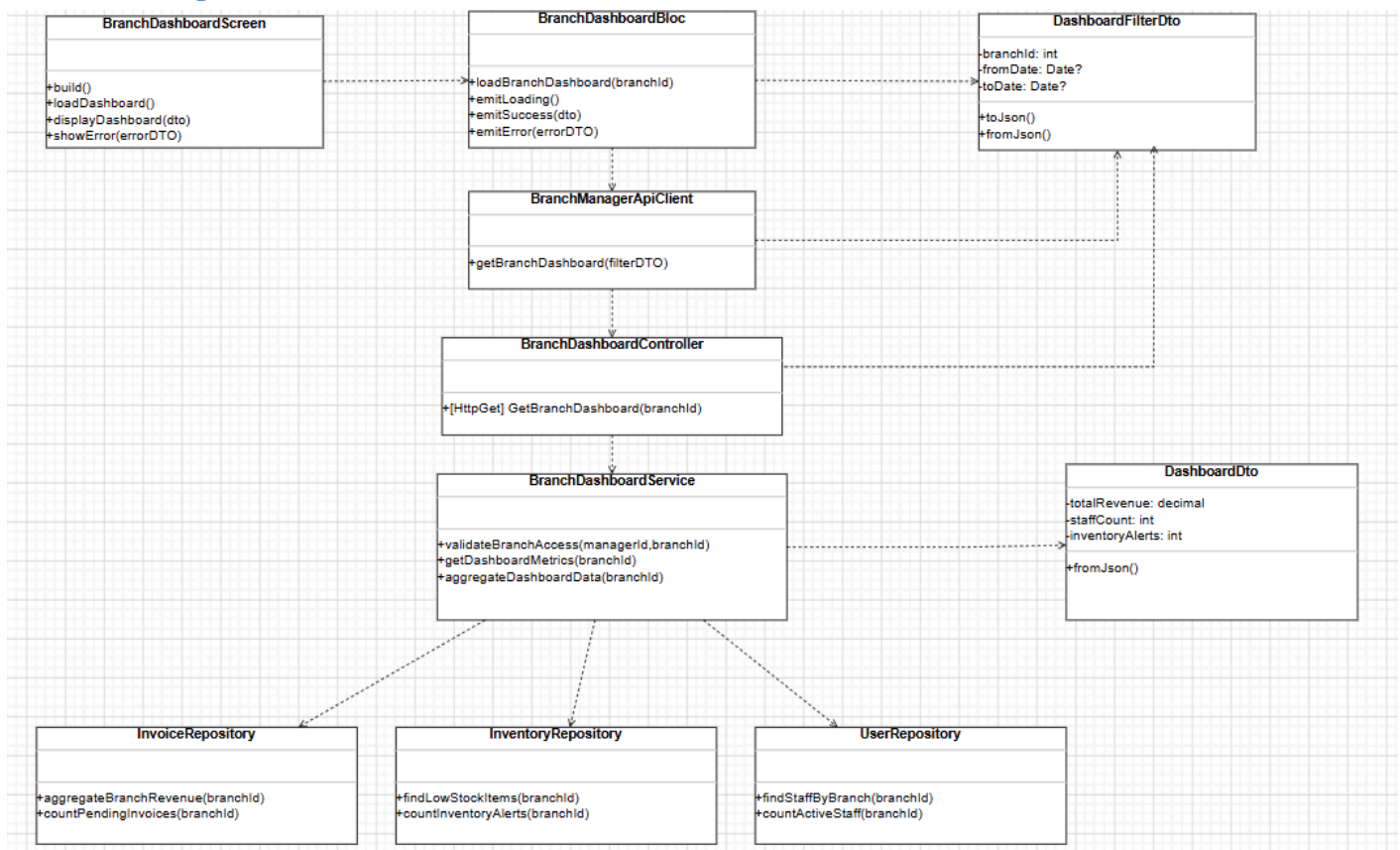


Figure 3.4.1.1: Class Diagram - UC09 View Branch Dashboard

Class Specifications - UC09 View Branch Dashboard

Class	Type	Main Responsibility	Main Operations	Related Data
BranchDashboardScreen	<<boundary>>	Displays branch dashboard information, including revenue, staff statistics, and inventory alerts.	build(); loadDashboard(); displayDashboard(dto); showError(errorDTO)	DashboardDTO
BranchDashboardBloc	<<control>>	Manages dashboard state and coordinates data loading between the UI and API layers.	loadBranchDashboard(branchId); emitLoading(); emitSuccess(dto); emitError(errorDTO)	DashboardDTO; DashboardFilterDTO
BranchManagerApiClient	<<boundary>>	Sends dashboard requests to backend services and receives responses.	getBranchDashboard(filterDTO)	DashboardDTO; DashboardFilterDTO
BranchDashboardController	<<control>>	Receives dashboard requests and forwards them to the service layer.	getBranchDashboard(branchId);	DashboardDTO; DashboardFilterDTO
BranchDashboardService	<<control>>	Aggregates dashboard data and applies business rules for dashboard processing.	validateBranchAccess(managerId, branchId); getDashboardMetrics(branchId); aggregateDashboardData(branchId);	Invoice; User; Inventory
InvoiceRepository	<<entity>>	Provides revenue and invoice data for dashboard statistics.	aggregateBranchRevenue(branchId); countPendingInvoices(branchId)	INVOICE
InventoryRepository	<<entity>>	Provides inventory information and identifies low-stock items.	findLowStockItems(branchId); countInventoryAlerts(branchId)	INVENTORY
UserRepository	<<entity>>	Provides staff information and statistics for the dashboard.	findStaffByBranch(branchId); countActiveStaff(branchId)	USER
DashboardDTO	<<entity>>	Transfers dashboard data between application layers.	fromJson()	totalRevenue; staffCount; inventoryAlerts; branchEfficiency
DashboardFilterDTO	<<entity>>	Carries dashboard filtering parameters such as branch and date range.	toJson(); fromJson()	branchId; fromDate; toDate

3.4.1.2 Sequence Diagram

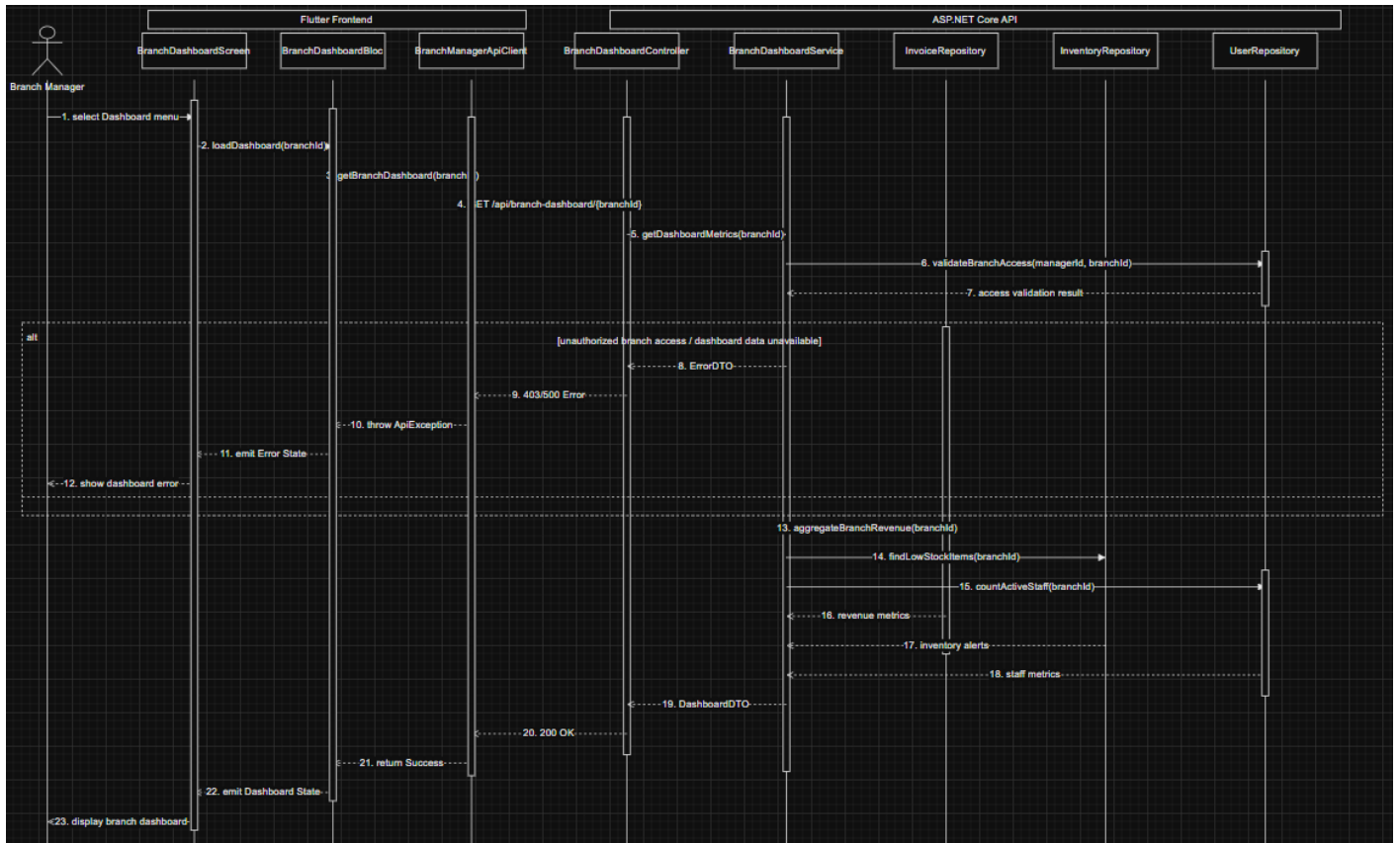


Figure 3.4.1.1: Sequence Diagram - UC09 View Branch Dashboard

3.4.2 UC10 View Branch Revenue Statistics

3.4.2.1 Class Diagram

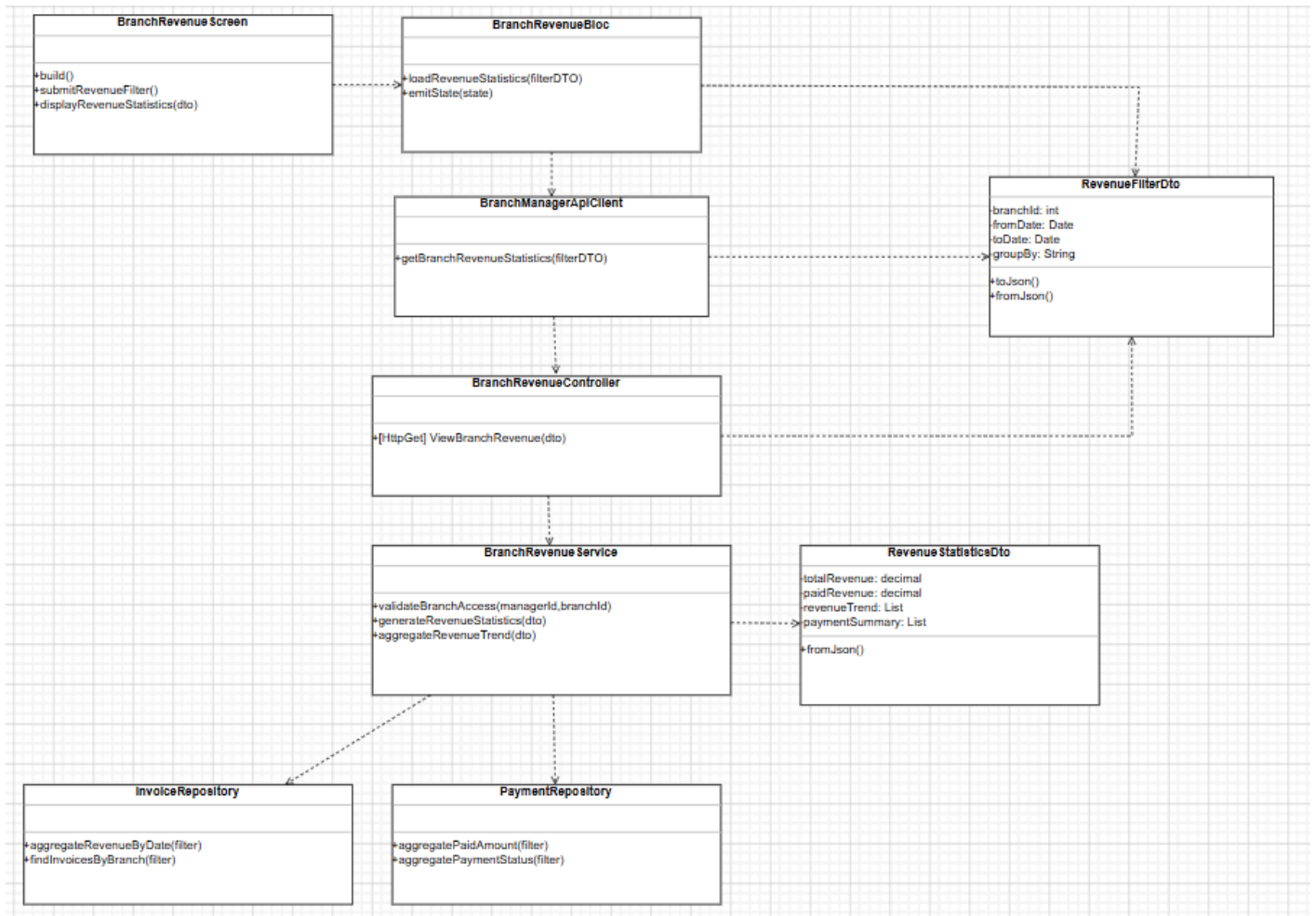


Figure 3.4.2.1: Class Diagram - UC10 View Branch Revenue Statistics

Class Specifications - UC10 View Branch Revenue Statistics

Class	Type	Main Responsibility	Main Operations	Related Data
BranchRevenueScreen	<<boundary>>	Displays branch revenue statistics and allows branch managers to filter reports by date range and grouping criteria.	build(); submitRevenueFilter(); displayRevenueStatistics(dto)	RevenueStatisticsDTO
BranchRevenueBloc	<<control>>	Manages revenue statistics state and coordinates communication between the presentation layer and API layer.	loadRevenueStatistics(filterDTO); emitState(state)	RevenueStatisticsDTO; RevenueFilterDTO
BranchManagerApiClient	<<boundary>>	Sends revenue statistics requests to backend services and receives responses.	getBranchRevenueStatistics(filter DTO)	RevenueStatisticsDTO; RevenueFilterDTO
BranchRevenueController	<<control>>	Receives branch revenue statistics requests and forwards them to the service layer for processing.	viewBranchRevenue(filterDTO)	RevenueStatisticsDTO; RevenueFilterDTO

BranchRevenueService	<<control>>	Validates branch access permissions, aggregates revenue information, and generates revenue statistics and trend analysis.	validateBranchAccess(managerId, branchId); generateRevenueStatistics(filterDTO); aggregateRevenueTrend(filterDTO)	Invoice; Payment
InvoiceRepository	<<entity>>	Provides invoice data and revenue aggregation information used for generating branch revenue reports.	aggregateRevenueByDate(filterDTO); findInvoicesByBranch(filterDTO)	INVOICE
PaymentRepository	<<entity>>	Provides payment information and payment status summaries for revenue analysis.	aggregatePaidAmount(filterDTO); aggregatePaymentStatus(filterDTO)	PAYMENT
RevenueStatisticsDTO	<<entity>>	Transfers branch revenue statistics data between application layers.	fromJson()	totalRevenue; paidRevenue; revenueTrend; paymentSummary
RevenueFilterDTO	<<entity>>	Carries filtering parameters used for generating branch revenue statistics reports.	toJson(); fromJson()	branchId; fromDate; toDate; groupBy

3.4.2.2 Sequence Diagram

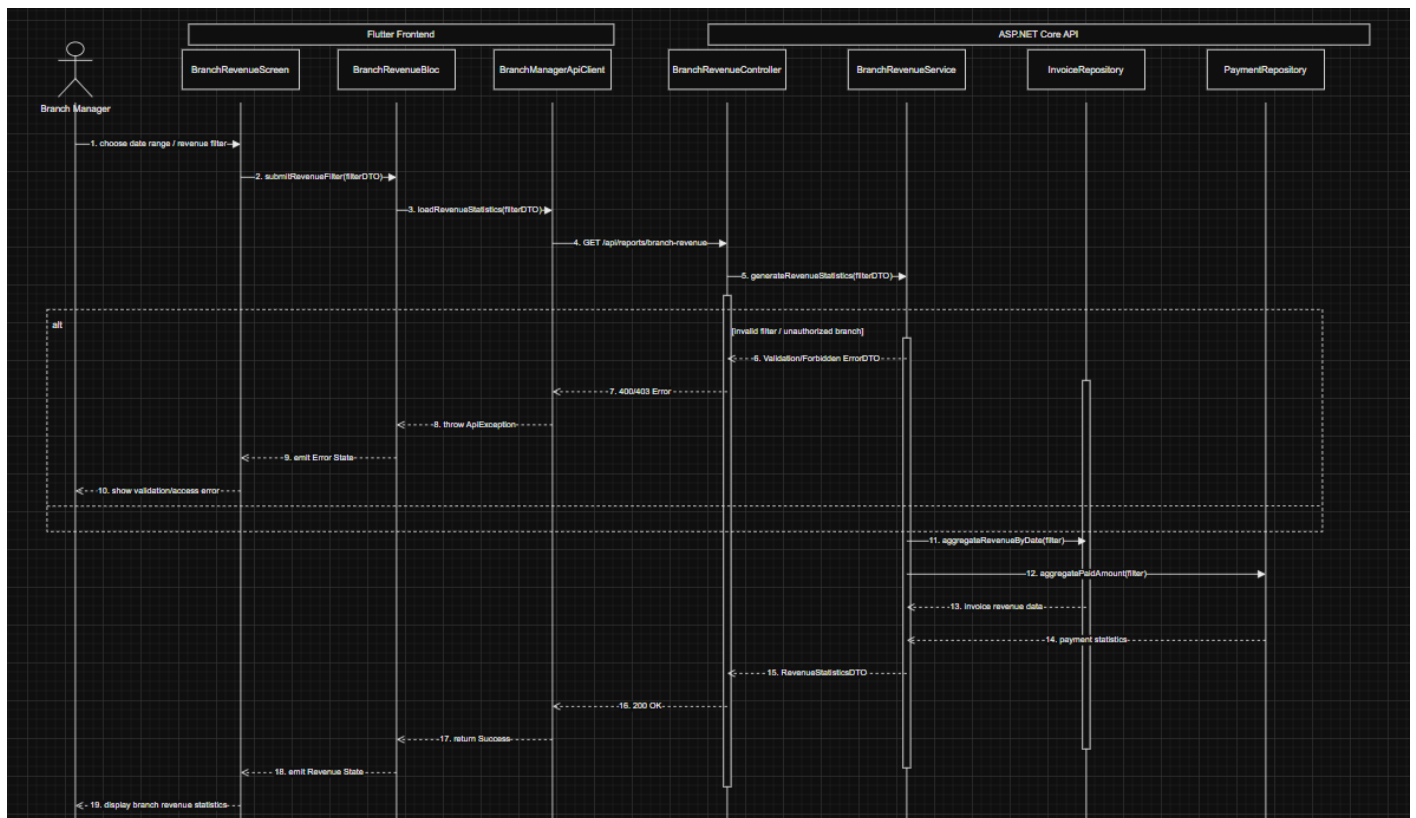


Figure 3.4.2.2: Sequence Diagram - UC10 View Branch Revenue Statistics

3.4.3 UC32 Monitor Staff Performance

3.4.3.1 Class Diagram

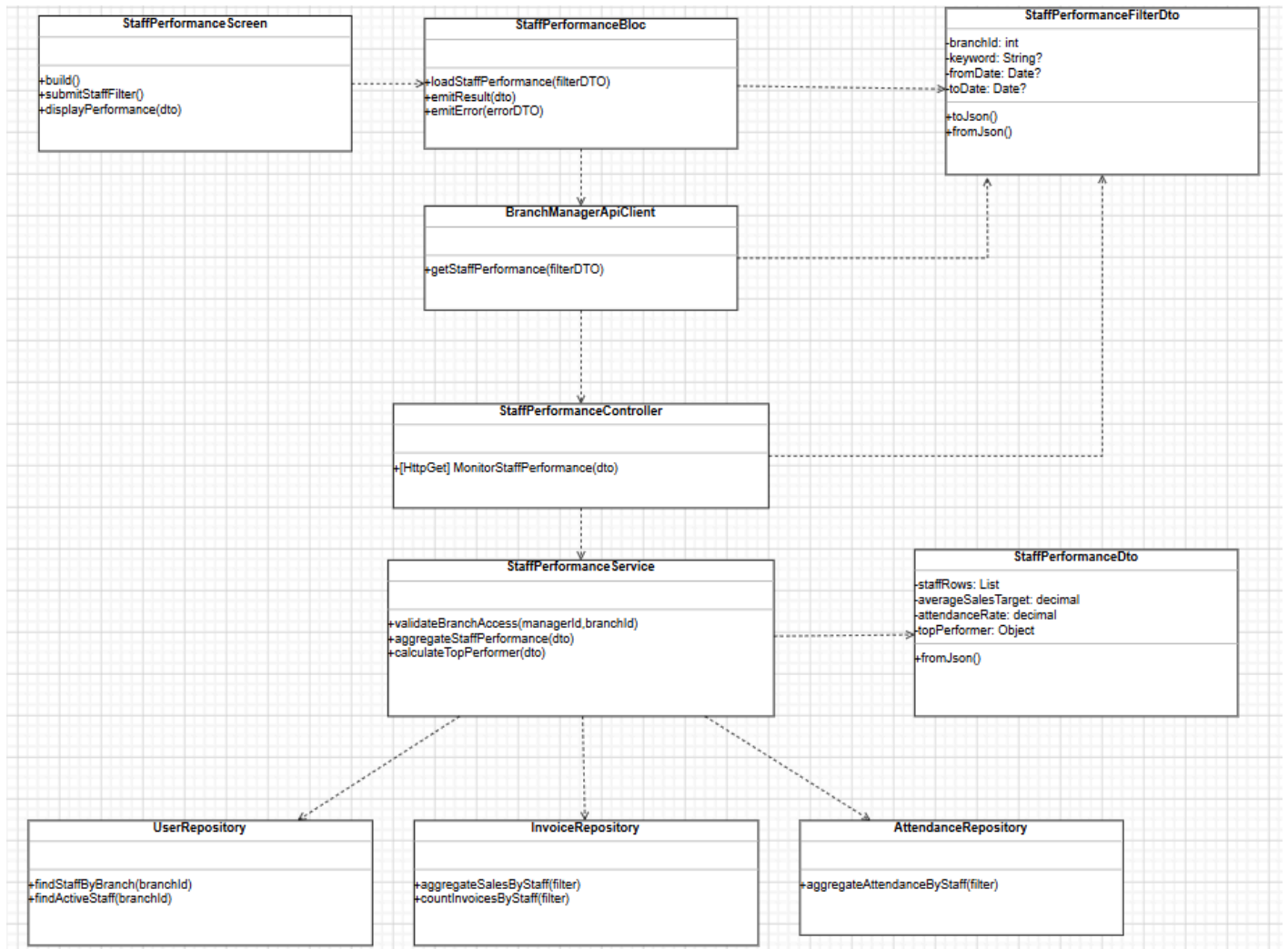


Figure 3.4.3.1: Class Diagram - UC32 Monitor Staff Performance

Class Specifications - UC32 Monitor Staff Performance

Class	Type	Main Responsibility	Main Operations	Related Data
StaffPerformanceScreen	<<boundary>>	Displays staff performance statistics and allows branch managers to filter staff performance reports by date range and keyword.	build(); submitStaffFilter(); displayPerformance(dto)	StaffPerformanceDTO
StaffPerformanceBloc	<<control>>	Manages staff performance state and coordinates communication between the presentation layer and API layer.	loadStaffPerformance(filterDTO); emitResult(dto); emitError(errorDTO)	StaffPerformanceDTO; StaffPerformanceFilterDTO
BranchManagerApiClient	<<boundary>>	Sends staff performance requests to backend services and receives responses.	getStaffPerformance(filterDTO)	StaffPerformanceDTO; StaffPerformanceFilterDTO

StaffPerformanceController	<<control>>	Receives staff performance requests and forwards them to the service layer for processing.	monitorStaffPerformance(filterDTO)	StaffPerformanceDTO; StaffPerformanceFilterDTO
StaffPerformanceService	<<control>>	Validates branch access permissions, aggregates staff performance information, calculates attendance and sales statistics, and identifies top-performing staff members.	validateBranchAccess(managerId, branchId); aggregateStaffPerformance(filterDTO); calculateTopPerformer(dto)	User; Invoice; Attendance
UserRepository	<<entity>>	Provides staff information belonging to a branch and retrieves active staff records for performance analysis.	findStaffByBranch(branchId); findActiveStaff(branchId)	USER
InvoiceRepository	<<entity>>	Provides invoice data and sales aggregation information used for staff performance evaluation.	aggregateSalesByStaff(filterDTO); countInvoicesByStaff(filterDTO)	INVOICE
AttendanceRepository	<<entity>>	Provides attendance information and attendance statistics used for staff performance evaluation.	aggregateAttendanceByStaff(filterDTO)	ATTENDANCE
StaffPerformanceDTO	<<entity>>	Transfers staff performance statistics data between application layers.	fromJson()	staffRows; averageSalesTarget; attendanceRate; topPerformer
StaffPerformanceFilterDTO	<<entity>>	Carries filtering parameters used for generating staff performance reports.	toJson(); fromJson()	branchId; keyword; fromDate; toDate

3.4.3.2 Sequence Diagram

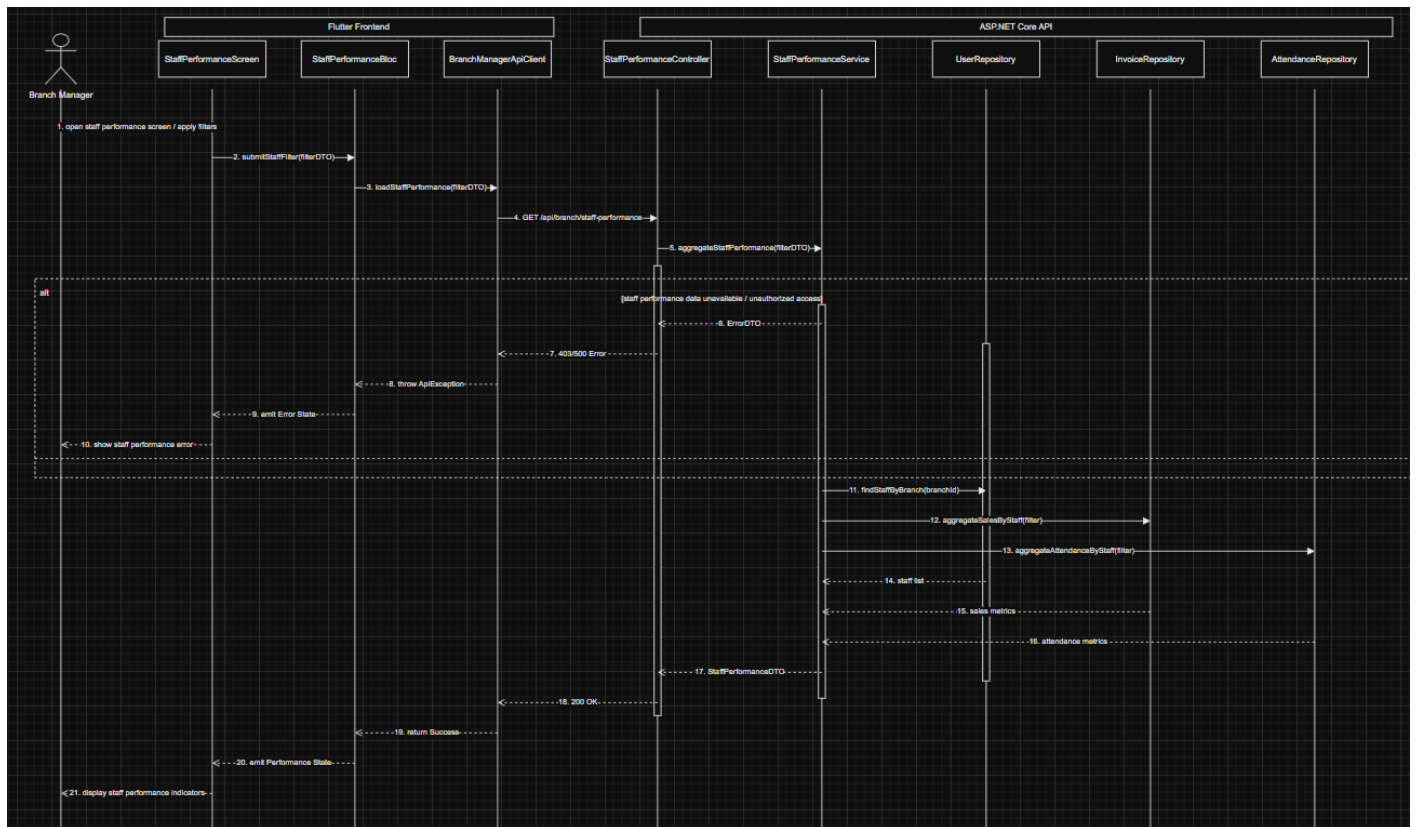


Figure 3.4.3.2: SequenceDiagram - UC32 Monitor Staff Performance

3.4.4 UC33 Monitor Branch Inventory

3.4.4.1 Class Diagram

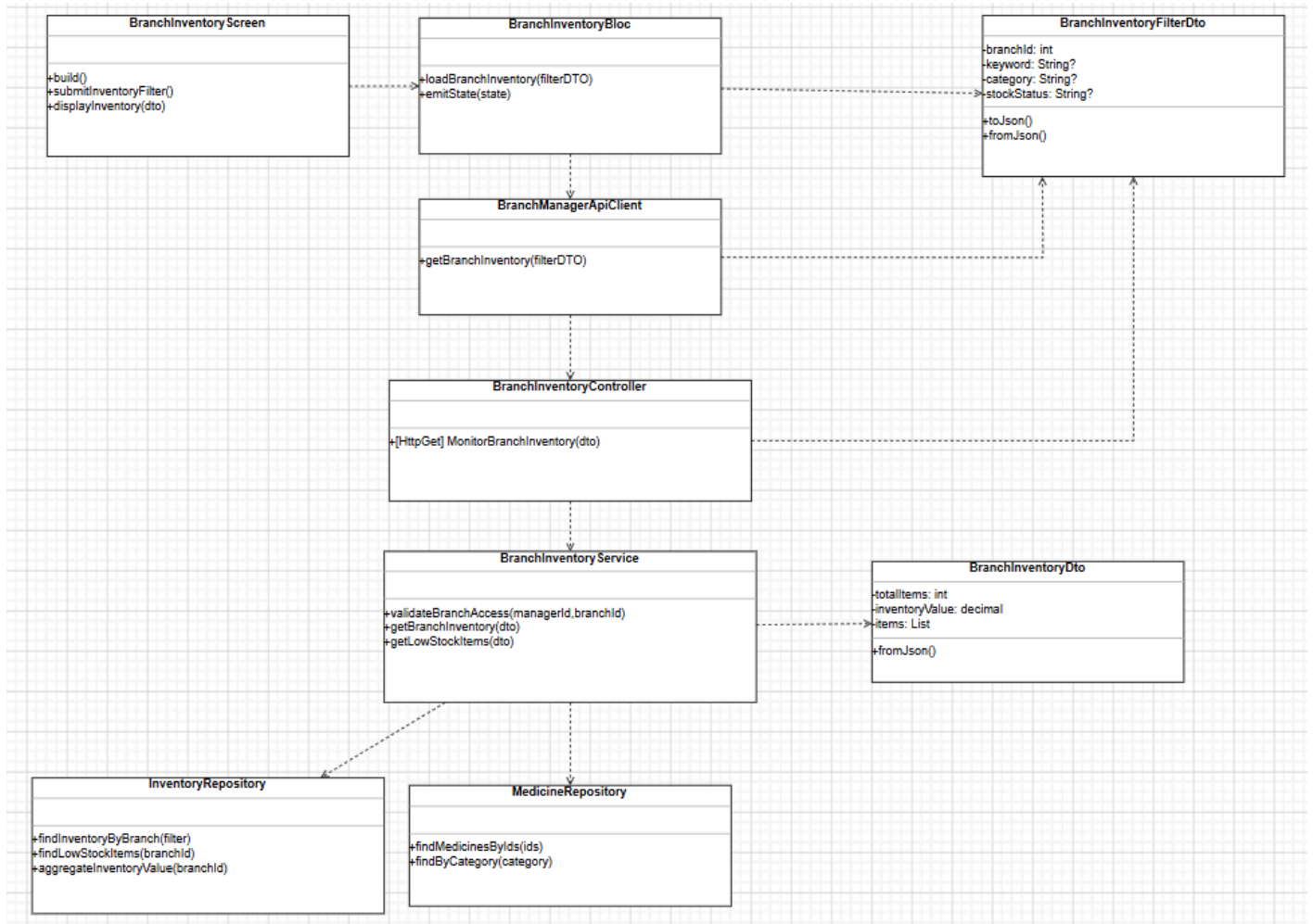


Figure 3.4.4.1: Class Diagram - UC33 Monitor Branch Inventory

Class Specifications - UC33 Monitor Branch Inventory

Class	Type	Main Responsibility	Main Operations	Related Data
BranchInventoryScreen	<<boundary>>	Displays branch inventory information and allows branch managers to filter inventory by category, keyword, and stock status.	build(); submitInventoryFilter(); displayInventory(dto)	BranchInventoryDTO
BranchInventoryBloc	<<control>>	Manages inventory monitoring state and coordinates communication between the presentation layer and API layer.	loadBranchInventory(filterDTO); emitState(state)	BranchInventoryDTO; BranchInventoryFilterDTO
BranchManagerApiClient	<<boundary>>	Sends inventory monitoring requests to backend services and receives responses.	getBranchInventory(filterDTO)	BranchInventoryDTO; BranchInventoryFilterDTO

BranchInventoryController	<<control>>	Receives inventory monitoring requests and forwards them to the service layer for processing.	monitorBranchInventory(filterDTO)	BranchInventoryDTO; BranchInventoryFilterDTO
BranchInventoryService	<<control>>	Validates branch access permissions, retrieves inventory information, calculates inventory value, and identifies low-stock items.	validateBranchAccess(managerId, branchId); getBranchInventory(filterDTO); getLowStockItems(filterDTO)	Inventory; Medicine
InventoryRepository	<<entity>>	Provides inventory data belonging to a branch and retrieves inventory statistics for monitoring purposes.	findInventoryByBranch(filterDTO); findLowStockItems(branchId); aggregateInventoryValue(branchId)	INVENTORY
MedicineRepository	<<entity>>	Provides medicine information used to enrich inventory monitoring results and category filtering.	findMedicinesByIds(ids); findByCategory(category)	MEDICINE
BranchInventoryDTO	<<entity>>	Transfers branch inventory statistics and inventory details between application layers.	fromJson()	totalItems; inventoryValue; items
BranchInventoryFilterDTO	<<entity>>	Carries filtering parameters used for inventory monitoring requests.	toJson(); fromJson()	branchId; keyword; category; stockStatus

3.4.4.2 Sequence Diagram

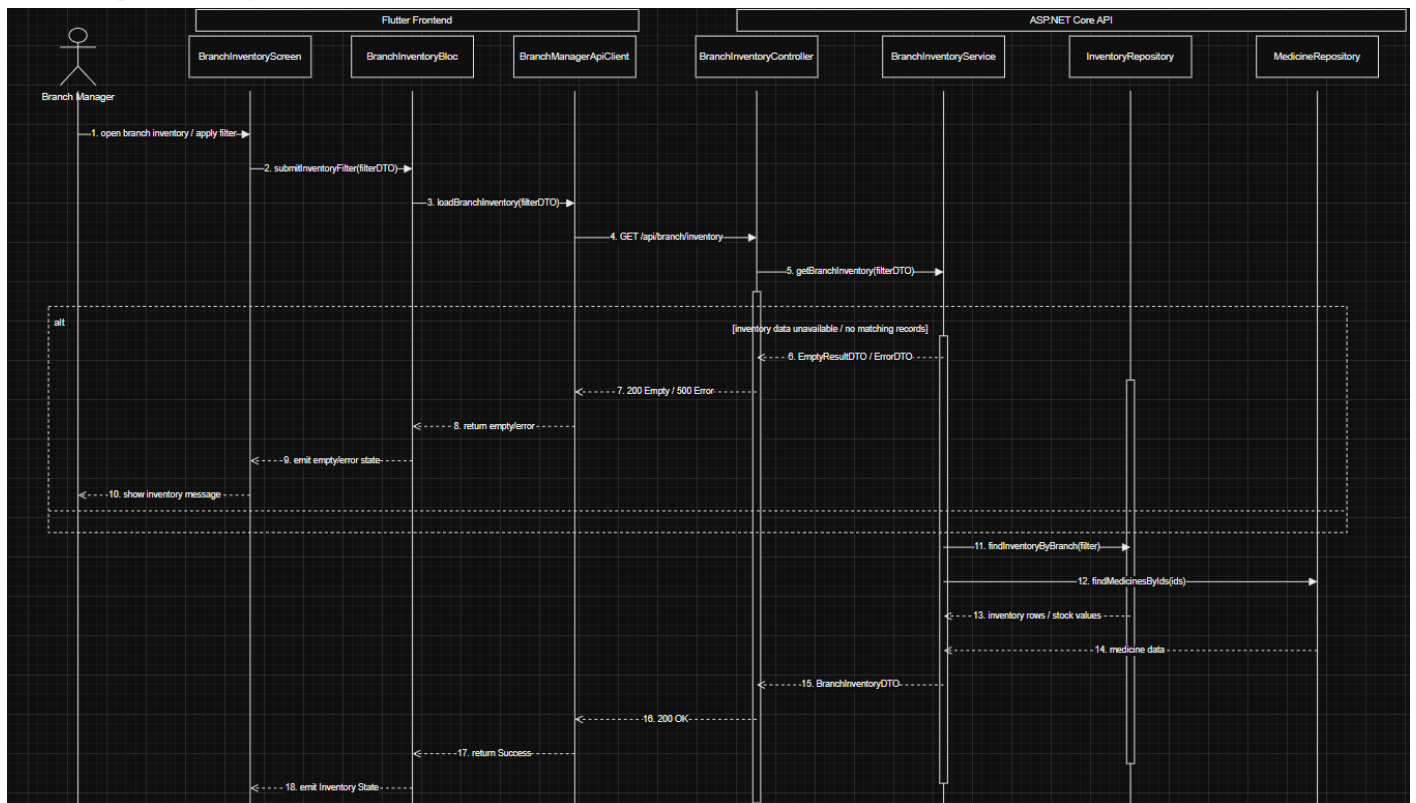


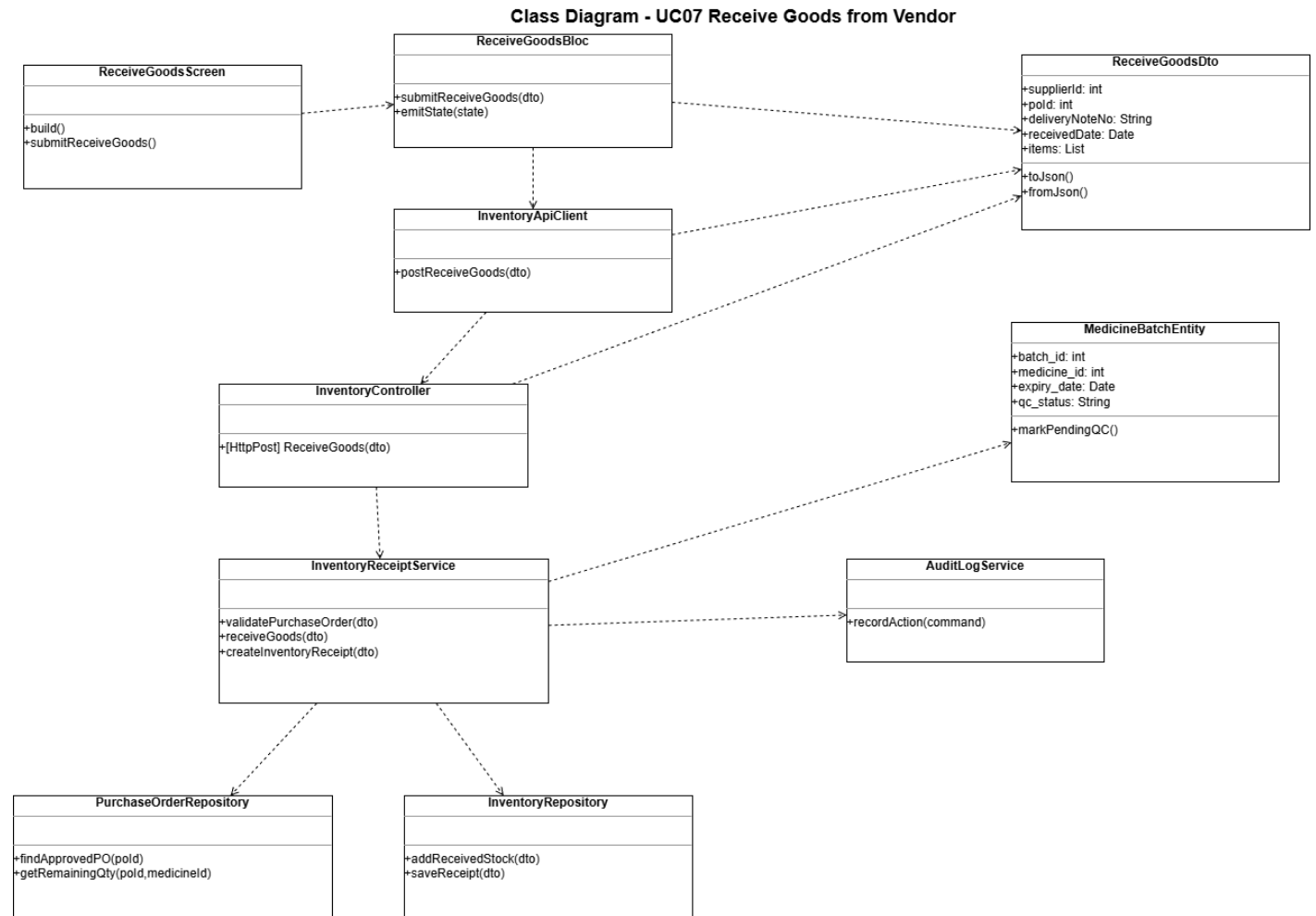
Figure 3.4.4.2: Sequence Diagram - UC33 Monitor Branch Inventory

3.5 Inventory Management

Inventory Management includes Receive Goods from Vendor, QC Inspection, Issue Stock to Stores, Approve Internal Transfers, Perform Stocktake, Set Safety Stock Levels, Identify Expired/Damaged Stock, Track Batch & Lot Number, and View Inventory Reports. The design supports FEFO and batch traceability.

3.5.1. UC07 Receive Goods from Vendor

3.5.1.1 Class Diagram

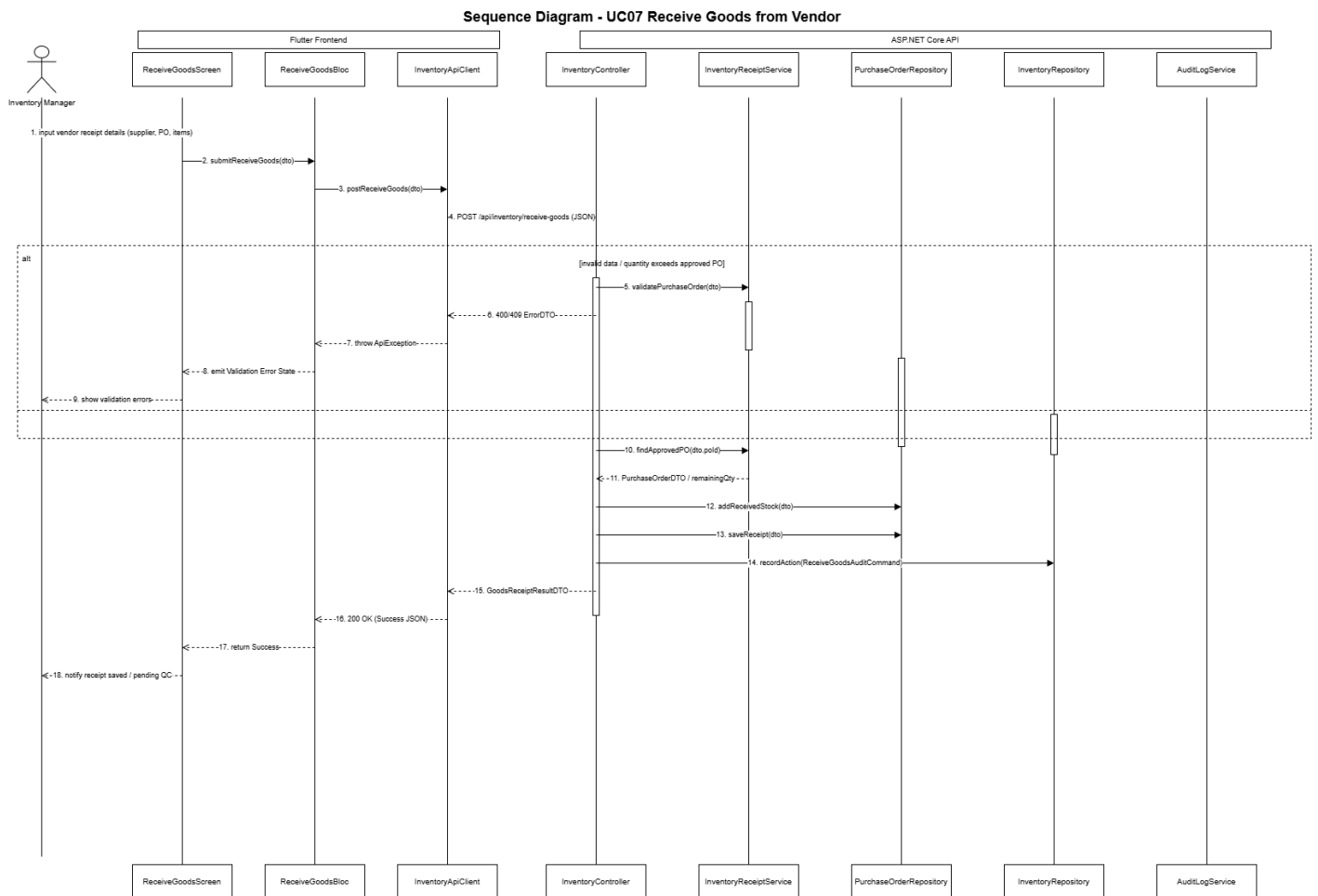


3.5.1.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
ReceiveGoodsScreen	<<boundary>>	Provides interface for Inventory Manager to input vendor receipt details (supplier, PO, items).	build(); submitReceiveGoods()	N/A
ReceiveGoodsBloc	<<control>>	Coordinates frontend workflow for stock receiving, handles state emissions, and dispatches requests.	submitReceiveGoods(dto); emitState(state)	ReceiveGoodsDto
InventoryApiClient	<<boundary>>	Sends HTTP POST payloads for receiving vendor	postReceiveGoods(dto)	ReceiveGoodsDto

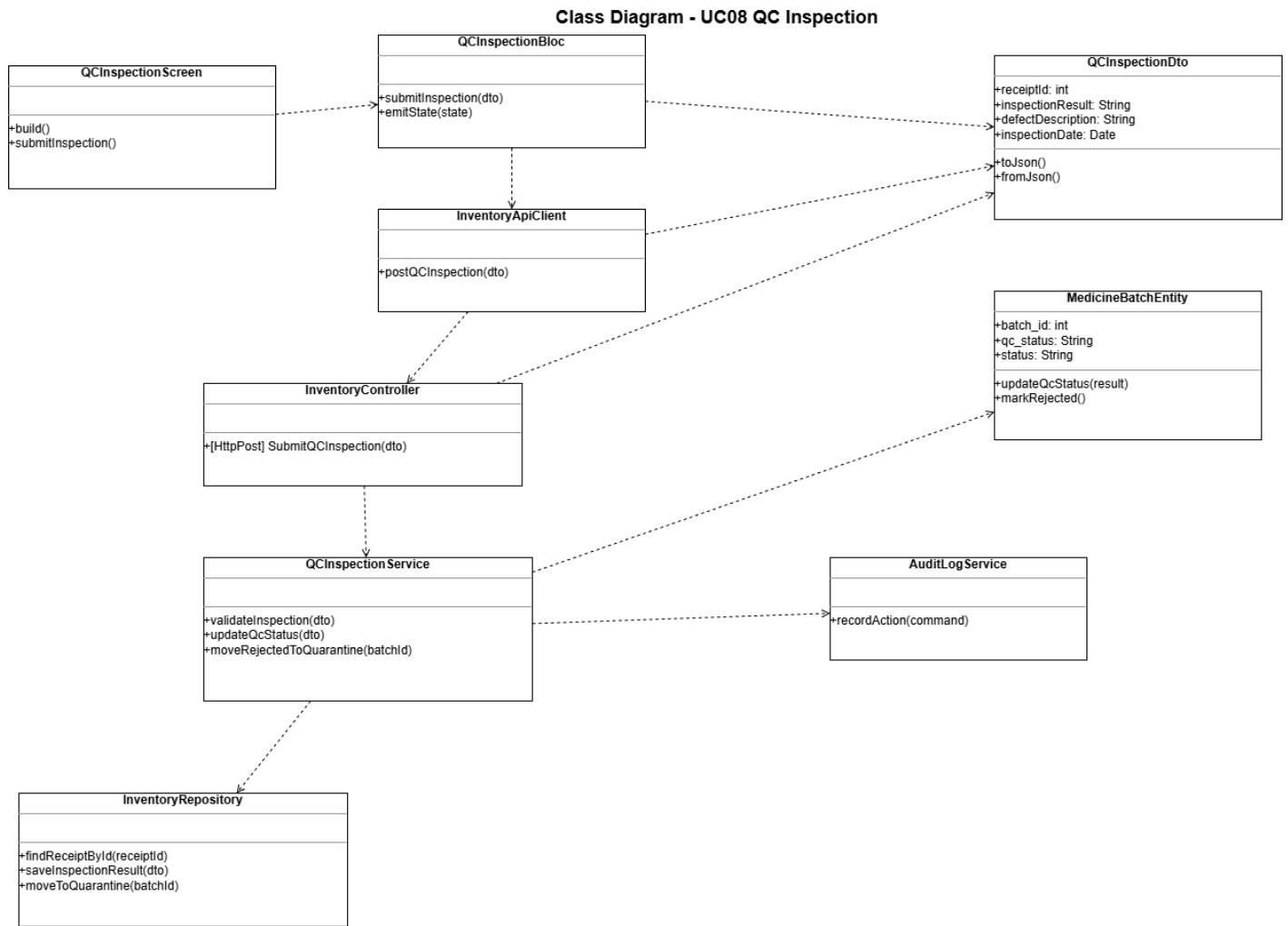
		shipments to backend controllers.		
ReceiveGoodsDto	<<DTO>>	Carries receiving session logs, supplier references, and item batches across application boundaries.	toJson(); fromJson()	supplierId: int; pold: int; deliveryNoteNo: String; receivedDate: Date; items: List
InventoryController	<<control>>	Exposes backend endpoints to accept inbound material receipts and delegates them to core logic.	[HttpPost] ReceiveGoods(dto)	ReceiveGoodsDto
InventoryReceiptService	<<control>>	Handles vendor order cross-verifications, registers incoming batch metadata, and generates physical receipts.	validatePurchaseOrder(dto); receiveGoods(dto); createInventoryReceipt(dto)	ReceiveGoodsDto; MedicineBatchEntity
PurchaseOrderRepository	<<interface>>	Manages query paths to validate approved source purchase orders and compute leftover open quantities.	findApprovedPO(pold); getRemainingQty(pold, medicineId)	N/A
InventoryRepository	<<interface>>	Performs main database insertions to append raw material levels and record concrete receipt bills.	addReceivedStock(dto); saveReceipt(dto)	N/A
AuditLogService	<<control>>	Registers secure historical traces confirming new ledger entries coming from purchase operations.	recordAction(command)	ReceiveGoodsAuditCommand
MedicineBatchEntity	<<entity>>	Represents newly arrived production components inside storage locations waiting for clearance.	markPendingQC()	batch_id: int; medicine_id: int; expiry_date: Date; qc_status: String

3.5.1.3 Approve Internal Transfers Sequence Diagram



3.5.2. UC08 QC Inspection

3.5.2.1 Class Diagram

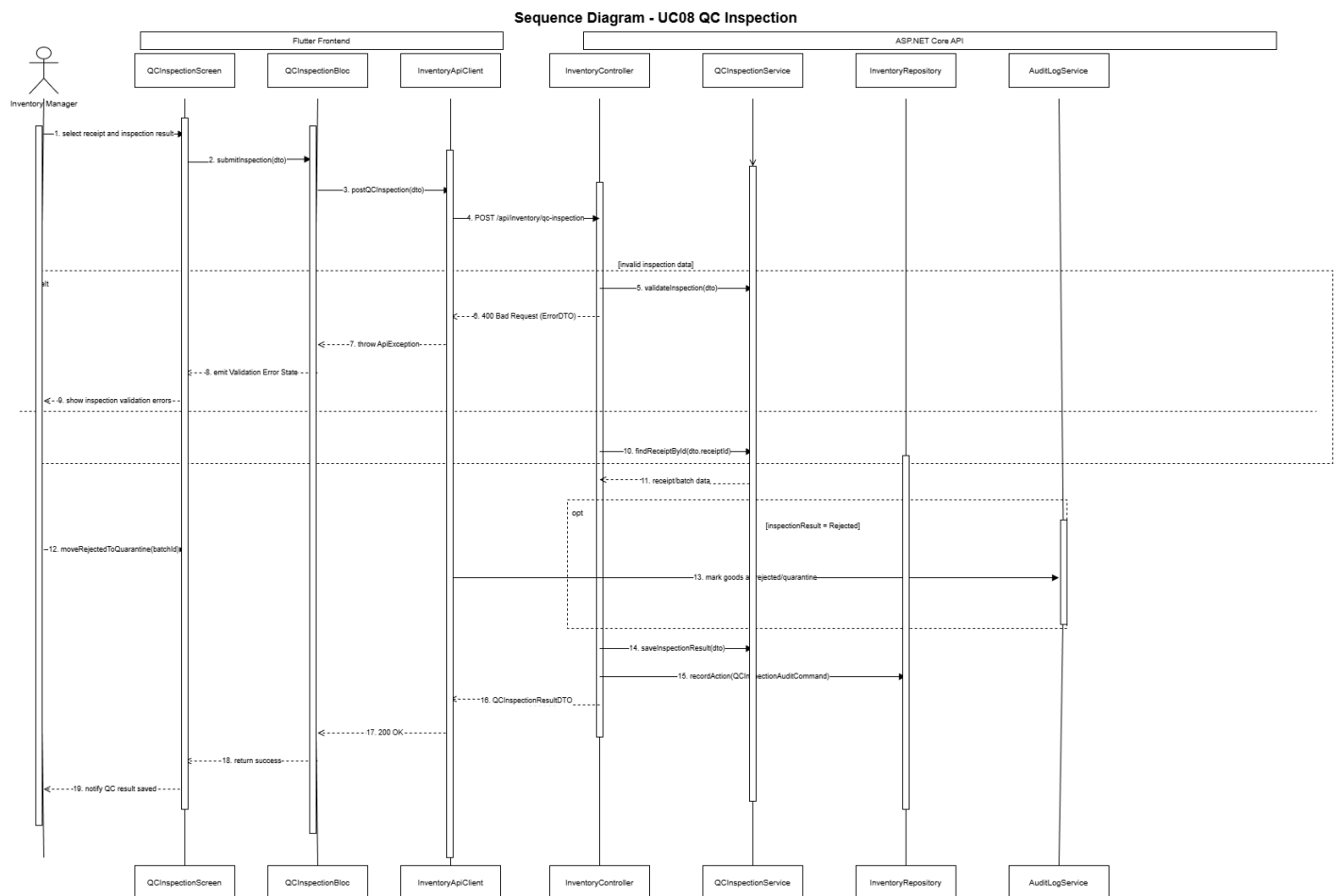


3.5.2.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
QCInspectionScreen	<<boundary>>	Provides interface for Inventory Manager to select receipts, enter inspection outcomes, and submit QC details.	build(); submitInspection()	N/A
QCInspectionBloc	<<control>>	Coordinates frontend presentation state changes for quality control steps and dispatches validation signals.	submitInspection(dto); emitState(state)	QCInspectionDto
InventoryApiClient	<<boundary>>	Transmits HTTP POST requests packed with quality verification records to backend API systems.	postQCInspection(dto)	QCInspectionDto
QCInspectionDto	<<DTO>>	Encapsulates receipt identifiers, quality flags, defect descriptions, and timestamps between boundaries.	toJson(); fromJson()	receiptId: int; inspectionResult: String; defectDescription: String; inspectionDate: Date

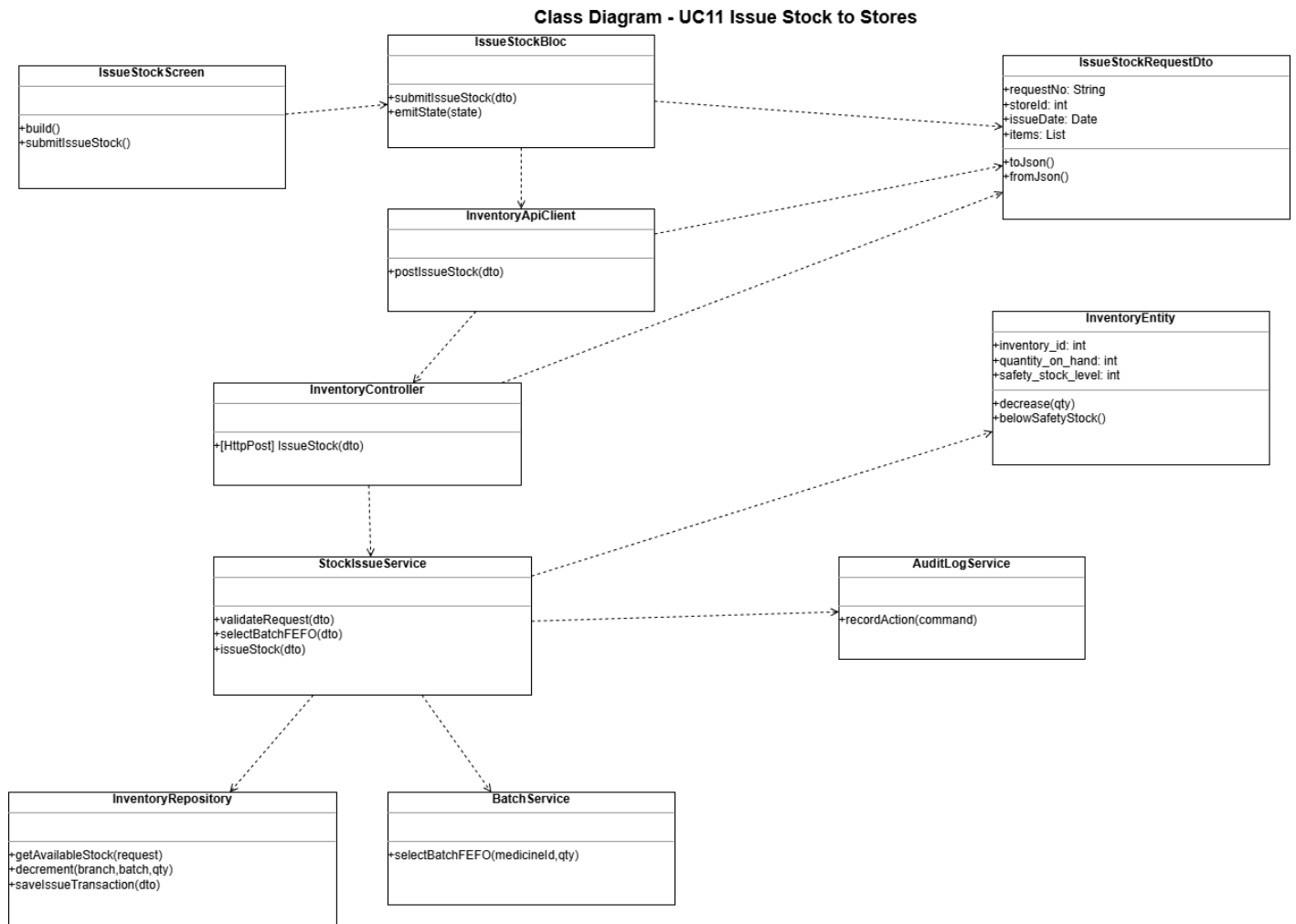
InventoryController	<<control>>	Receives external API quality inspection payloads and routes them to appropriate application layer components.	[HttpPost] SubmitQCInspection(dto)	QCInspectionDto
QCInspectionService	<<control>>	Orchestrates quality control procedures: checks inputs, updates lot data, and handles quarantine movements for rejected batches.	validateInspection(dto); updateQcStatus(dto); moveRejectedToQuarantine(batchId)	QCInspectionDto; MedicineBatchEntity
InventoryRepository	<<interface>>	Directs persistent database methods to access receipt info, save evaluation status, and isolate defective products.	findReceiptById(receiptId); saveInspectionResult(dto); moveToQuarantine(batchId)	MedicineBatchEntity
AuditLogService	<<control>>	Chronicles official quality approvals and material isolation changes into system history tables.	recordAction(command)	QCInspectionAuditCommand
MedicineBatchEntity	<<entity>>	Holds structural quality properties and statuses of an isolated batch, enforcing state modification logic.	updateQcStatus(result); markRejected()	batch_id: int; qc_status: String; status: String

3.5.2.3 QC Inspection Sequence Diagram



3.5.3. UC11 Issue Stock to Stores

3.5.3.1 Class Diagram

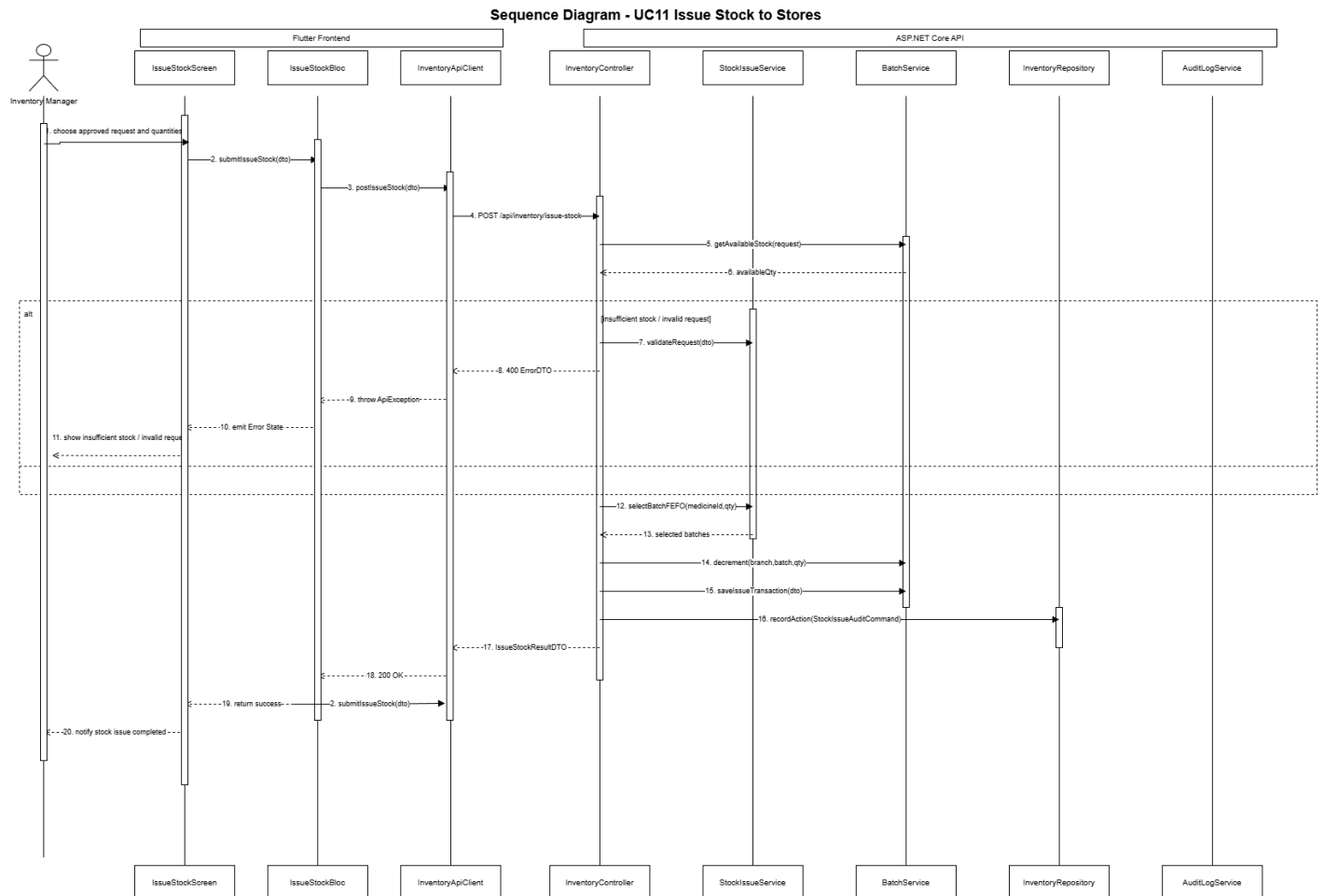


3.5.3.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
IssueStockScreen	<<boundary>>	Provides interface for Inventory Manager to select approved requests, set quantities, and dispatch stock issues.	build(); submitIssueStock()	N/A
IssueStockBloc	<<control>>	Handles frontend presentation states for issuing processes and controls submission triggers.	submitIssueStock(dto); emitState(state)	IssueStockRequestDto
InventoryApiClient	<<boundary>>	Forwards HTTP POST payloads containing stock issue transactions to target api controllers.	postIssueStock(dto)	IssueStockRequestDto
IssueStockRequestDto	<<DTO>>	Carries target store info, issue numbers, dates, and item line arrays across processing boundaries.	toJson(); fromJson()	requestNo: String; storeId: int; issueDate: Date; items: List
InventoryController	<<control>>	Exposes backend endpoints for handling stock distribution,	[HttpPost] IssueStock(dto)	IssueStockRequestDto

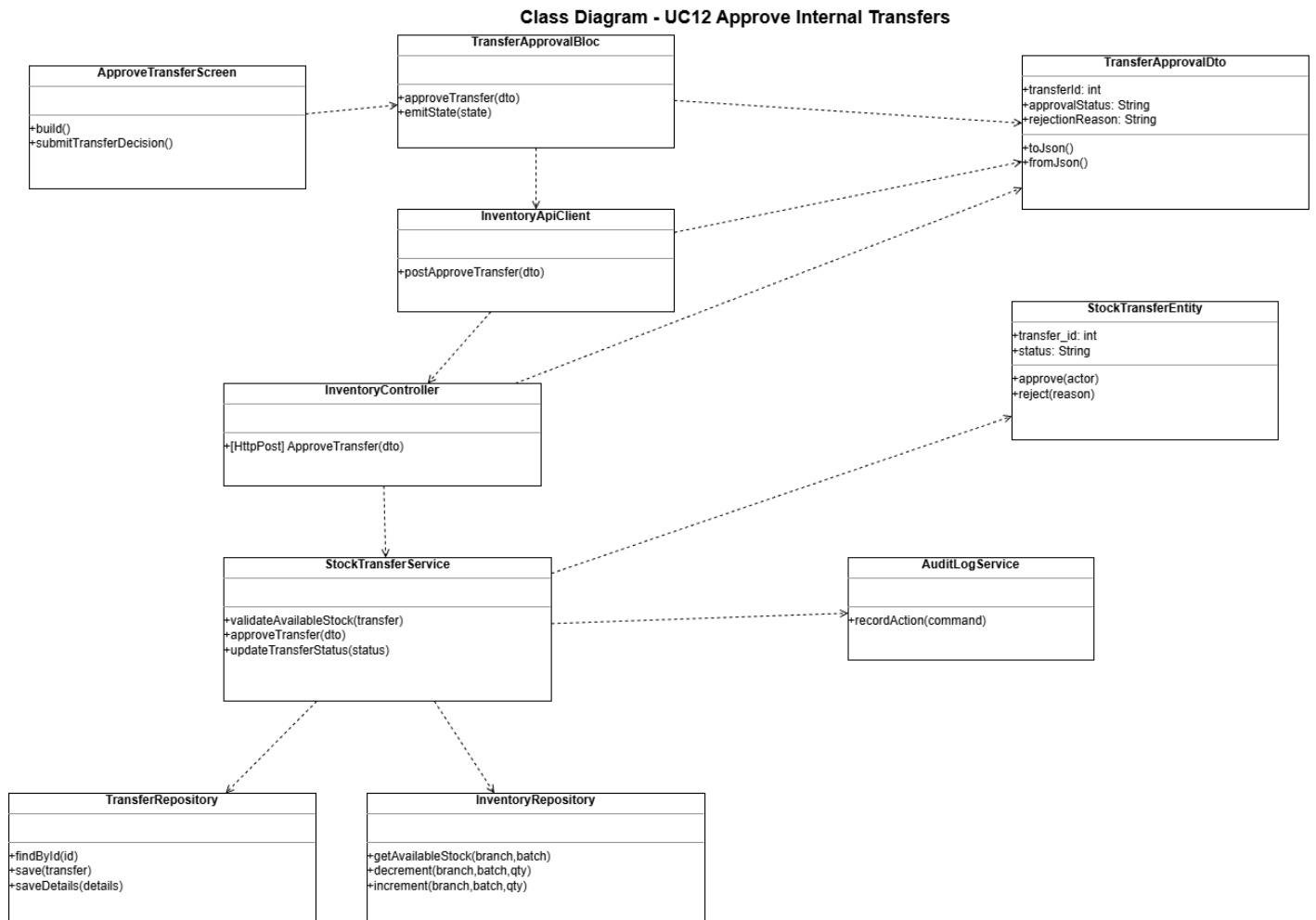
		routing details to core business logic.		
StockIssueService	<<control>>	Manages distribution validations, coordinates FEFO batch strategies, and executes actual repository decrements.	validateRequest(dto); selectBatchFEFO(dto); issueStock(dto)	IssueStockRequestDto; InventoryEntity
BatchService	<<control>>	Implements specialized FEFO (First Expired, First Out) rules to evaluate and query ideal medicine batches.	selectBatchFEFO(medicineId, qty)	N/A
InventoryRepository	<<interface>>	Performs primary database mutations to lower physical counts and save complete distribution transaction logs.	getAvailableStock(request); decrement(branch, batch, qty); saveIssueTransaction(dto)	InventoryEntity
AuditLogService	<<control>>	Persists historical logging commands reflecting product distribution actions for downstream accountability.	recordAction(command)	StockIssueAuditCommand
InventoryEntity	<<entity>>	Models physical slot dimensions, controlling current on-hand levels and executing basic threshold checks.	decrease(qty); belowSafetyStock()	inventory_id: int; quantity_on_hand: int; safety_stock_level: int

3.5.3.3 Issue Stock to Stores Sequence Diagram



3.5.4. UC12 Approve Internal Transfers

3.5.4.1 Class Diagram

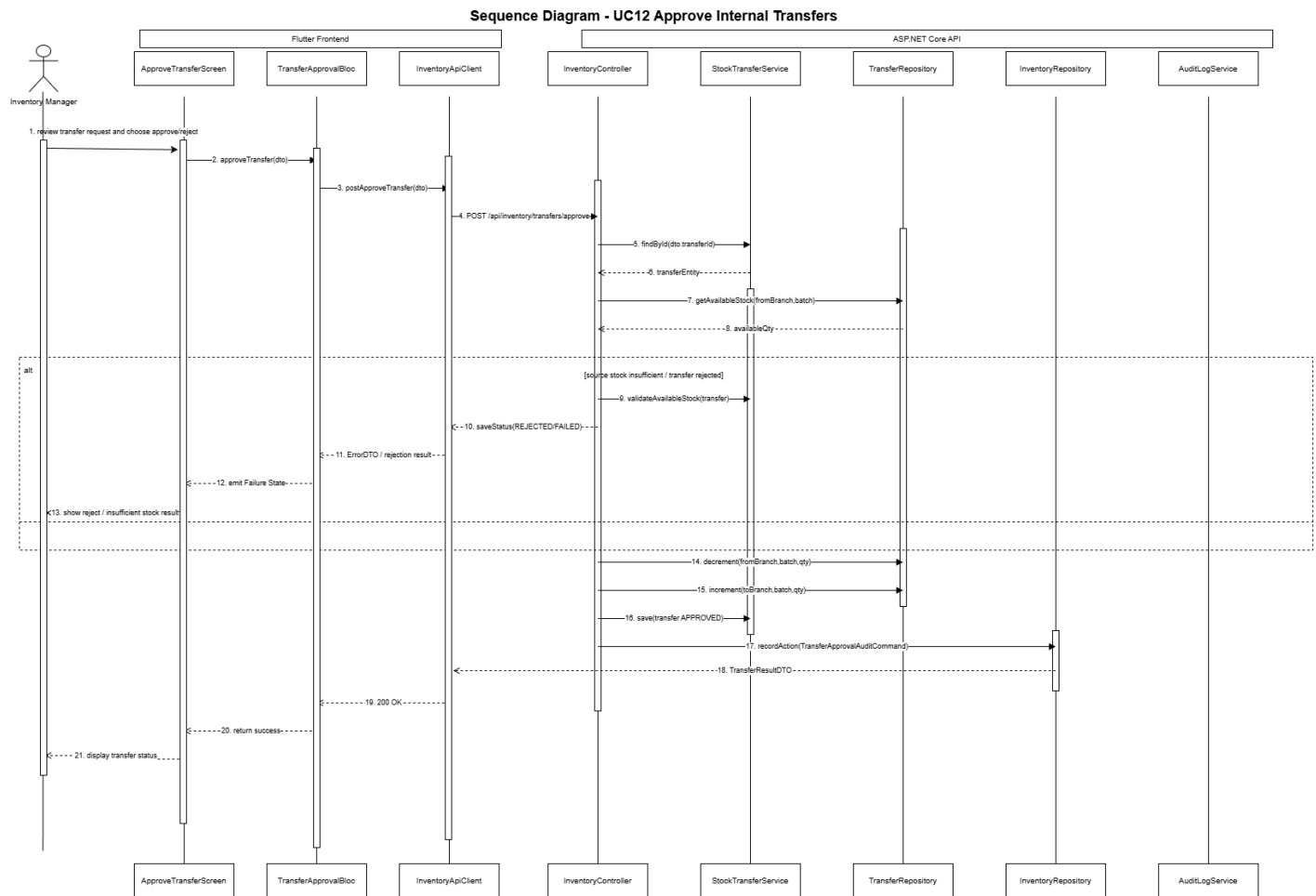


3.5.4.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
ApproveTransferScreen	<<boundary>>	Provides interface for Inventory Manager to review transfer requests and submit approval or rejection decisions.	build(); submitTransferDecision()	N/A
TransferApprovalBloc	<<control>>	Coordinates frontend approval workflows, handles state updates, and triggers action submissions.	approveTransfer(dto); emitState(state)	TransferApprovalDto
InventoryApiClient	<<boundary>>	Forwards HTTP POST approval decisions and transaction status payloads to backend API endpoints.	postApproveTransfer(dto)	TransferApprovalDto
TransferApprovalDto	<<DTO>>	Carries transfer approval data details, statuses, and reasons between application layers.	toJson(); fromJson()	transferId: int; approvalStatus: String; rejectionReason: String
InventoryController	<<control>>	Exposes backend API endpoint to handle inbound stock transfer verification and authorization actions.	[HttpPost] ApproveTransfer(dto)	TransferApprovalDto

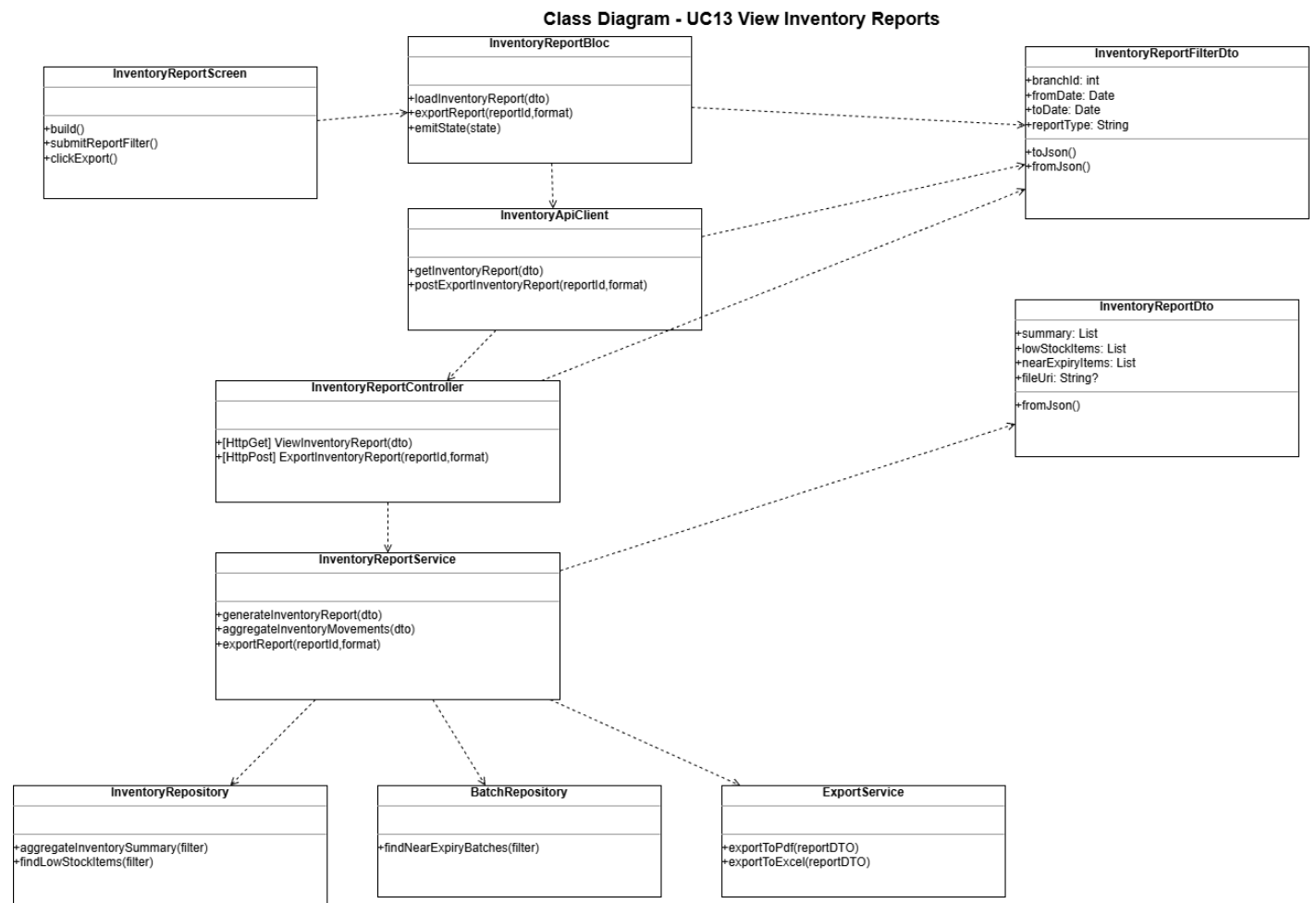
StockTransferService	<<control>>	Manages internal logic rules: stock availability validations, state transitions, repository persistences, and logs changes.	validateAvailableStock(transfer); approveTransfer(dto); updateTransferStatus(status)	TransferApprovalDto; StockTransferEntity
TransferRepository	<<interface>>	Provides abstraction layer to query transfer request metadata, save master states, and detailed line records.	findById(id); save(transfer); saveDetails(details)	StockTransferEntity
InventoryRepository	<<interface>>	Orchestrates persistent queries to assert safe storage quantities and physically decrement/increment batch figures.	getAvailableStock(branch, batch); decrement(branch, batch, qty); increment(branch, batch, qty)	N/A
AuditLogService	<<control>>	Records physical stock balancing results and formal management permissions into historical logging ledgers.	recordAction(command)	TransferApprovalAuditComm and
StockTransferEntity	<<entity>>	Encapsulates structural identifiers, workflow states, and processing logic for internal warehouse stock reallocations.	approve(actor); reject(reason)	transfer_id: int; status: String

3.5.4.3 Approve Internal Transfers Sequence Diagram



3.5.5. UC13 View Inventory Reports

3.5.5.1 Class Diagram

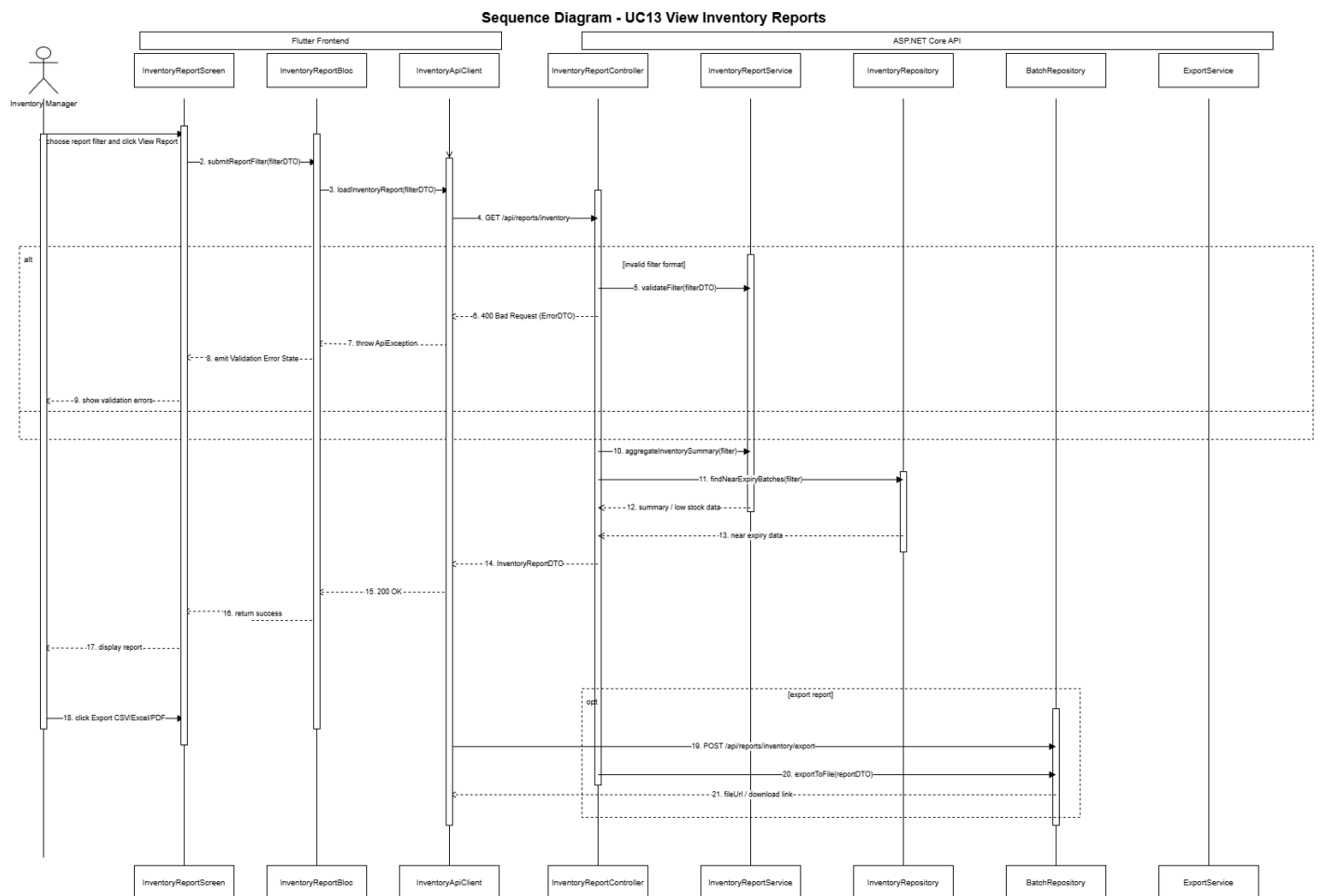


3.5.5.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
InventoryReportScreen	<<boundary>>	Renders report filters, triggers view summaries, and processes file download/export clicks.	build(); submitReportFilter(); clickExport()	N/A
InventoryReportBloc	<<control>>	Drives presentation logic for rendering data visualizations, analytical grids, and asset export triggers.	loadInventoryReport(dto); exportReport(reportId, format); emitState(state)	InventoryReportFilterDto
InventoryApiClient	<<boundary>>	Triggers HTTP network channels to fetch live operational counts or requests compiled documents.	getInventoryReport(dto); postExportInventoryReport(reportId, format)	InventoryReportFilterDto
InventoryReportFilterDto	<<DTO>>	Houses search scopes, chronological bounds, and specialized compilation classifications.	toJson(); fromJson()	branchId: int; fromDate: Date; toDate: Date; reportType: String
InventoryReportController	<<control>>	Exposes high-level reporting API end-points to compile logs or output spreadsheet files.	[HttpGet] ViewInventoryReport(dto); [HttpPost] ExportInventoryReport(reportId, format)	InventoryReportFilterDto; InventoryReportDto

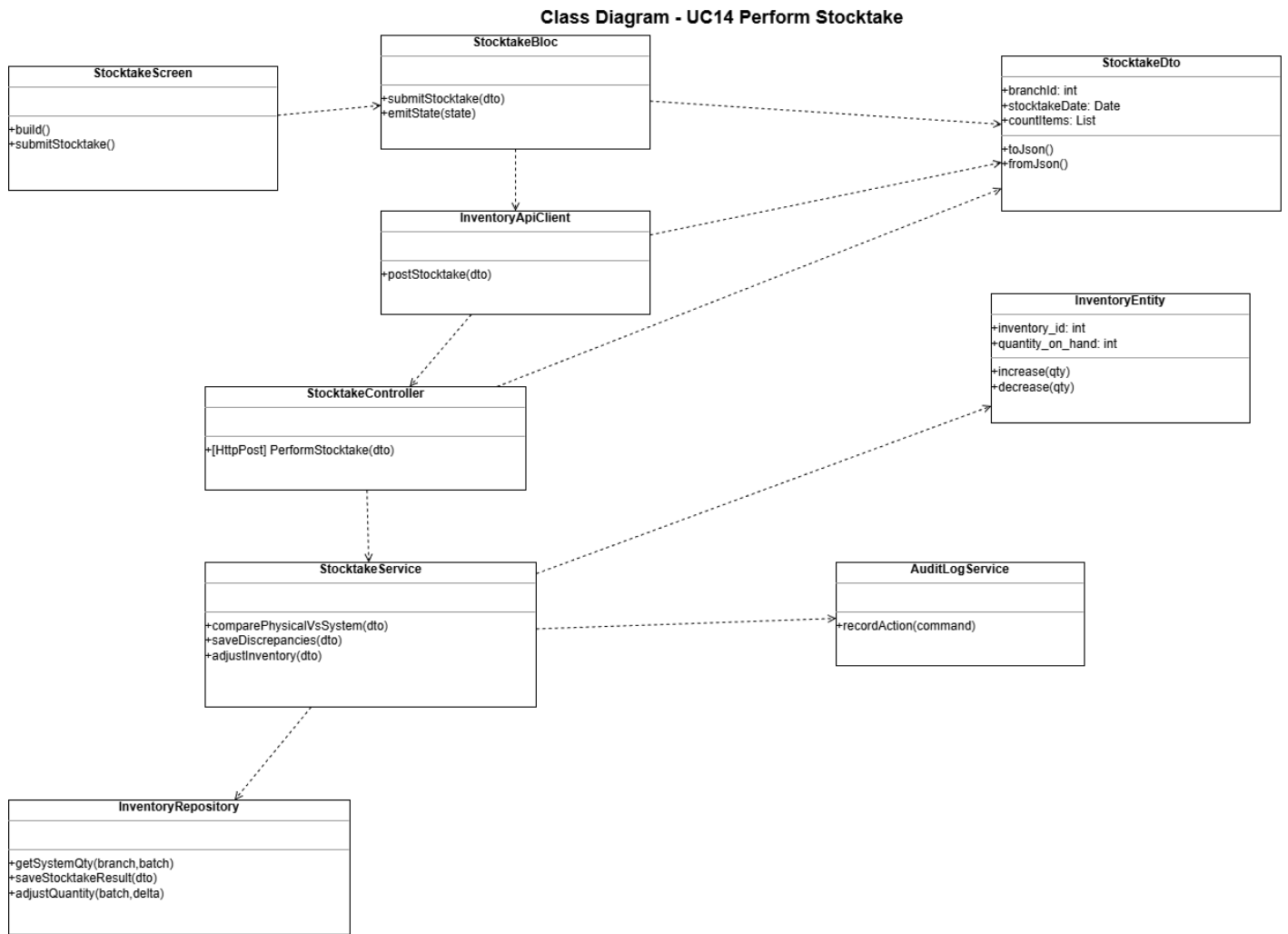
InventoryReportService	<<control>>	Aggregates material distributions, flags low stock bounds, detects nearing expirations, and triggers doc compiler.	generateInventoryReport(dto); aggregateInventoryMovements(dto); exportReport(reportId, format)	InventoryReportFilterDto; InventoryReportDto
InventoryRepository	<<interface>>	Queries structured data targets to construct inventory summary snapshots and retrieve low stock metrics.	aggregateInventorySummary(filter); findLowStockItems(filter)	N/A
BatchRepository	<<interface>>	Resolves targeted database inquiries focused on listing and identifying stock lots nearing maturity caps.	findNearExpiryBatches(filter)	N/A
ExportService	<<control>>	Adapts compiled internal raw system datasets into clean, human-readable PDF or Excel assets.	exportToPdf(reportDTO); exportToExcel(reportDTO)	InventoryReportDto
InventoryReportDto	<<DTO>>	Bundles operational indices, close-out records, and generated download uniform links back to clients.	fromJson()	summary: List; lowStockItems: List; nearExpiryItems: List; fileUri: String?

3.5.5.3 View Inventory Reports Sequence Diagram



3.5.6. UC14 Perform Stocktake

3.5.6.1 Class Diagram

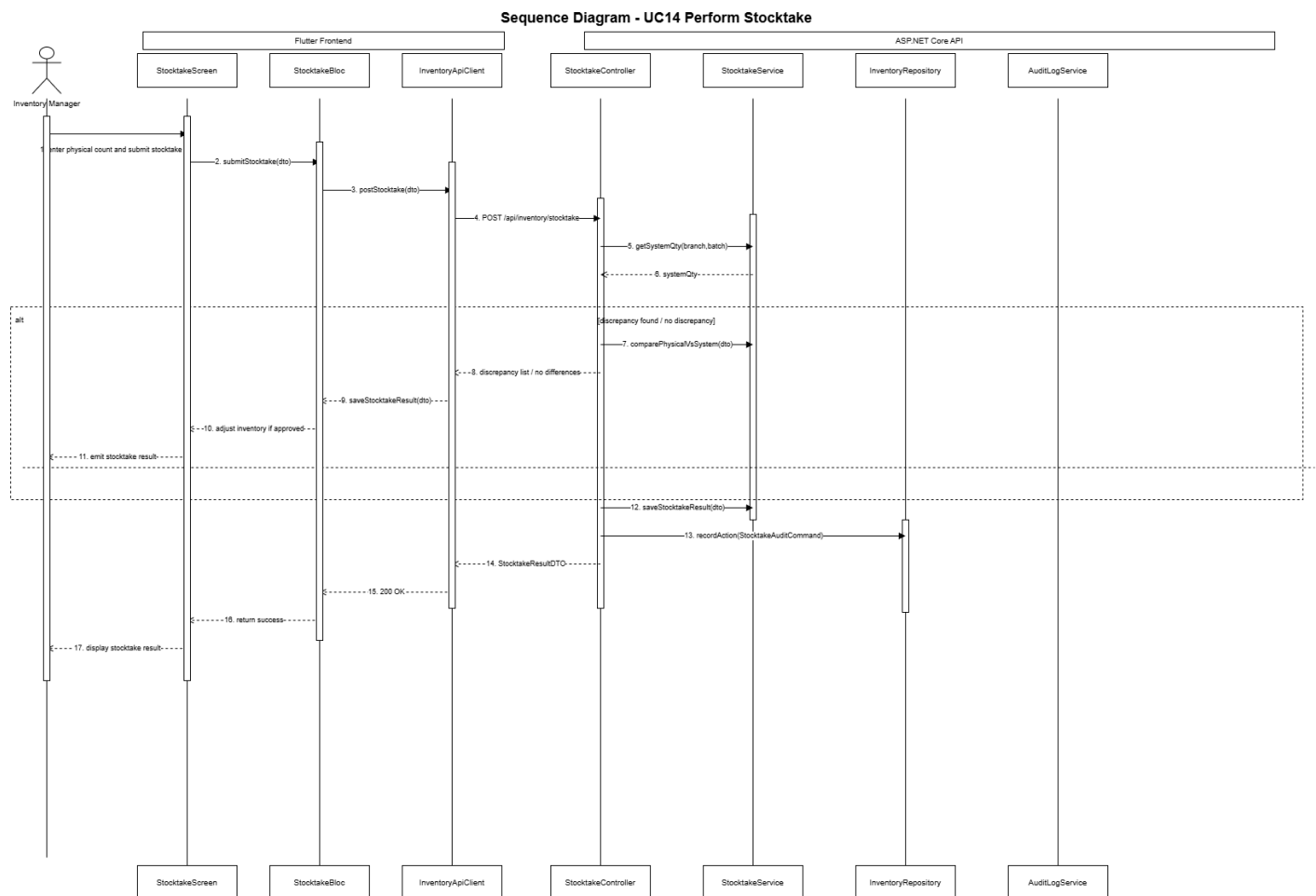


3.5.6.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
StocktakeScreen	<<boundary>>	Provides interface for Inventory Manager to enter physical count items and submit stocktake results.	build(); submitStocktake()	N/A
StocktakeBloc	<<control>>	Handles UI state management for stocktaking operations and coordinates form submissions.	submitStocktake(dto); emitState(state)	StocktakeDto
InventoryApiClient	<<boundary>>	Transmits HTTP POST stocktake data payloads to the target backend API service endpoint.	postStocktake(dto)	StocktakeDto
StocktakeDto	<<DTO>>	Packs stocktake snapshot details, timestamps, and item arrays for communication between layers.	toJson(); fromJson()	branchId: int; stocktakeDate: Date; countItems: List
StocktakeController	<<control>>	Exposes backend API endpoint to accept stocktake logs and coordinates with the logic layer.	[HttpPost] PerformStocktake(dto)	StocktakeDto

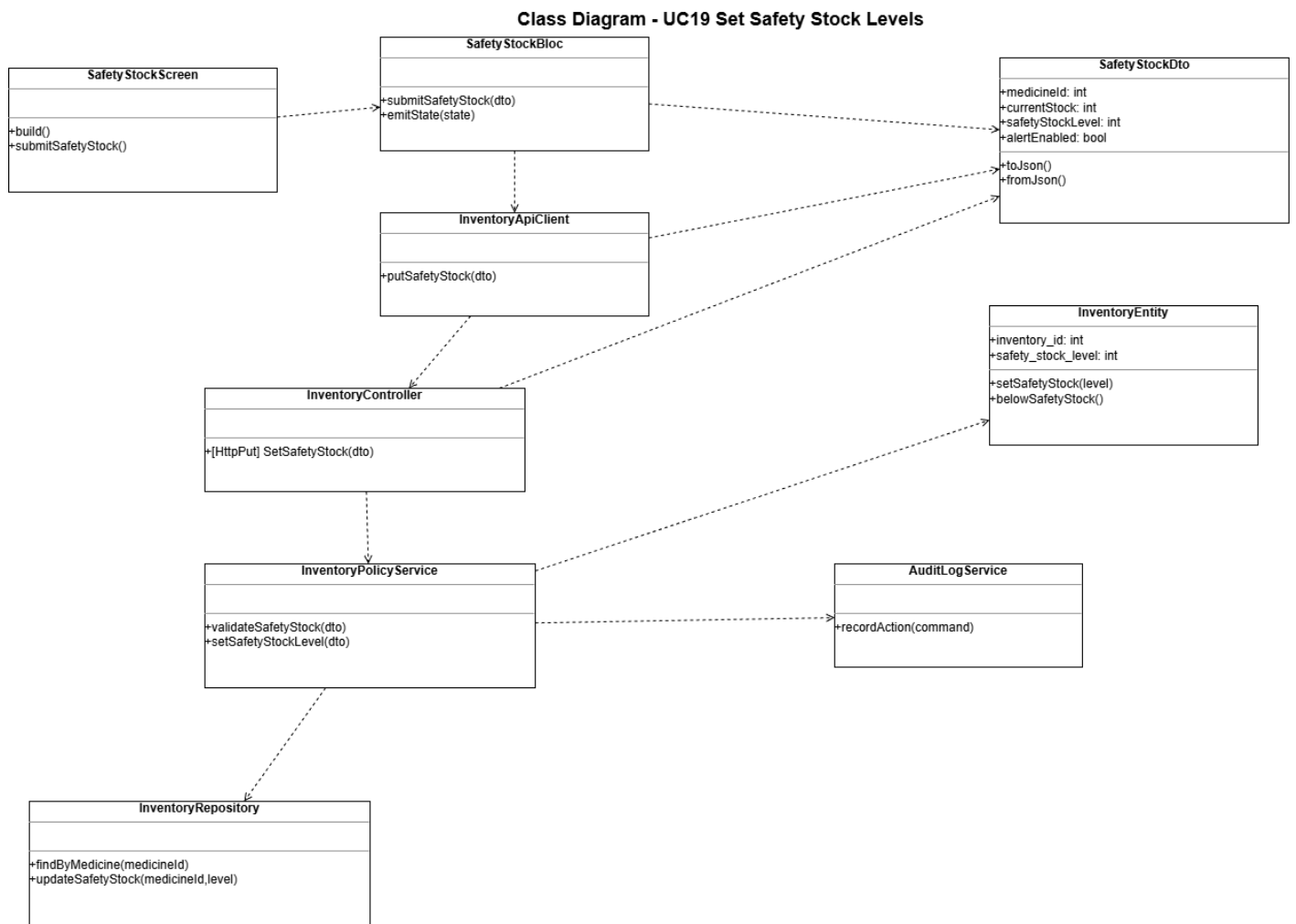
StocktakeService	<<control>>	Executes core variance checks between physical data vs system records, updates quantities, and manages adjustments.	comparePhysicalVsSystem(dto); saveStocktakeResult(dto); adjustInventory(dto)	StocktakeDto; InventoryEntity
InventoryRepository	<<interface>>	Manages query mechanisms to fetch system quantity snapshots and updates physical inventory stock balances.	getSystemQty(branch, batch); saveStocktakeResult(dto); adjustQuantity(batch, delta)	InventoryEntity
AuditLogService	<<control>>	Captures critical administrative stock verification and balancing movements inside the secure ledger.	recordAction(command)	StocktakeAuditCommand
InventoryEntity	<<entity>>	Tracks standard physical properties of individual storage slots, holding current actual quantities on hand.	increase(qty); decrease(qty)	inventory_id: int; quantity_on_hand: int

3.5.6.3 Perform Stocktake Sequence Diagram



3.5.7.UC19 Set Safety Stock Levels

3.5.7.1 Class Diagram

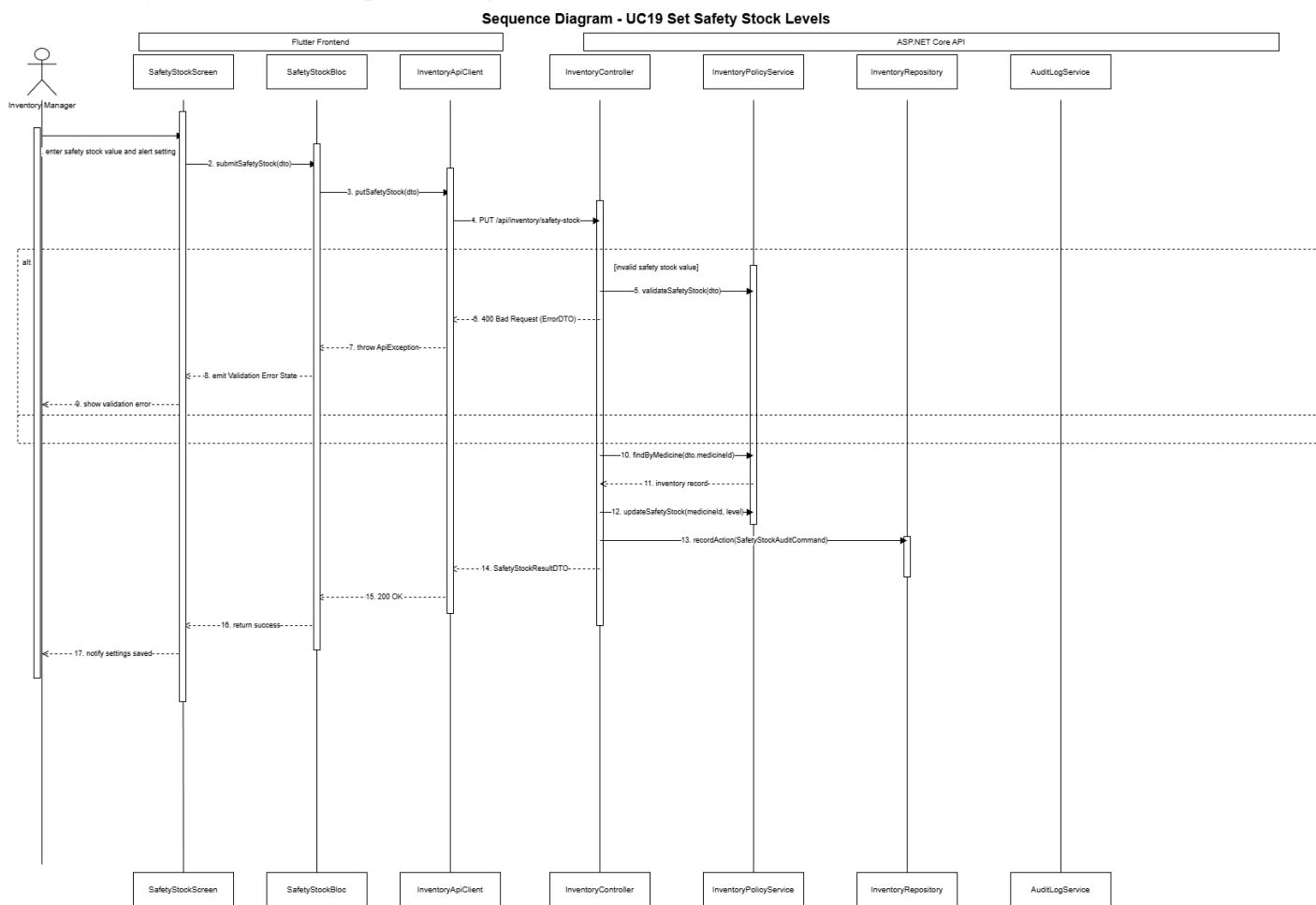


3.5.7.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
SafetyStockScreen	<<boundary>>	Provides interface for Inventory Manager to enter safety stock values and alert settings.	build(); submitSafetyStock()	N/A
SafetyStockBloc	<<control>>	Manages frontend state changes for setting safety stock levels and dispatches input validation.	submitSafetyStock(dto); emitState(state)	SafetyStockDto
InventoryApiClient	<<boundary>>	Dispatches HTTP PUT requests containing safety	putSafetyStock(dto)	SafetyStockDto

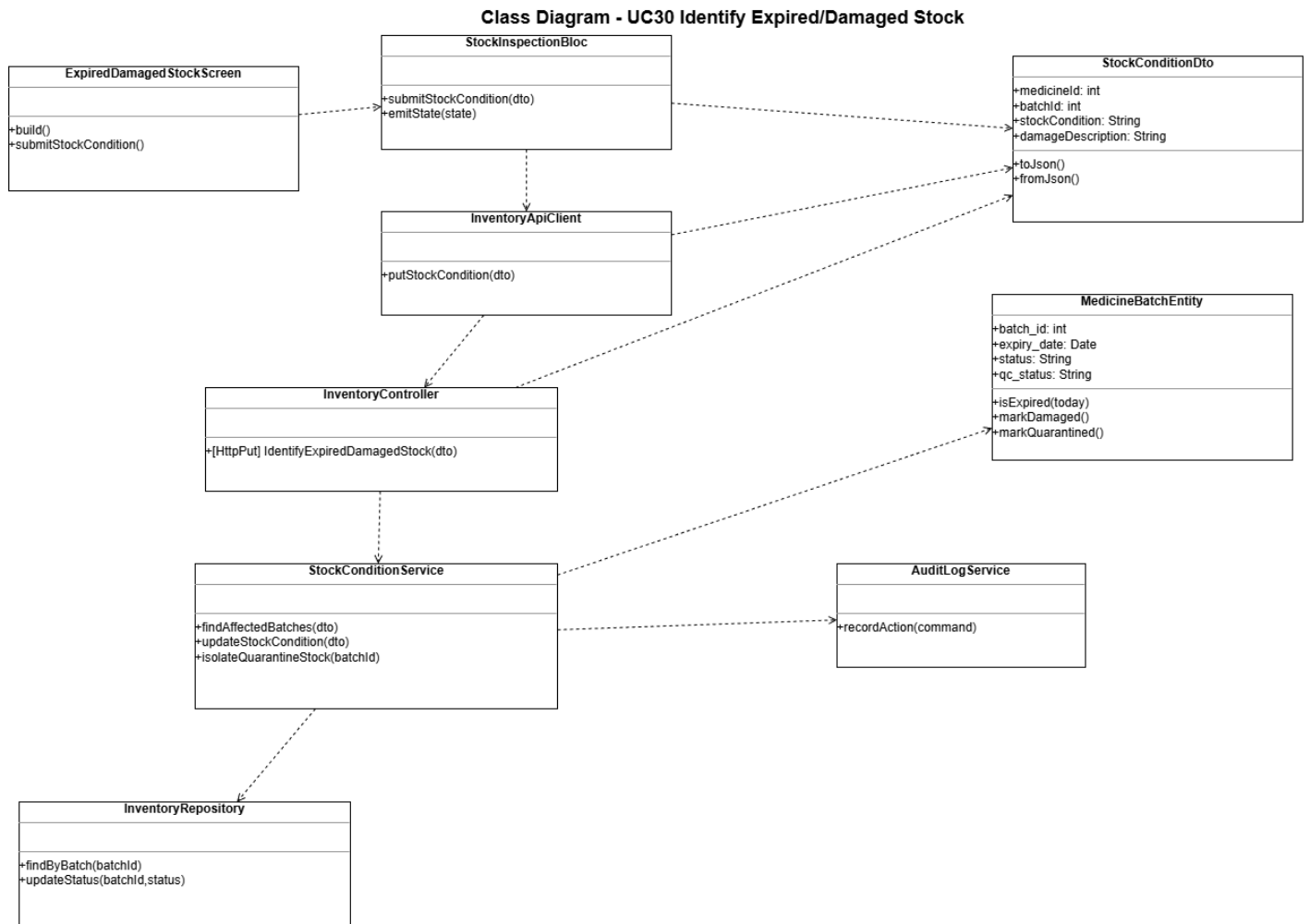
		stock configurations to the backend API.		
SafetyStockDto	<<DTO>>	Carries safety stock parameters and notification configurations between the frontend and backend.	toJson(); fromJson()	medicineId: int; currentStock: int; safetyStockLevel: int; alertEnabled: bool
InventoryController	<<control>>	Receives network API requests for stock thresholds and forwards the payload to the domain policy service.	[HttpPut] SetSafetyStock(dto)	SafetyStockDto
InventoryPolicyService	<<control>>	Evaluates safety stock business policies, runs logical checks, and coordinates persistent updates and auditing.	validateSafetyStock(dto); setSafetyStockLevel(dto)	SafetyStockDto; InventoryEntity
InventoryRepository	<<interface>>	Executes database queries and updates to read medicine records or persist adjusted safety stock limits.	findByMedicine(medicineId); updateSafetyStock(medicineId, level)	InventoryEntity
AuditLogService	<<control>>	Records formal administrative policy adjustments to the historical tracking database for audit compliance.	recordAction(command)	SafetyStockAuditCommand
InventoryEntity	<<entity>>	Represents the quantitative status, stock level thresholds, and alert states of a specific inventory item.	setSafetyStock(level); belowSafetyStock()	inventory_id: int; safety_stock_level: int

3.5.7.3 Set Safety Stock Levels Sequence Diagram



3.5.8. UC30 Identify Expired/Damaged Stock

3.5.8.1 Class Diagram

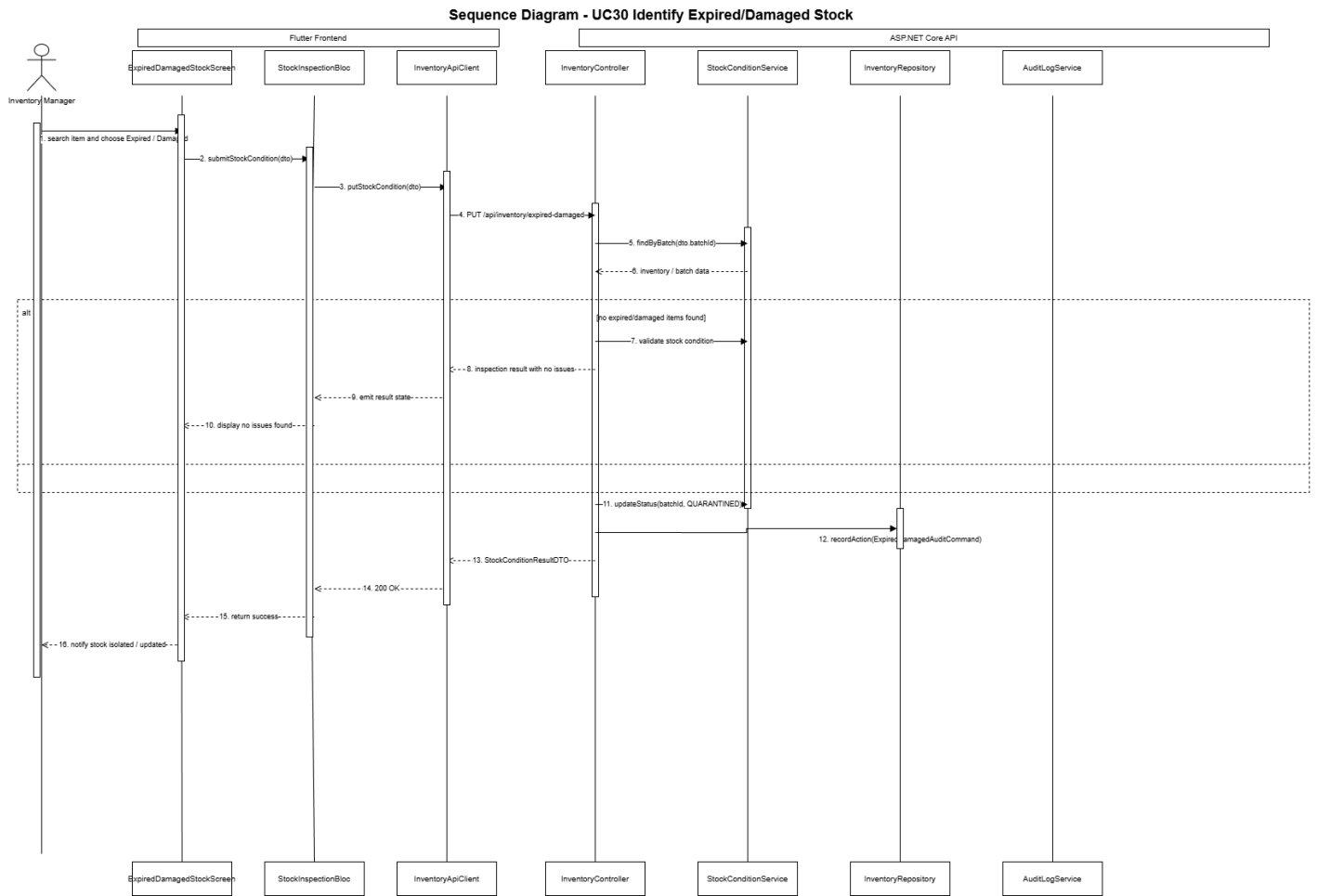


3.5.8.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
ExpiredDamagedStockScreen	<<boundary>>	Provides interface for Inventory Manager to search items and record expired or damaged stock conditions.	build(); submitStockCondition()	N/A
StockInspectionBloc	<<control>>	Manages frontend state for stock inspection workflows and coordinates submission actions.	submitStockCondition(dto); emitState(state)	StockConditionDto
InventoryApiClient	<<boundary>>	Dispatches HTTP PUT requests to update stock	putStockCondition(dto)	StockConditionDto

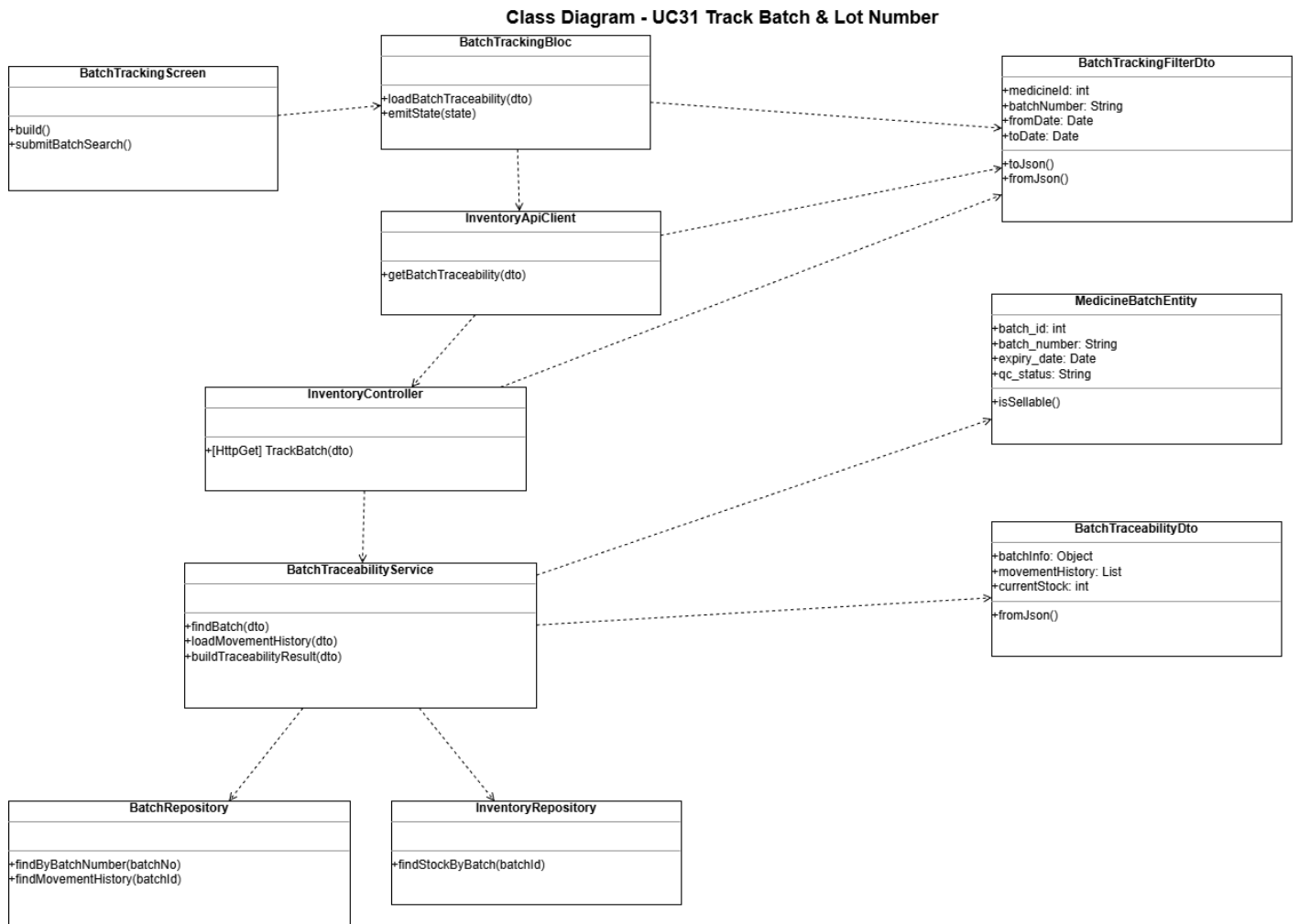
		condition data on the backend API.		
StockConditionDto	<<DTO>>	Encapsulates stock condition and damage details collected from the user interface.	toJson(); fromJson()	medicineId: int; batchId: int; stockCondition: String; damageDescription: String
InventoryController	<<control>>	Handles incoming backend API requests to flag expired or damaged stock and routes them to the appropriate service.	[HttpPut] IdentifyExpiredDamagedStock(dto)	StockConditionDto
StockConditionService	<<control>>	Processes inventory inspection rules, updates stock states, and triggers isolation logic for contaminated/expired lots.	findAffectedBatches(dto); updateStockCondition(dto); isolateQuarantineStock(batchId)	StockConditionDto; MedicineBatchEntity
InventoryRepository	<<interface>>	Manages physical database operations for querying and changing inventory batch statuses.	findByBatch(batchId); updateStatus(batchId, status)	MedicineBatchEntity
AuditLogService	<<control>>	Provides centralized logging mechanism to record critical stock adjustments and system actions for auditing.	recordAction(command)	ExpiredDamagedAuditCommand
MedicineBatchEntity	<<entity>>	Encapsulates core state and business validations for an individual inventory medicine batch.	isExpired(today); markDamaged(); markQuarantined()	batch_id: int; expiry_date: Date; status: String; qc_status: String

3.5.8.3 Identify Expired/Damaged Stock Sequence Diagram



3.5.9. UC31 Track Batch & Lot Number

3.5.9.1 Class Diagram

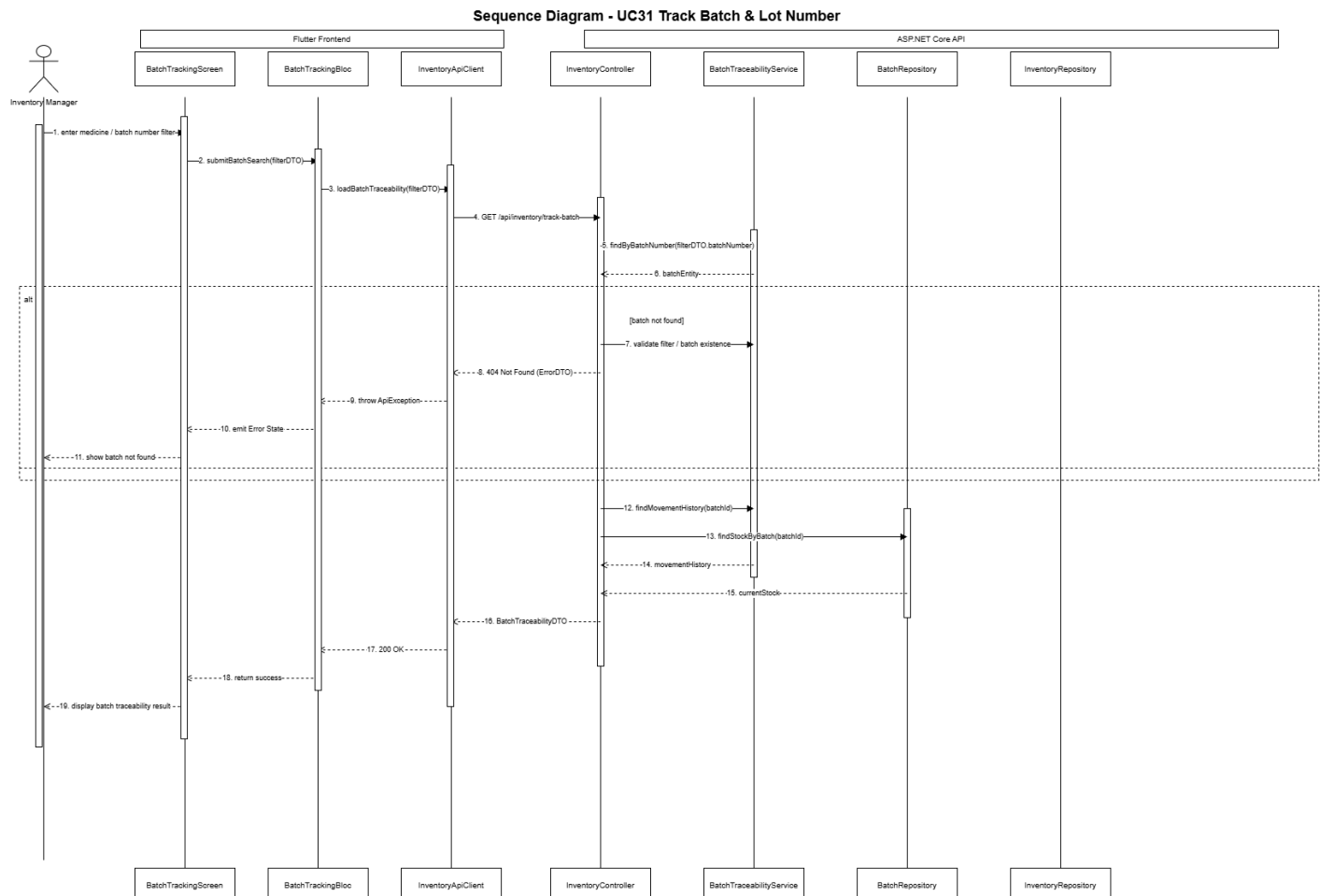


3.5.9.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
BatchTrackingScreen	<<boundary>>	Provides interface for Inventory Manager to enter medicine/batch filters and search.	build(); submitBatchSearch()	N/A
BatchTrackingBloc	<<control>>	Coordinates batch tracking workflow on the frontend, manages search states, and emits results or errors.	loadBatchTraceability(dto); emitState(state)	BatchTrackingFilterDto
InventoryApiClient	<<boundary>>	Sends HTTP GET requests to the backend API endpoint for batch tracking.	getBatchTraceability(dto)	BatchTrackingFilterDto
BatchTrackingFilterDto	<<DTO>>	Transfers search filter criteria from frontend to backend.	toJson(); fromJson()	medicineId: int; batchNumber: String; fromDate: Date; toDate: Date
InventoryController	<<control>>	Exposes backend API endpoint to receive batch tracking	[HttpGet] TrackBatch(dto)	BatchTrackingFilterDto

		requests and routes them to the application service.		
BatchTraceabilityService	<<control>>	Executes business logic for tracking: validates batch presence, coordinates repository calls to fetch movement history and stock.	findBatch(dto); loadMovementHistory(dto); buildTraceabilityResult(dto)	BatchTrackingFilterDto; MedicineBatchEntity; BatchTraceabilityDto
BatchRepository	<<interface>>	Provides methods to persist and query batch and movement history details from the database.	findByBatchNumber(batchNo); findMovementHistory(batchId)	MedicineBatchEntity
InventoryRepository	<<interface>>	Provides methods to query physical stock levels associated with a specific batch.	findStockByBatch(batchId)	N/A
MedicineBatchEntity	<<entity>>	Represents core medicine batch information stored in the system.	isSellable()	batch_id: int; batch_number: String; expiry_date: Date; qc_status: String
BatchTraceabilityDto	<<DTO>>	Encapsulates comprehensive batch tracking results to return to the frontend.	fromJson()	batchInfo: Object; movementHistory: List; currentStock: int

3.5.9.3 Identify Expired/Damaged Stock Sequence Diagram



3.4.1 Class Diagram

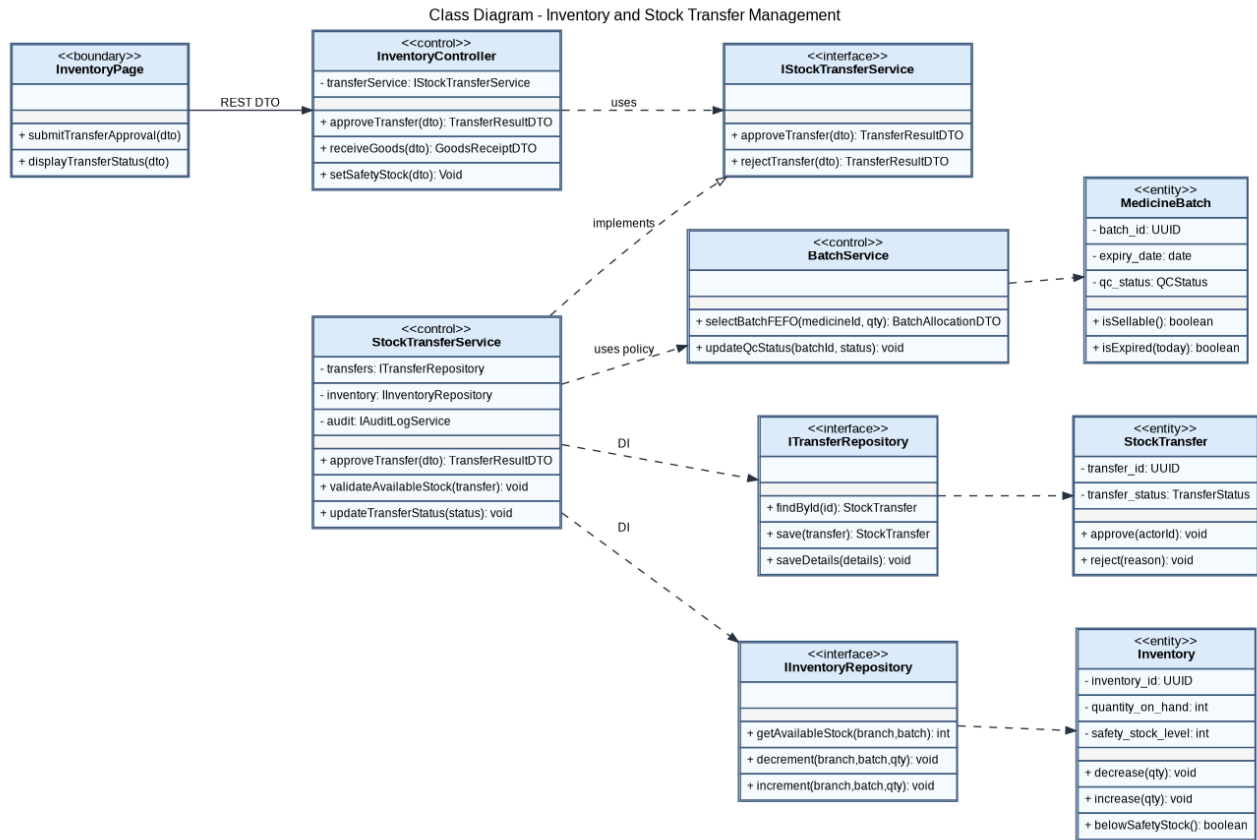


Figure 3-5: Class Diagram - Inventory Management

3.4.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
InventoryPage	<<boundary>>	Displays inventory operations and transfer approval results.	submitTransferApproval(dto), displayTransferStatus(dto)	TransferResultDTO
InventoryController	<<control>>	Receives inventory requests and delegates to inventory/transfer services.	approveTransfer(dto), receiveGoods(dto), setSafetyStock(dto)	InventoryDTO
IStockTransferService	<<interface>>	Defines stock transfer approval/rejection operations.	approveTransfer(dto), rejectTransfer(dto)	TransferResultDTO
StockTransferService	<<control>>	Validates transfers, checks stock availability, updates quantities transactionally, and records audit events.	approveTransfer(dto), validateAvailableStock(transfer), updateTransferStatus(status)	StockTransfer, Inventory
BatchService	<<control>>	Tracks batch number, expiry date, supplier source, FEFO selection, and QC status.	selectBatchFEFO(medicine_id, qty), updateQcStatus(batch_id,status), checkExpiry()	MedicineBatch
ITransferRepository	<<interface>>	Defines transfer persistence operations.	findById(id), save(transfer), saveDetails(details)	STOCK_TRANSFER
IInventoryRepository	<<interface>>	Defines inventory quantity operations.	getAvailableStock(branch,batch), decrement(branch,batch,qty), increment(branch,batch,qty)	INVENTORY
StockTransfer	<<entity>>	Encapsulates transfer header status and approval behavior.	approve(actor_id), reject(reason)	STOCK_TRANSFER
Inventory	<<entity>>	Encapsulates branch-level medicine quantity and safety stock rules.	decrease(qty), increase(qty), belowSafetyStock()	INVENTORY
MedicineBatch	<<entity>>	Encapsulates lot number, expiry, and QC state.	isSellable(), isExpired(today)	MEDICINE_BATCH

3.4.3 Approve Internal Transfer Sequence Diagram

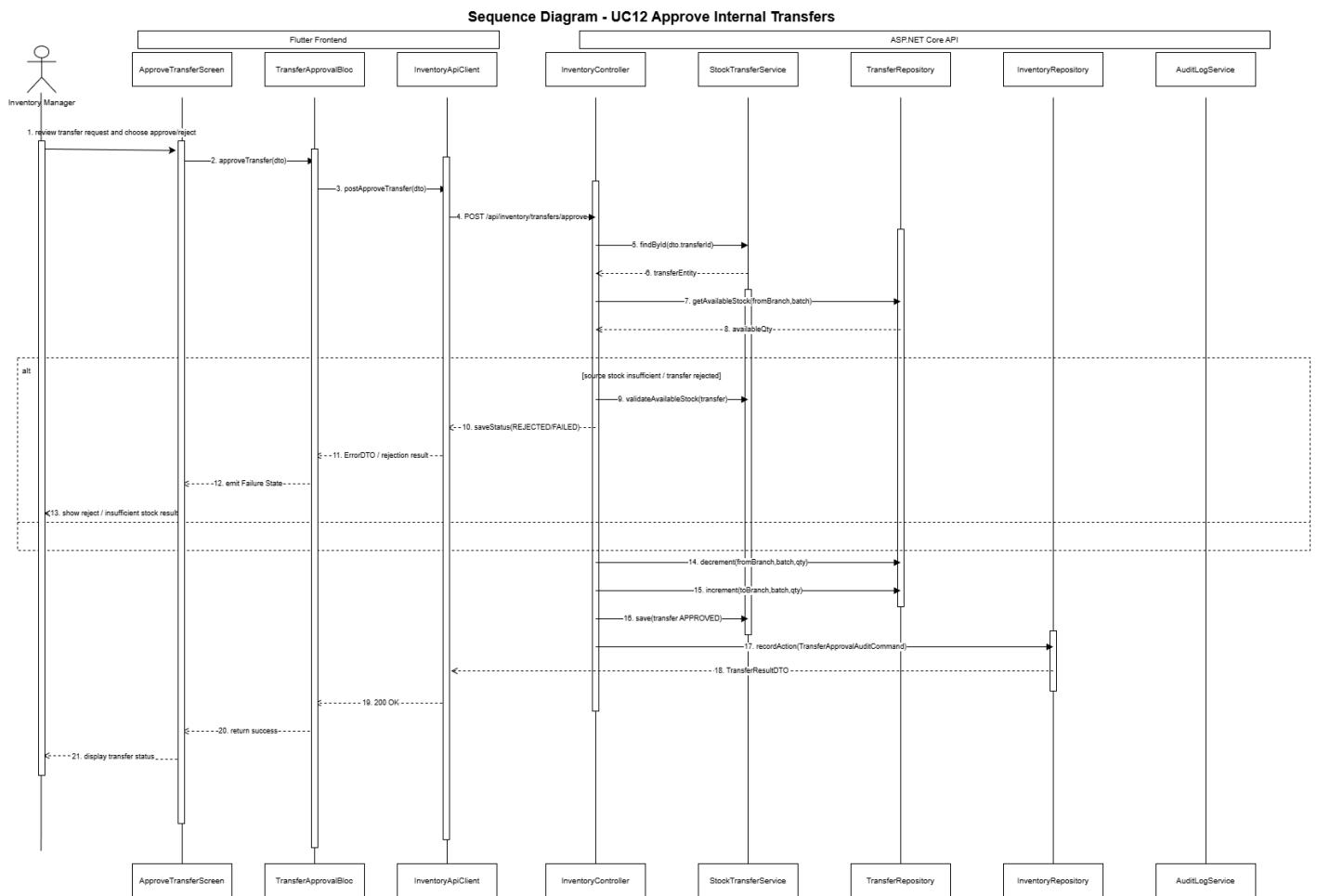


Figure 3-6: Sequence Diagram - Approve Internal Transfer

3.4.4 Design Notes

- Inventory quantities are adjusted only through InventoryService or StockTransferService methods; controllers cannot update quantities directly.
- Batch and expiry data support FEFO selection and medicine traceability.
- Safety stock thresholds are stored per branch/medicine/batch inventory record to support branch-specific restocking alerts.
- Internal transfers use header-detail tables so one request can contain multiple medicine batches.
- Rejected goods and expired/damaged stock are isolated by qc_status/status rather than removed from the database.

3.6 Sales and Payment Management

Sales and Payment Management covers medicine lookup, prescription-linked invoice generation, generated invoice viewing, payment transaction creation, payment verification, and transaction history. Staff users perform these functions during customer transactions.

3.6.1. Generate Invoice

3.6.1.1 Class Diagram

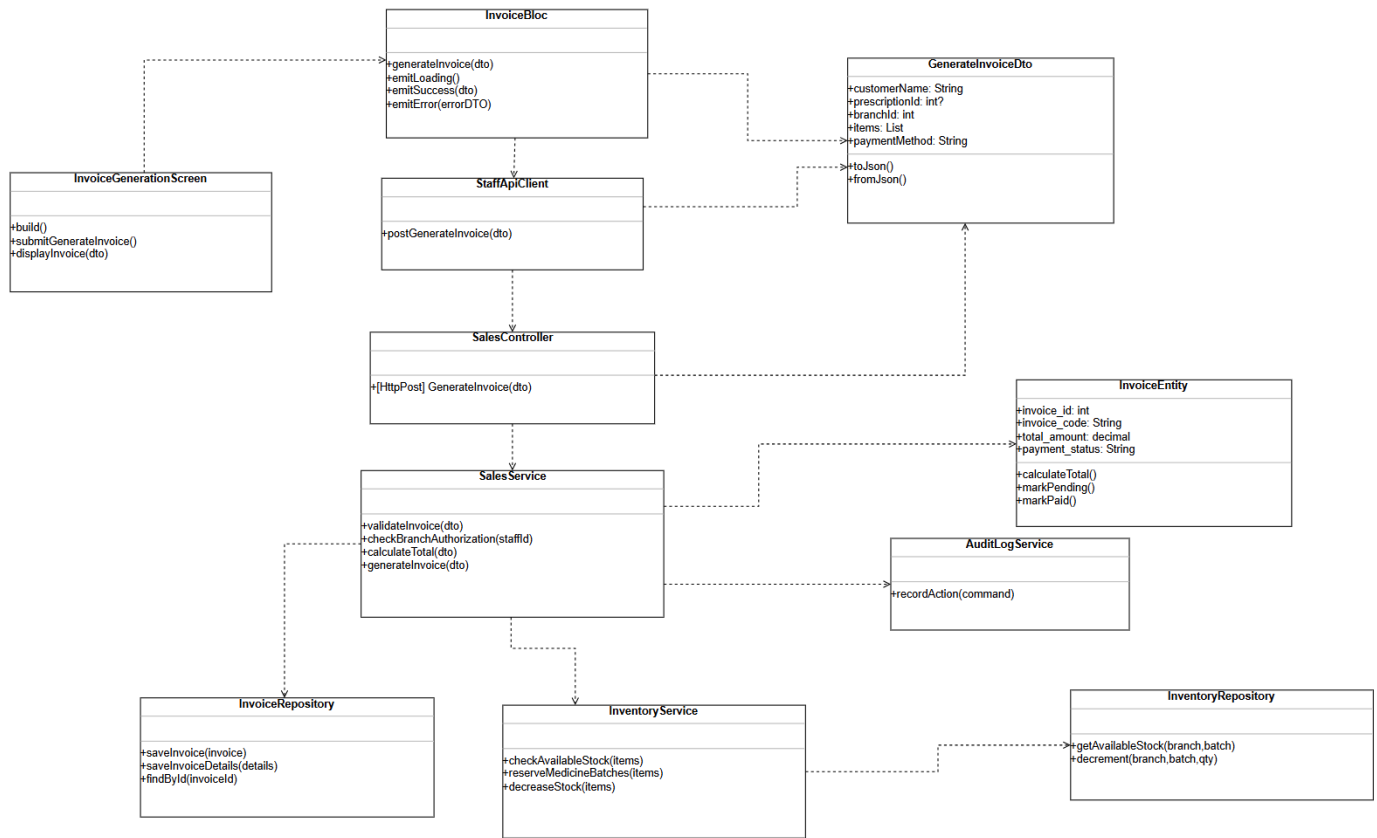


Figure 3-6-1-1: Class Diagram - Generate Invoice

3.6.1.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
InvoiceGenerationScreen	<<boundary>>	Provides the UI for staff to create and submit a new sales invoice.	build(); submitGenerateInvoice(); displayInvoice(dto)	GenerateInvoiceDto
InvoiceBloc	<<control>>	Manages invoice generation workflow and UI state transitions.	generateInvoice(dto); emitLoading(); emitSuccess(dto); emitError(errorDTO)	GenerateInvoiceDto; InvoiceEntity
StaffApiClient	<<boundary>>	Sends invoice generation requests from frontend to backend API.	postGenerateInvoice(dto)	GenerateInvoiceDto
GenerateInvoiceDto	<<DTO>>	Transfers invoice generation data between layers.	toJson(); fromJson()	customerName; prescriptionId; branchId; items; paymentMethod
SalesController	<<control>>	Receives invoice generation requests and delegates processing to SalesService.	GenerateInvoice(dto)	GenerateInvoiceDto
SalesService	<<control>>	Validates invoice information, calculates totals, and creates invoices.	validateInvoice(dto); checkBranchAuthorization(staffId); calculateTotal(dto); generateInvoice(dto)	InvoiceEntity; InvoiceDetail
InventoryService	<<control>>	Verifies medicine availability and updates inventory during invoice creation.	checkAvailableStock(items); reserveMedicineBatches(items); decreaseStock(items)	Inventory; MedicineBatch

InvoiceRepository	<<interface>>	Persists and retrieves invoice data.	saveInvoice(invoice); saveInvoiceDetails(details); findById(invoiceId)	InvoiceEntity; InvoiceDetail
InventoryRepository	<<interface>>	Provides inventory data access and stock updates.	getAvailableStock(branch,batch); decrement(branch,batch,qty)	Inventory; MedicineBatch
InvoiceEntity	<<entity>>	Represents a sales invoice and its business behavior.	calculateTotal(); markPending(); markPaid()	invoice_id; invoice_code; total_amount; payment_status
AuditLogService	<<control>>	Records invoice generation activities for auditing and compliance.	recordAction(command)	AuditLog

3.6.1.3 Generate Invoice Sequence Diagram

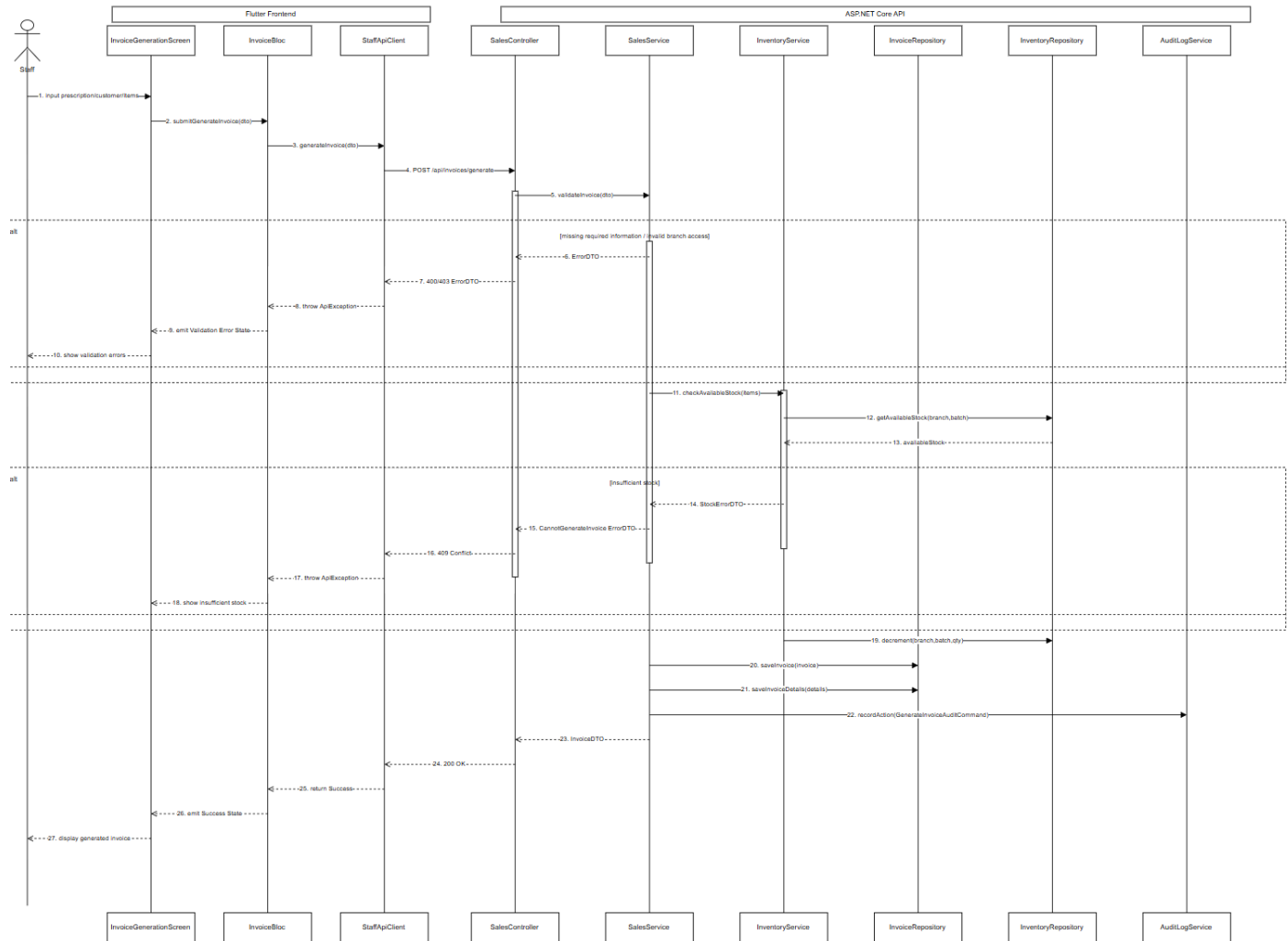


Figure 3-6-1-3: Sequence Diagram - Generate Invoice

3.6.2. Create Payment Transaction

3.6.2.1 Class Diagram

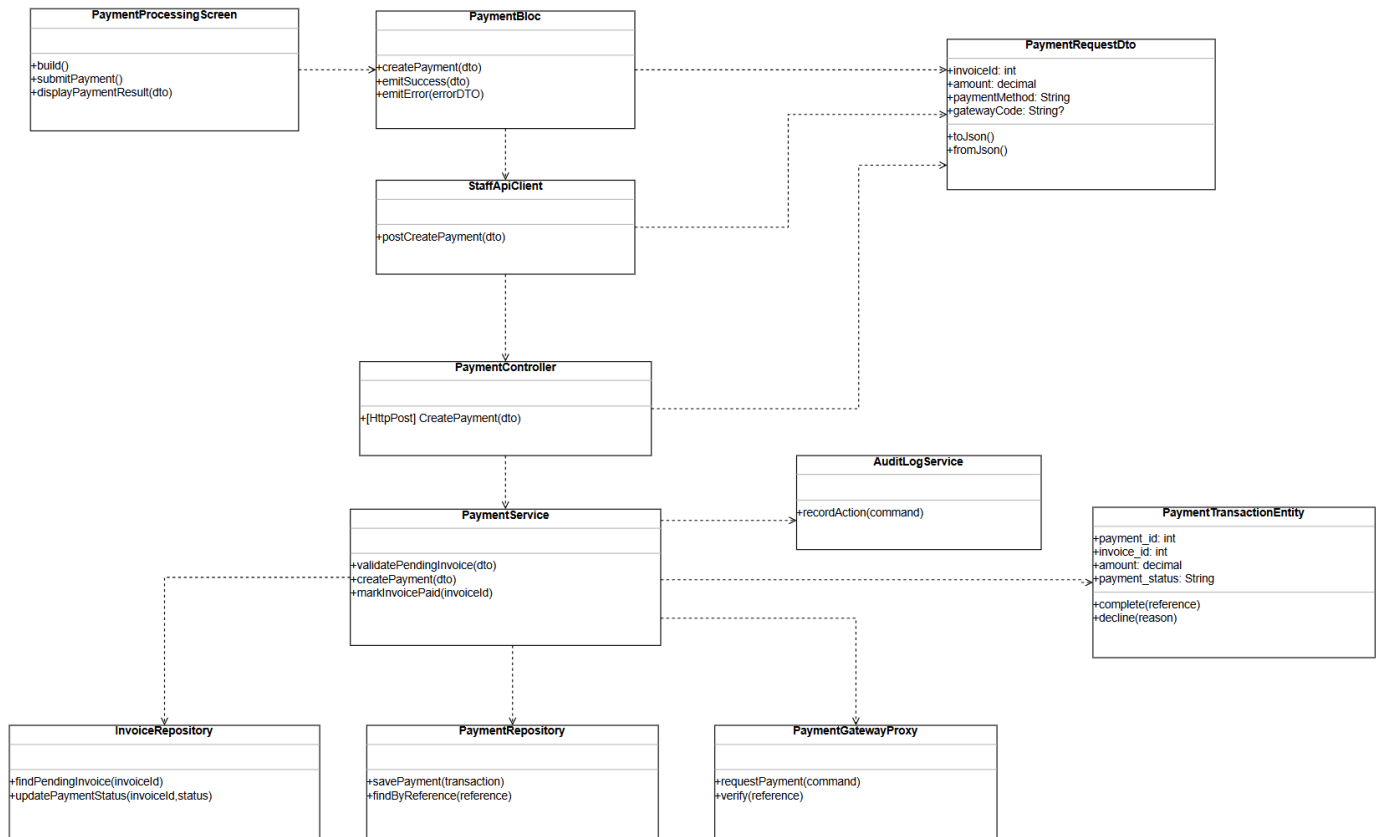


Figure 3-6-2-1: Class Diagram - Create Payment Transaction

3.6.2.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
PaymentProcessingScreen	<<boundary>>	Provides payment processing interface for staff.	build(); submitPayment(); displayPaymentResult(dto)	PaymentRequestDto
PaymentBloc	<<control>>	Coordinates payment transaction workflow and UI states.	createPayment(dto); emitSuccess(dto); emitError(errorDTO)	PaymentRequestDto; PaymentTransactionEntity
StaffApiClient	<<boundary>>	Sends payment requests to backend services.	postCreatePayment(dto)	PaymentRequestDto
PaymentRequestDto	<<DTO>>	Transfers payment request information.	toJson(); fromJson()	invoiceId; amount; paymentMethod; gatewayCode
PaymentController	<<control>>	Receives payment transaction requests and delegates business logic.	CreatePayment(dto)	PaymentRequestDto
PaymentService	<<control>>	Validates invoices, processes payments, and updates invoice status.	validatePendingInvoice(dto); createPayment(dto); markInvoicePaid(invoiceId)	PaymentTransactionEntity; InvoiceEntity
InvoiceRepository	<<interface>>	Provides invoice validation and payment status updates.	findPendingInvoice(invoiceId); updatePaymentStatus(invoiceId, status)	InvoiceEntity
PaymentRepository	<<interface>>	Persists and retrieves payment transaction records.	savePayment(transaction); findByReference(reference)	PaymentTransactionEntity
PaymentGatewayProxy	<<boundary>>	Integrates with external payment gateway services.	requestPayment(command); verify(reference)	Payment Gateway Response

PaymentTransactionEntity	<<entity>>	Represents a customer payment transaction.	complete(reference); decline(reason)	payment_id; invoice_id; amount; payment_status
AuditLogService	<<control>>	Records payment activities and gateway interactions.	recordAction(command)	AuditLog

3.6.2.3 Create Payment Transaction Sequence Diagram

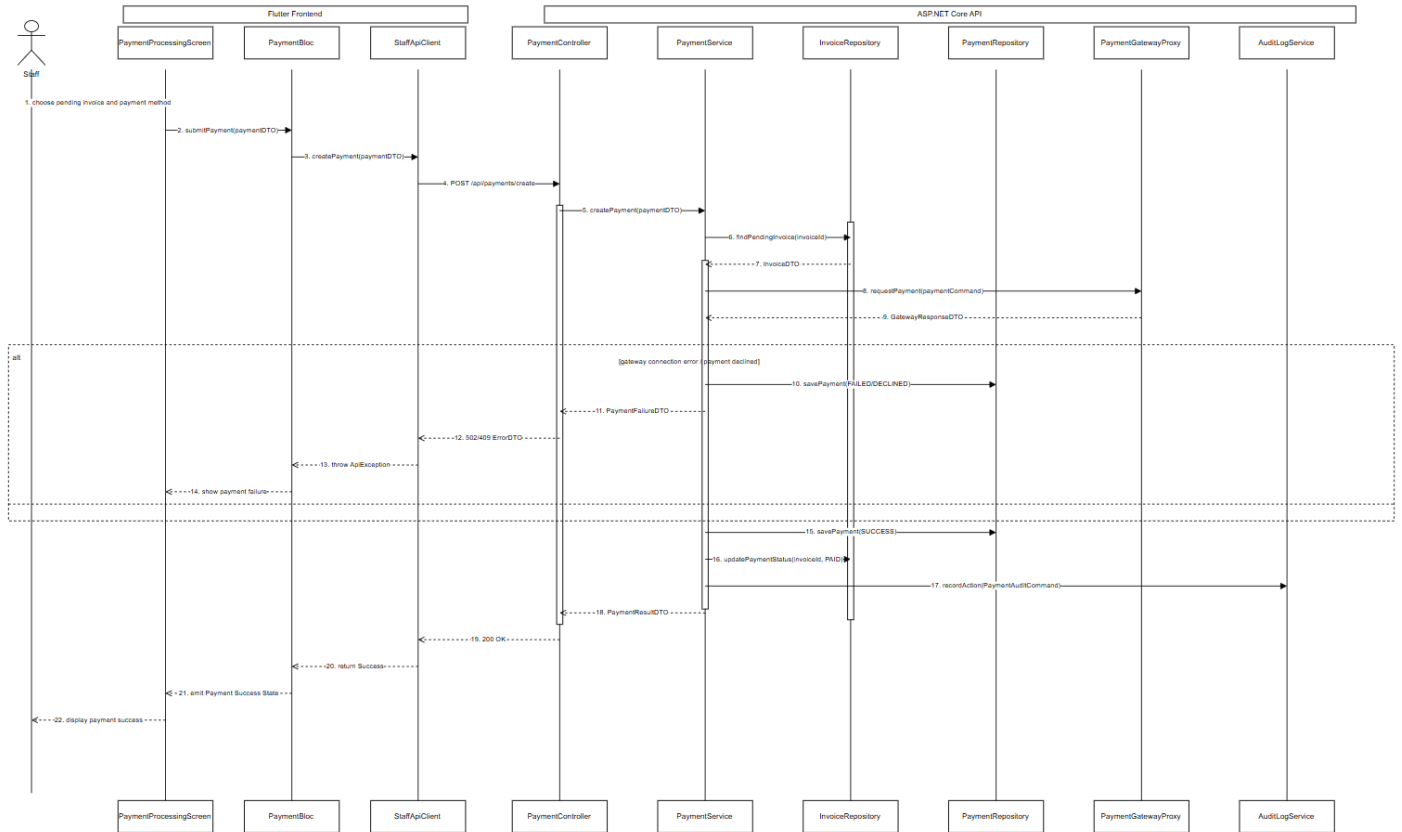


Figure 3-6-2-3: Sequence Diagram - Create Payment Transaction

3.6.3. View Generated Invoices

3.6.3.1 Class Diagram

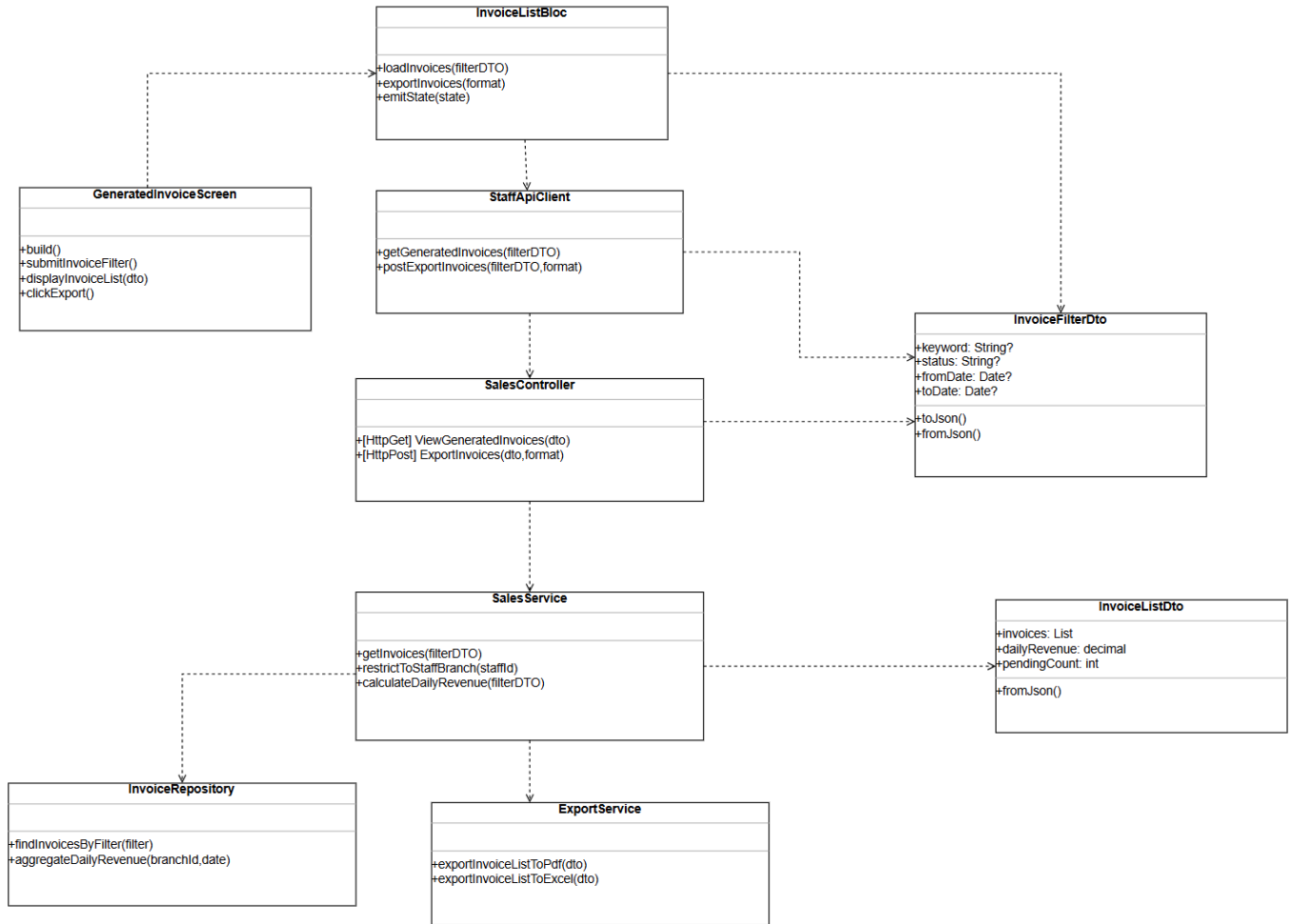


Figure 3-6-3-1: Class Diagram - View Generated Invoices

3.6.2.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
GeneratedInvoiceScreen	<<boundary>>	Displays invoice history and filtering options.	build(); submitInvoiceFilter(); displayInvoiceList(dto); clickExport()	InvoiceFilterDto; InvoiceListDto
InvoiceListBloc	<<control>>	Manages invoice list retrieval, filtering, and export actions.	loadInvoices(filterDTO); exportInvoices(format); emitState(state)	InvoiceFilterDto; InvoiceListDto
StaffApiClient	<<boundary>>	Retrieves invoice data and export files from backend services.	getGeneratedInvoices(filterDTO); postExportInvoices(filterDTO, format)	InvoiceFilterDto
InvoiceFilterDto	<<DTO>>	Encapsulates invoice search and filtering parameters.	toJson(); fromJson()	keyword; status; fromDate; toDate
SalesController	<<control>>	Handles invoice listing and export requests.	ViewGeneratedInvoices(dto); ExportInvoices(dto, format)	InvoiceFilterDto
SalesService	<<control>>	Retrieves invoice history and revenue summaries for staff branches.	getInvoices(filterDTO); restrictToStaffBranch(staffId); calculateDailyRevenue(filterDTO)	InvoiceEntity

InvoiceRepository	<<interface>>	Provides invoice filtering and revenue aggregation operations.	findInvoicesByFilter(filter); aggregateDailyRevenue(branchId ,date)	InvoiceEntity
ExportService	<<control>>	Generates export files for invoice reports.	exportInvoiceListToPdf(dto); exportInvoiceListToExcel(dto)	InvoiceListDto
InvoiceListDto	<<DTO>>	Carries invoice list and summary statistics to the UI.	fromJson()	invoices; dailyRevenue; pendingCount

3.6.2.3 View Generated Invoices Sequence Diagram

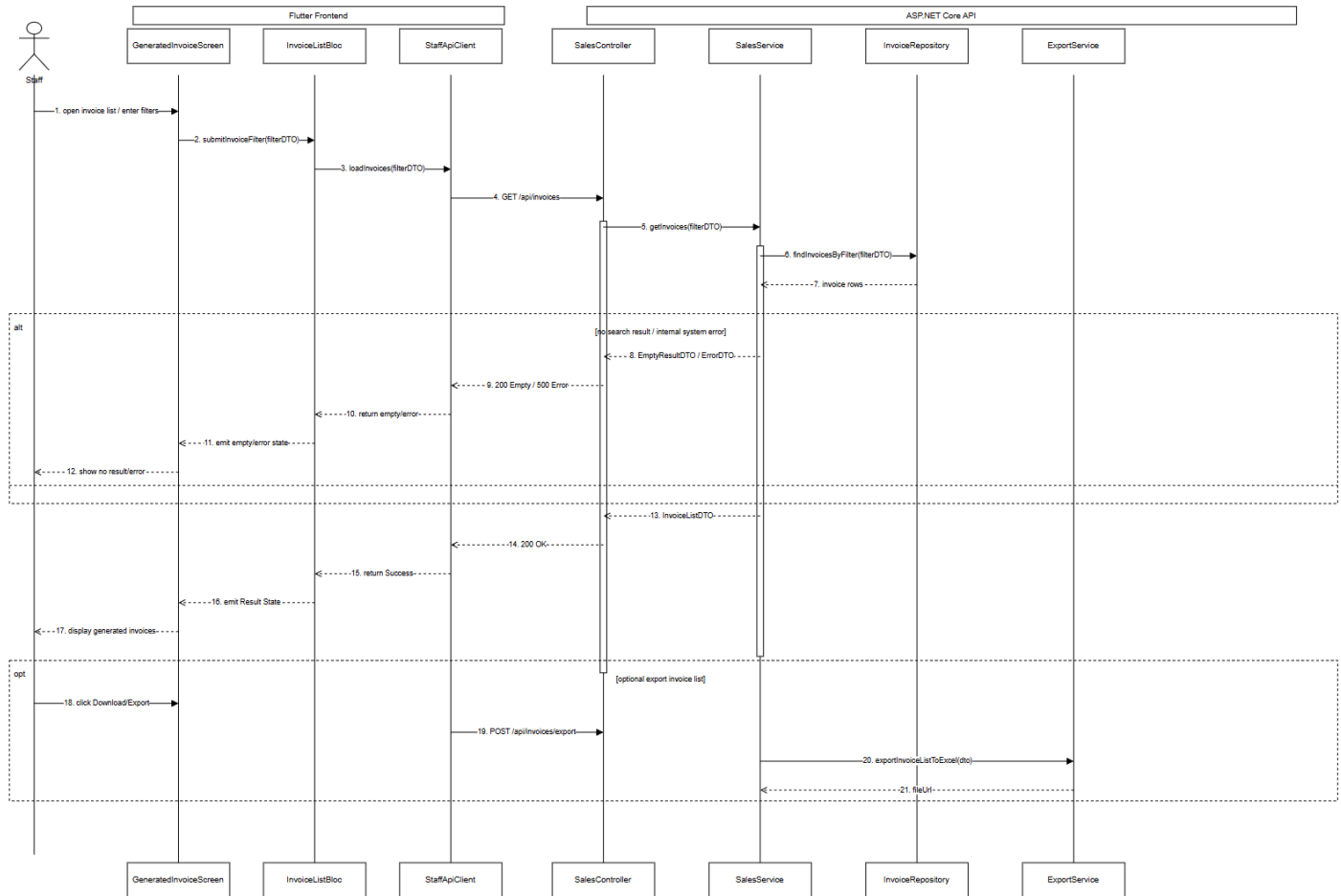


Figure 3-6-3-3: Sequence Diagram - View Generated Invoices

3.6.4. View Payment Transaction

3.6.4.1 Class Diagram

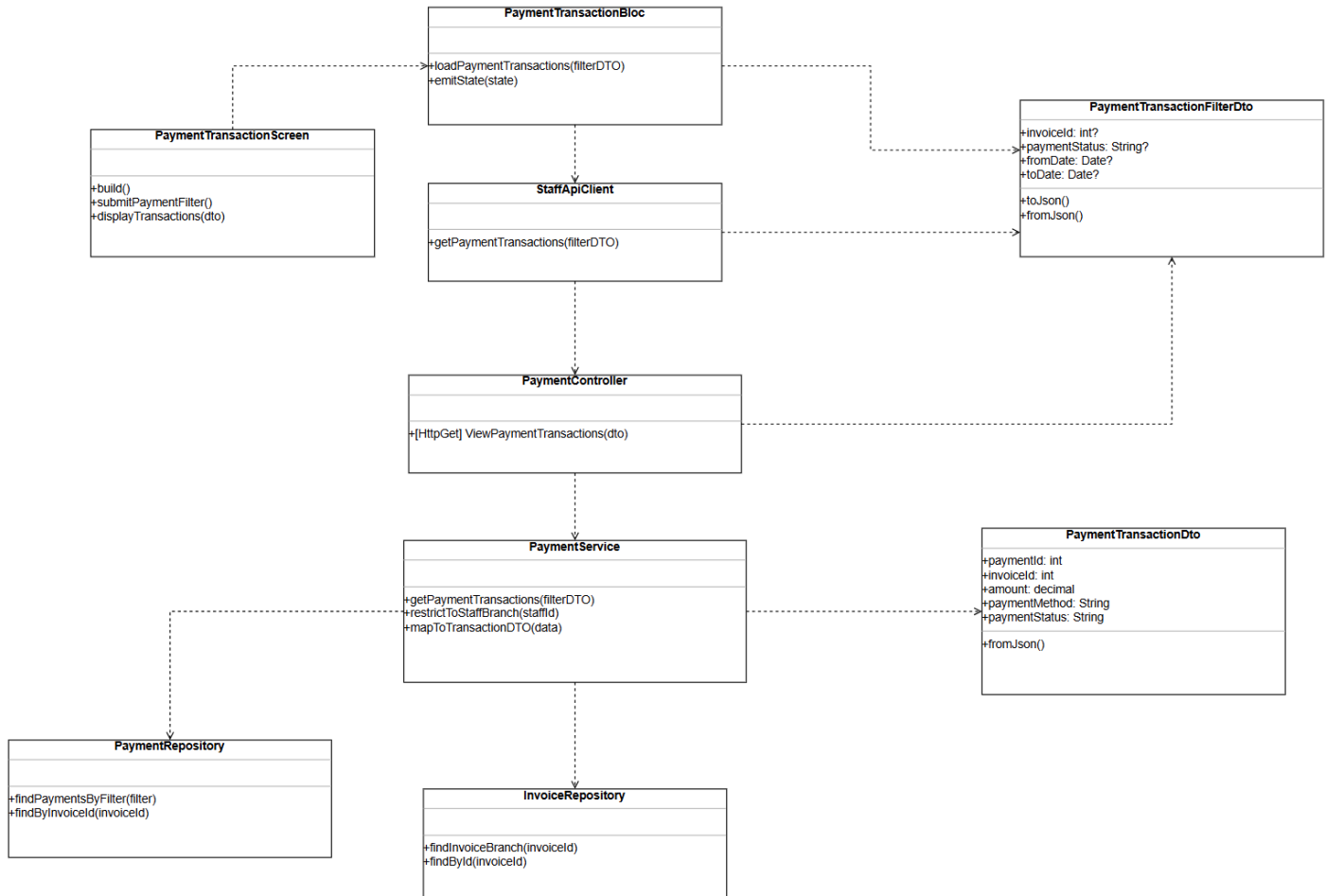


Figure 3-6-4-1: Class Diagram - View Payment Transaction

3.6.4.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
PaymentTransactionScreen	<<boundary>>	Provides payment transaction search and viewing interface.	build(); submitPaymentFilter(); displayTransactions(dto)	PaymentTransactionFilterDto; PaymentTransactionDto
PaymentTransactionBloc	<<control>>	Handles transaction retrieval and state management.	loadPaymentTransactions(filterDTO); emitState(state)	PaymentTransactionFilterDto; PaymentTransactionDto
StaffApiClient	<<boundary>>	Requests payment transaction information from backend services.	getPaymentTransactions(filterDTO)	PaymentTransactionFilterDto
PaymentTransactionFilterDto	<<DTO>>	Encapsulates transaction search criteria.	toJson(); fromJson()	invoiceId; paymentStatus; fromDate; toDate
PaymentController	<<control>>	Handles payment transaction query requests.	ViewPaymentTransactions(dto)	PaymentTransactionFilterDto
PaymentService	<<control>>	Retrieves and transforms payment transaction information for presentation.	getPaymentTransactions(filterDTO); restrictToStaffBranch(staffId); mapToTransactionDTO(data)	PaymentTransactionEntity
PaymentRepository	<<interface>>	Provides payment transaction query operations.	findPaymentsByFilter(filter); findByInvoiceId(invoiceId)	PaymentTransactionEntity

InvoiceRepository	<<interface>>	Supplies invoice information associated with payment records.	findInvoiceBranch(invoiceId); findById(invoiceId)	InvoiceEntity
PaymentTransactionDto	<<DTO>>	Transfers payment transaction details to the UI layer.	fromJson()	paymentId; invoiceId; amount; paymentMethod; paymentStatus

3.6.4.3 View Payment Transaction Sequence Diagram

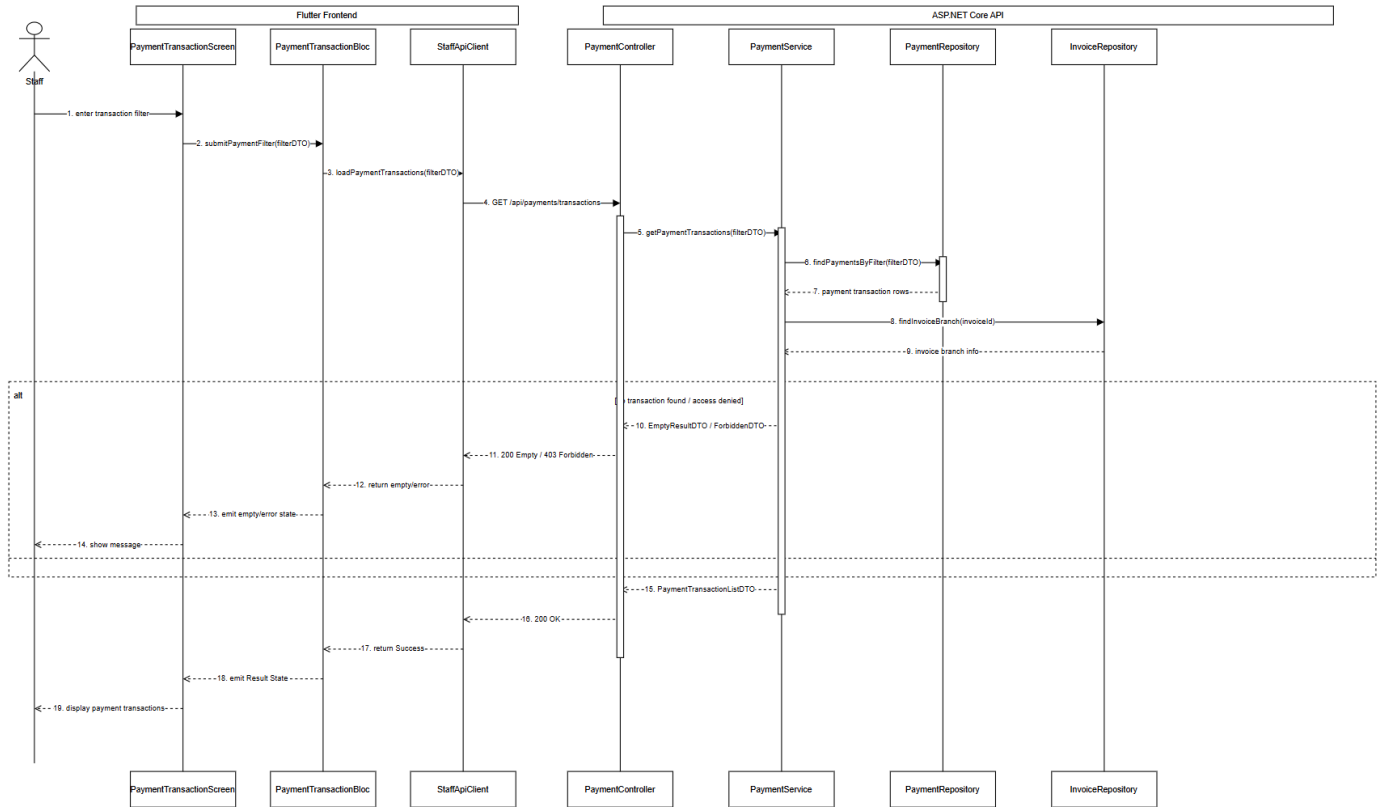


Figure 3-6-4-3: Sequence Diagram - View Payment Transaction

3.6.5. View Medicine Information

3.6.5.1 Class Diagram

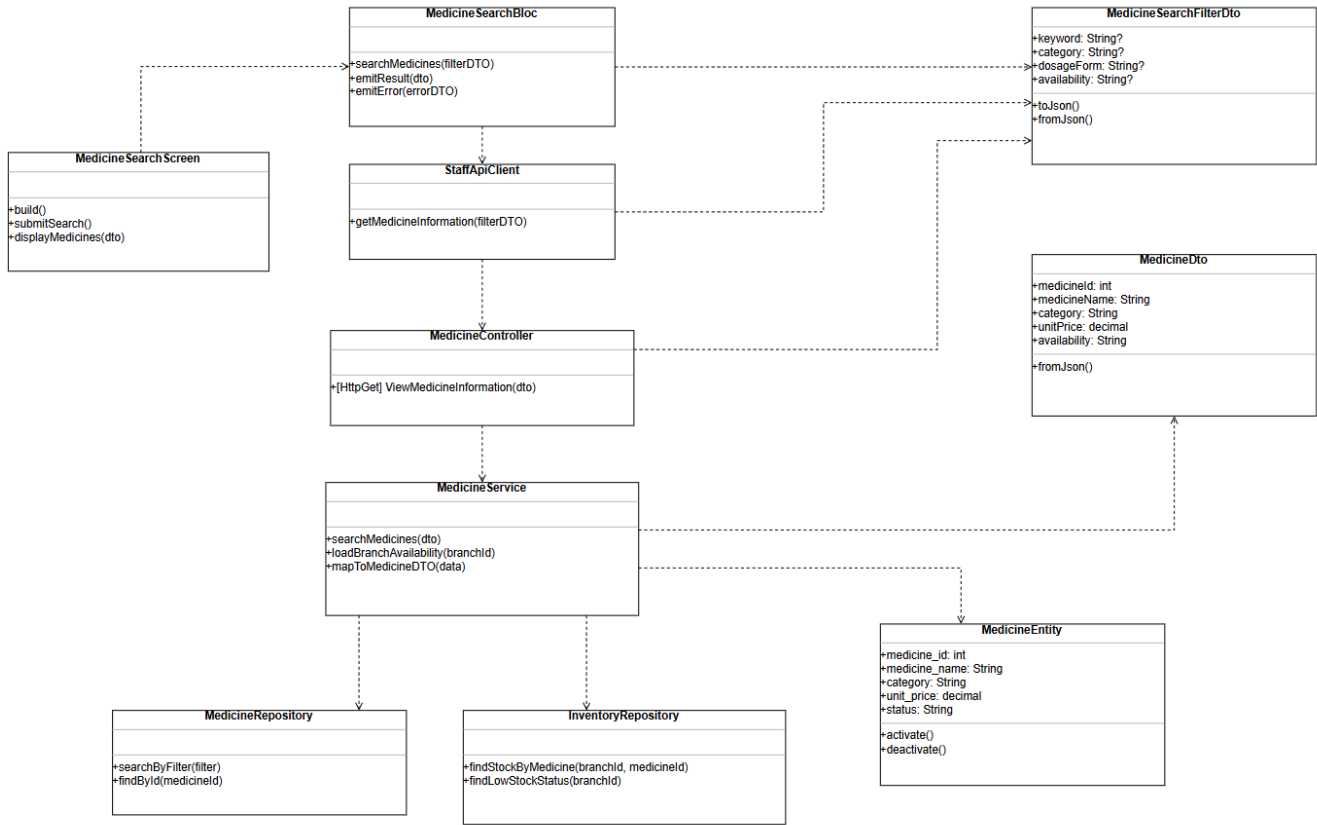


Figure 3-6-5-1: Class Diagram - View Medicine Information

3.6.5.2 Class Specifications

Class	Type	Main Responsibility	Main Operations	Related Data
MedicineSearchScreen	<<boundary>>	Provides medicine search and viewing interface for staff.	build(); submitSearch(); displayMedicines(dto)	MedicineSearchFilterDto; MedicineDto
MedicineSearchBloc	<<control>>	Handles medicine search logic and screen state management.	searchMedicines(filterDTO); emitResult(dto); emitError(errorDTO)	MedicineSearchFilterDto; MedicineDto
StaffApiClient	<<boundary>>	Sends medicine search requests to backend services.	getMedicineInformation(filterDTO)	MedicineSearchFilterDto
MedicineSearchFilterDto	<<DTO>>	Encapsulates medicine search criteria.	toJson(); fromJson()	keyword; category; dosageForm; availability
MedicineController	<<control>>	Handles medicine information requests from the API layer.	ViewMedicineInformation(dto)	MedicineSearchFilterDto
MedicineService	<<control>>	Searches medicines and enriches data with branch inventory information.	searchMedicines(dto); loadBranchAvailability(branchId); mapToMedicineDTO(data)	MedicineEntity; Inventory
MedicineRepository	<<interface>>	Provides medicine lookup and filtering operations.	searchByFilter(filter); findById(medicineId)	MedicineEntity
InventoryRepository	<<interface>>	Supplies stock information for medicines.	findStockByMedicine(branchId, medicineId); findLowStockStatus(branchId)	Inventory
MedicineDto	<<DTO>>	Transfers medicine information to presentation layer.	fromJson()	medicineId; medicineName; category; unitPrice; availability

MedicineEntity	<<entity>>	Represents medicine master data.	activate(); deactivate()	medicine_id; medicine_name; category; unit_price; status
----------------	------------	----------------------------------	--------------------------	---

3.6.5.3 View Medicine Information Sequence Diagram

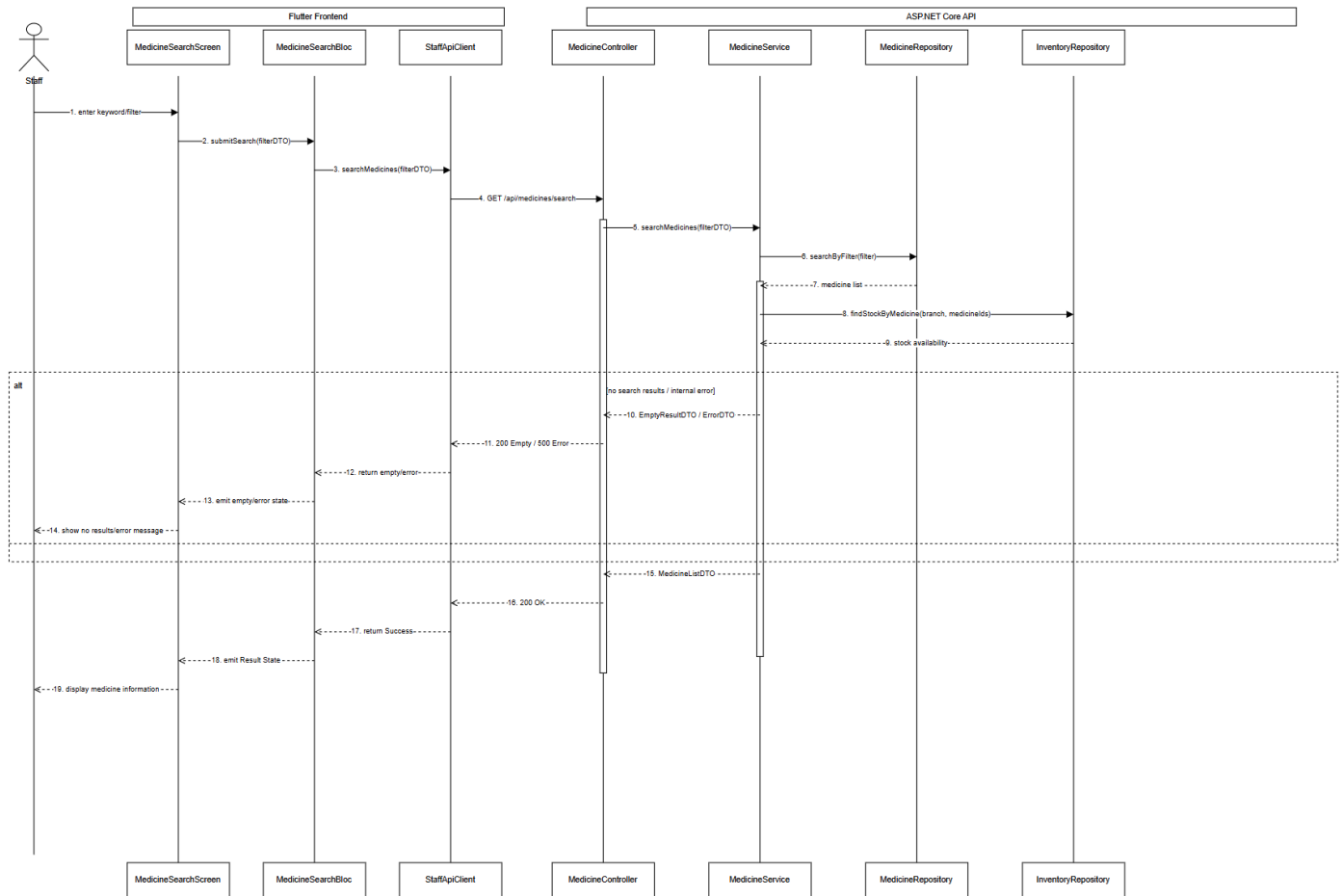


Figure 3-6-5-3: Sequence Diagram - View Medicine Information